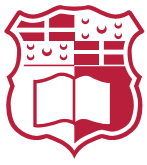
Learning to Select Actions for Complex Hand Tasks: A Comparative Study of Visuomotor Policies and Latent World Models

Graham Pellegrini

Supervisor: Prof. Alexiei Dingli

[PLACEHOLDER: Submission Date]

Submitted in partial fulfilment of the requirements for the degree of Master of Science in Artificial Intelligence.



L-Università ta' Malta
Faculty of Information & Communication Technology

Plagiarism Declaration

I, the undersigned, hereby declare that the work contained in this dissertation is my own original work and that I have not previously in its entirety or in part submitted it at any university for a degree.

I declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person, except where due acknowledgement is made in the text.

I declare that the use of artificial intelligence tools in the preparation of this dissertation was limited to writing assistance, code suggestions, and literature search support. All research design, experimental decisions, implementation choices, result interpretation, and academic claims are my own.

Signature: _____

Name: Graham Pellegrini

Date: _____

Abstract

This dissertation focuses on latent world models, a recent research area that has been gaining traction within the wider field of machine learning. A latent world model learns to predict how a scene will evolve in a compact learned representation rather than in raw pixels, and its promise for robotics lies in judging a proposed action by its predicted consequences before that action is executed. The guiding idea behind this work casts the imitation policy as an actor and the world model as the brain that evaluates the actor's proposed motions, so that the pair can select better and safer actions leading to more plausible end states.

The research question is whether this pairing improves the selection of future bimanual hand motions under a controlled offline benchmark. The study deliberately favours quality of evidence over quantity of tests. The benchmark evaluates 7,607 held-out windows from the EgoDex egocentric manipulation dataset, where each window pairs 16 observed frames with a 16-frame prediction target and actions are represented as 18-dimensional bimanual wrist-motion chunks. Policy and world model are interfaced through three selection branches: the policy alone, world-model scoring with the policy's preferred candidate retained, and world-model scoring with that candidate excluded. All models are trained under a matched fair-training budget, and one pairing emerged as the clean, fully controlled positive result. Diffusion Policy candidates scored by V-JEPA2-AC reduce first-step translation error by 6.41% and rotation error by 1.01 degrees relative to the policy alone. This improvement arrives without any retraining, since the world model simply re-ranks the five motion proposals the policy already produces at inference time. The same constraint sets a natural ceiling on what any selector could achieve: a perfect selector with access to ground truth would reduce translation error by at most 16.9 per cent on the same candidate pool, so the world model recovers roughly 38 per cent of the attainable headroom. Every other implemented route (ACT, DexWM, LeWM, DINO-WM, and JEPA-WM) returns a negative or bounded outcome, and each is followed far enough to trace its shortfall to a concrete caveat such as candidate collapse or an unavailable pretrained checkpoint.

Beyond the headline numbers, the dissertation contributes a reproducible protocol for separating genuine world-model guidance from misleading shortcuts, together with a documented account of the process that exposes further areas of future research. All conclusions concern offline wrist-trajectory prediction on recorded demonstrations.

Acknowledgements

Placeholder for acknowledgements.

Contents

List of Figures

List of Tables

List of Abbreviations

ACT Action Chunking Transformer.

ALOHA A Low-cost Open-source Hardware System for Bimanual Teleoperation.

BC Behaviour Cloning.

CDiT Conditional Diffusion Transformer.

CEMP Cross-Entropy Method with Policy.

CNN Convolutional Neural Network.

CVAE Conditional Variational Autoencoder.

DDIM Denoising Diffusion Implicit Models.

DDPM Denoising Diffusion Probabilistic Models.

DexWM Dexterous Manipulation World Model.

DiT Diffusion Transformer.

DP Diffusion Policy.

HDF5 Hierarchical Data Format 5.

I-JEPA Image Joint Embedding Predictive Architecture.

JEPA Joint Embedding Predictive Architecture.

LeWM LeWorldModel.

RMSE Root Mean Square Error.

SSL Self-Supervised Learning.

V-JEPA2 Video Joint Embedding Predictive Architecture 2.

V-JEPA2-AC Action-Conditioned Video Joint Embedding Predictive Architecture 2.

VAE Variational Autoencoder.

ViT Vision Transformer.

VLA Vision Language Action Model.

VLM Vision Language Model.

List of Notations

I Identity matrix; gives the isotropic unit Gaussian covariance.

$\mathcal{N}(0, I)$ Standard isotropic Gaussian prior.

ε Reparameterisation noise, $\varepsilon \sim \mathcal{N}(0, I)$.

μ, σ Encoder-predicted mean and standard deviation.

z CVAE latent variable sampled from the prior $\mathcal{N}(0, I)$.

$\bar{\alpha}_t$ Cumulative noise-schedule product at diffusion timestep t.

ε_θ Noise-prediction network parameterised by θ.

t_{prev} Next step index in a subsampled inference schedule.

$\hat{x}_0$ Predicted clean sample in a DDIM reverse step.

x_T Noise sample at the diffusion schedule endpoint T.

B_{global} Effective global batch size; equals $B \times G \times A$ where B is the per-device batch size, G the gradient-accumulation steps, and A the number of accelerator devices.

E Effective training exposure, measured as total training items consumed, equal to optimisation steps multiplied by effective global batch size.

K Number of sampled candidate action chunks scored for one observation window.

N Number of evaluated samples or windows, depending on the metric context; pairwise comparisons report the number of held-out windows or candidate pairs explicitly.

S Number of optimisation steps.

$\mathbf{a}_{t:t+15}$ Sixteen-step future action chunk predicted from sixteen observed frames; each step has 18 action dimensions.

a_t Action vector at timestep t; in this thesis each wrist-action step is an 18-dimensional bimanual vector, with left and right hands each represented by 3 translation and 6 rotation coordinates.

z_t Latent embedding of observation frame or video state at timestep t; dimensionality depends on the encoder family.

1 Introduction

Machine learning offers two broad routes to acquiring behaviour in robotics. Reinforcement learning improves a policy through trial and error against a reward signal, while imitation learning acquires behaviour directly from demonstrations [1, 2]. This dissertation works primarily in the imitation setting, which is attractive for manipulation because useful behaviour often depends on subtle motion patterns that are difficult to encode by hand, and demonstrations capture those patterns without requiring a hand-designed reward. Behaviour cloning provides one direct route by learning a mapping from observations to demonstrated actions, while inverse reinforcement learning approaches the same problem by trying to infer the objectives that make a demonstration sensible [2, 3]. Both perspectives frame learning as extraction from observed behaviour, but neither resolves how a system should judge whether a predicted future motion is physically plausible. Reward-driven adaptation returns later in the dissertation, when closely related work that fine-tunes policies inside learned world models is examined.

This issue becomes sharper in dexterous manipulation. Human hand motion is temporally structured, object dependent, and sensitive to small changes in pose. This follows from the timing of natural prehension, the visuomotor transformation required for grasping objects, and the postural synergies used to shape the hand for tools and objects [4–6]. In imitation settings, this matters because small prediction errors can compound across a future sequence, so local agreement with a demonstration does not guarantee sequence-level alignment [7]. This thesis is therefore concerned with prediction and evaluation of future hand motion, rather than deployed control. The work remains relevant to robotic control because robotic manipulation systems ultimately require reliable motion selection, but the evidence in this dissertation is restricted to an offline computational benchmark.

Visuomotor policies are a common way to connect visual observations with future motion. This dissertation studies two policy families that span the main design axes of current practice: the ACT, which regresses temporally extended action chunks in a single deterministic pass [8], and DP, which samples action chunks from a learned generative process [9]. Both share the chunked formulation, which suits hand motion because it unfolds over short sequences rather than through single-frame decisions. However, a policy that proposes future chunks still requires a criterion for choosing between candidate futures, and this is the opening for world models. World models learn predictive representations of how a state may evolve, so they can judge a candidate motion by the future it implies. In this benchmark the world model never generates motion of its own. Every candidate in the pool is proposed by the policy, and

the world model acts only at the moment of selection. The comparison then varies how much weight the policy's own preference carries. One branch keeps the policy's preferred candidate available while the world model re-ranks the pool, and another removes that preferred candidate so the world model must rank the remaining proposals without any bias toward the policy's choice. This graded transfer of the final decision from policy to world model motivates the comparative structure of the thesis.

This dissertation studies that structure through a comparative benchmark framework for evaluating whether world-model support changes the quality of future hand-motion prediction. The work began as a broad comparison across several world-model architectures, and during implementation that plan exposed a second research problem: published world-model systems are not equally reproducible, action-compatible, or fair to compare under one offline benchmark. The study therefore goes beyond training models and reporting scores. Closely related systems are reproduced and examined, each candidate route is instrumented with diagnostic side experiments, and every shortfall is traced to a concrete cause such as an unavailable pretrained checkpoint or an incompatible action space. The final contribution is a controlled benchmark with one clean positive pairing, several boundary cases, and a fully documented account of what world-model adaptation requires in practice before it can support headline claims.

1.1 Motivation

The motivation for this work sits at the intersection of robotics and machine learning. Modern manipulation research increasingly depends on learned models that can process visual observations, predict future motion, and adapt to complex physical interaction. At the same time, the pace of open-source machine learning development can make it difficult to separate robust scientific progress from promising but weakly benchmarked demonstrations. This concern is especially important for industrially relevant settings, where unreliable claims about physical reasoning or manipulation capability can lead to poor engineering decisions.

The direction of the work follows a wider debate in the field. LeCun has argued that systems which generate actions without predicting the consequences of those actions cannot plan or act reliably. He directs this critique at the VLAs that currently dominate robot learning, describing the scaling of autoregressive architectures for physical intelligence as a dead end and agentic systems built on them as "intrinsically unsafe" [10, 11]. His proposed alternative places a learned world model at the centre of the agent, so that candidate actions are evaluated through their predicted future states before any action is taken. That position gained a concrete instance in DiWA,

which fine-tunes a diffusion policy entirely inside a frozen world model's imagination using reinforcement learning [12]. Reading DiWA strengthened the motivation and sharpened the question: DiWA uses the world model as a training substrate that updates the policy, so a complementary study could ask whether a frozen world model can instead improve the action selected at inference time, without retraining the policy. That question came to form this work.

The work was also shaped by the RSO I activity and the University of Malta M.AI.ESTRO project, where the convergence of robotics, machine learning, and Industry 4.0 raised a practical need for clearer evaluation of emerging model families. This dissertation does not claim to deliver a deployment system. It uses that practical motivation to ask a narrower benchmark question about offline candidate selection on recorded hand-motion demonstrations.

1.2 Research Question and Scope

The central research question is:

> *Do latent world models improve the selection of future bimanual hand-motion chunks when they act as the final decision stage over candidates proposed by frozen visuomotor policies (ACT and DP), under a controlled offline benchmark?*

The phrase *final decision stage* marks the boundary of the claim. The world model is not integrated into the policy and never updates its weights, in contrast to adaptation methods such as DiWA where the world model serves as a training substrate for the policy itself [12]. Here every candidate motion is produced by a frozen policy, and the world model only chooses among those candidates.

This wording is a rescoping of the original proposal question. The proposal asked whether integrating a latent world model into an imitation learning policy would improve physical grounding and robustness, measured through latent agreement error and task success under perturbations. The final implementation does not evaluate deployed task success or perturbation robustness, and latent mean-squared error is used only as a training diagnostic for the world-model transition head rather than as the main thesis result. The finished thesis therefore frames the question around offline prediction quality: whether world model support selects future hand-motion chunks that are closer to demonstrated motion under a shared benchmark.

The benchmark focuses on future bimanual hand motion and transform state. In implementation terms, the main prediction surface is built around future left-hand and right-hand transform chunks, but the detailed representation is defined later in Chapter 4 and Chapter 5.

The scope is deliberately bounded. The thesis does not test closed-loop robotic deployment, real-world task completion, force interaction, contact stability, or general machine common sense. It also does not claim that any one world model family is universally superior. Instead, it asks whether selected world-model families improve an offline prediction benchmark when compared under a shared evaluation surface. The final evidence focuses on DP candidates scored by V-JEPA2-AC as the clean positive pairing. ACT, DexWM, LeWM, DINO-WM, JEPA-WM, PointWorld proxy controls, and a secondary DiWA/CALVIN reproduction and guided-selection study are used to define the boundaries around that result.

This scope also explains the structure of the comparison. The benchmark is organised around three branches:

- **Policy-only branch:** the baseline, where the policy's preferred candidate provides the selected future motion without world-model involvement.
- **Hybrid branch:** the policy proposes a pool of candidate chunks, and the world model re-ranks the pool while the policy's preferred candidate remains available as a fallback.
- **World-model-driven branch:** the policy's preferred candidate is excluded from selection, so the world model ranks the remaining policy-generated proposals without any bias toward the policy's own choice.

In every branch the candidate pool itself comes from the policy, and the branches differ only in how much authority the world model holds over the final choice. The purpose of this structure is to separate the value of the world model from the value of the underlying policy as far as possible within an offline benchmark.

1.3 Contributions

This thesis contributes a comparative benchmark framework for studying world-model support in bimanual hand-motion prediction. The framework fixes the policy candidates, separates training support from held-out test windows, and records which information a selector is allowed to consume. This provides a controlled basis for asking whether predictive future-state models improve selected hand-motion chunks under the same offline evaluation conditions.

The thesis also contributes an implemented evidence hierarchy. V-JEPA2-AC is the main positive semantic world-model scorer. LeWM action-prior scoring is a secondary action-regularity result. DexWM, DINO-WM, JEPA-WM, ACT, and the

PointWorld proxy expose limitations involving checkpoint availability, pretraining scale, action-space compatibility, candidate diversity, and evaluation fairness.

Finally, the thesis contributes a reproducibility account. Several model routes were implemented far enough to identify why they could not support headline claims, and these negative and mixed outcomes are reported as part of the benchmark contribution because they show what must be true before world-model guidance becomes valid scientific evidence.

1.4 Dissertation Structure

——To be updated at the end ————

The remainder of the dissertation is organised as follows:

- Chapter 2 introduces the technical background required to follow the dissertation, covering transformers, self-supervised learning, variational autoencoders, rigid body pose, visuomotor policies, and world models at a foundational level.
- Chapter 3 positions the work within the relevant literature on imitation learning, visuomotor policies, world models, and offline evaluation.
- Chapter 4 defines the benchmark design, scope, branch structure, leakage controls, metrics, and reporting rules.
- Chapter 5 describes the implemented system, the model-specific routes, and the reproduction constraints encountered during the study.
- Chapter 6 presents the benchmark results.
- Chapter 7 interprets the findings and states the main limitations.
- Chapter 8 closes the dissertation and identifies future work.

2 Background

This chapter assumes familiarity with the basic mechanics of a neural network and introduces the concepts central to this work: the main learning paradigms, visuomotor policies, rigid-body pose in $\mathrm{SE}(3)$, latent representations, conditional VAEs, transformer attention, and self-supervised video prediction. These foundations support understanding of the specific architectures in Chapter 3 and underpin the candidate selection mechanism and world model evaluator design examined in Chapter 4. Together, they explain the thesis question: whether a latent world model, acting as the final decision stage, can improve the selection of bimanual action chunks proposed by frozen visuomotor policies before any action is executed.

2.1 Learning paradigms

Machine learning methods can be grouped by how they receive feedback during training, as illustrated in Figure 2.1. Supervised learning trains on data with explicit labels: the correct answer is provided for every input, and the model learns to reproduce it. Unsupervised learning receives no labels at all and instead recovers structure, such as clusters or compact descriptions, from the data alone. Reinforcement learning replaces labels with a reward signal: the model acts in an environment and learns which state-action sequences lead to good outcomes.

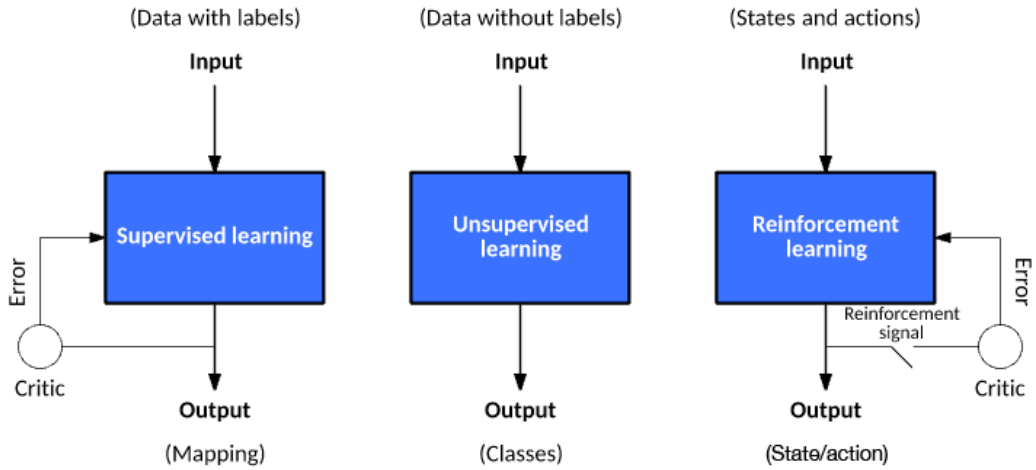


Figure 2.1 The three classical machine learning paradigms: supervised learning trains on labelled data, unsupervised learning extracts structure from unlabelled data, and reinforcement learning learns from a reward signal over states and actions. SSL, central to the world models in this work, is a modern form of learning from unlabelled data in which prediction targets are constructed from the data itself [13].

Reinforcement learning is defined by the interaction loop between an agent and an environment. At each step the agent observes a state, selects an action, and

receives a scalar reward, and the objective is a policy that maximises the expected cumulative reward over time [1]. Learning therefore proceeds by trial and error rather than by copying labelled examples. This allows an agent to improve beyond the behaviour it started with, but it requires an environment, real or simulated, in which the consequences of actions can be observed. The requirement matters later in the dissertation: the offline benchmark studied here provides no such environment, while closely related work discussed in Section 3.2.1 fine-tunes a policy with reinforcement learning inside a learned world model instead of the real world.

SSL sits between the supervised and unsupervised settings. It trains on unlabelled data by constructing prediction tasks from the data itself: part of the input is hidden, and the model learns to predict it from the remainder [14]. Behaviour Cloning (BC), introduced in the next section, is a direct example of the supervised setting in which the expert demonstration acts as the label, while the world models studied in this dissertation are trained in the self-supervised setting. The close of this chapter develops that idea for video.

2.2 Visuomotor policies

A visuomotor policy is a learned function that takes a visual observation as input and produces an action or sequence of future actions as output. In a manipulation setting, the visual observation is typically a single camera frame showing the hand and the object being manipulated, and the action describes how the hand should move next. The policy is called visuomotor because it connects vision directly to motor output, without requiring a hand-crafted intermediate representation of what the scene contains [2].

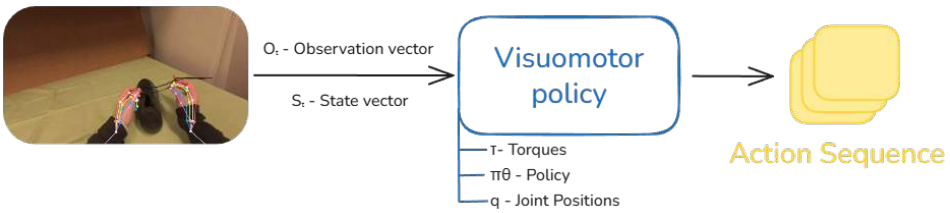


Figure 2.2 A visuomotor policy maps an EgoDex-style camera frame to future bimanual wrist actions in the thesis setting.

Learning this mapping from data is appealing because useful hand behaviour depends on subtle visual cues that are difficult to specify by hand. The timing of a reaching movement, the grip shape adopted before contact, and the postural adjustments made in response to object geometry all depend on information visible in the camera image but difficult to encode as explicit rules [4–6]. A visuomotor policy

learns these associations directly from recorded demonstrations, capturing the relationship between what the camera sees and what the hand does without requiring the programmer to describe it explicitly.

2.2.1 Behaviour cloning and compounding error

The most direct way to train a visuomotor policy from demonstrations is BC [2]. Here an observed frame predicts the action taken by the expert at that moment, and the policy is trained to minimise the difference between its predicted action and the recorded expert action across all frames in the dataset [7]. The problem arises when a small prediction error causes the hand to move to a state that is slightly different from any state in the training data. This results in a compounding error: as the policy encounters increasingly unfamiliar states, its prediction errors grow larger over time.

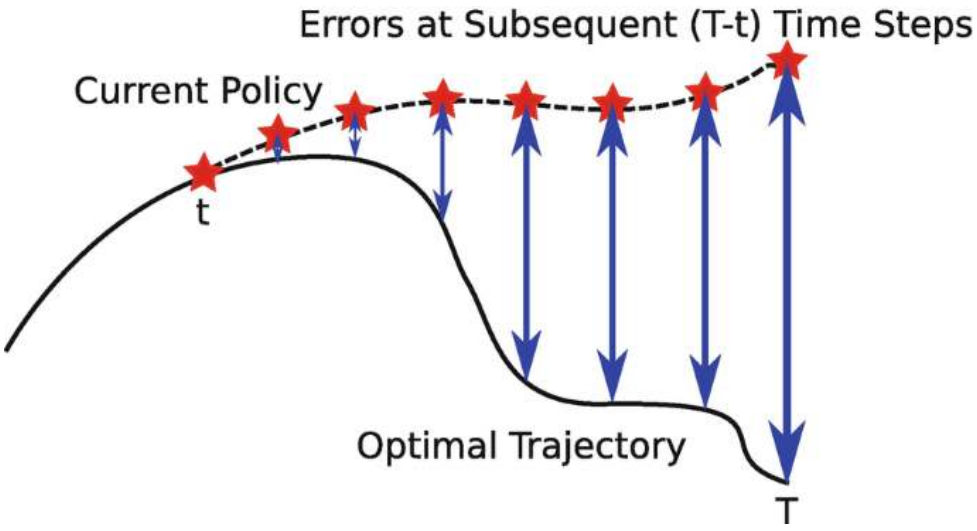


Figure 2.3 Compounding error in behaviour cloning. The policy trajectory drifts progressively further away at each step. The growing gap between the two paths illustrates how small errors can accumulate over time [15].

2.2.2 Action chunking

Action chunking addresses the compounding error problem by changing what the policy predicts. Instead of predicting a single next action at each step, the policy predicts a short sequence of future actions in one forward pass. This sequence is called an action chunk [8, 9]. In this benchmark, the policy observes 16 frames and predicts a 16-frame future action chunk. Each predicted step is an 18-dimensional vector: two hands, each with a six-dimensional rotation representation and a three-dimensional translation. This is why Figure 2.2 shows the action sequence as a stack of multiple action blocks. By committing to a chunk of k future steps, where k is a hyperparameter

set per task, the policy makes one prediction decision every k frames rather than one per frame. With fewer decision points, there are fewer opportunities for errors to accumulate, and the hand stays closer to the training distribution for longer [12].

2.3 Rigid body pose and the $\mathrm{SE}(3)$ representation

To predict how a hand will move, a system needs a precise way to describe where the hand is and which way it is pointing. These two quantities together are called the pose of a rigid body. For the human wrist, pose is fully described by its position in space and its orientation relative to a fixed reference frame [16].

Translation. Position is represented as a vector of three numbers, one for each spatial dimension: how far the wrist is to the right or left, how far forward or backward, and how far up or down [4]. This vector $\mathbf{t} = [t_x,\ t_y,\ t_z]^\top$ is straightforward to predict with no special constraints on the output.

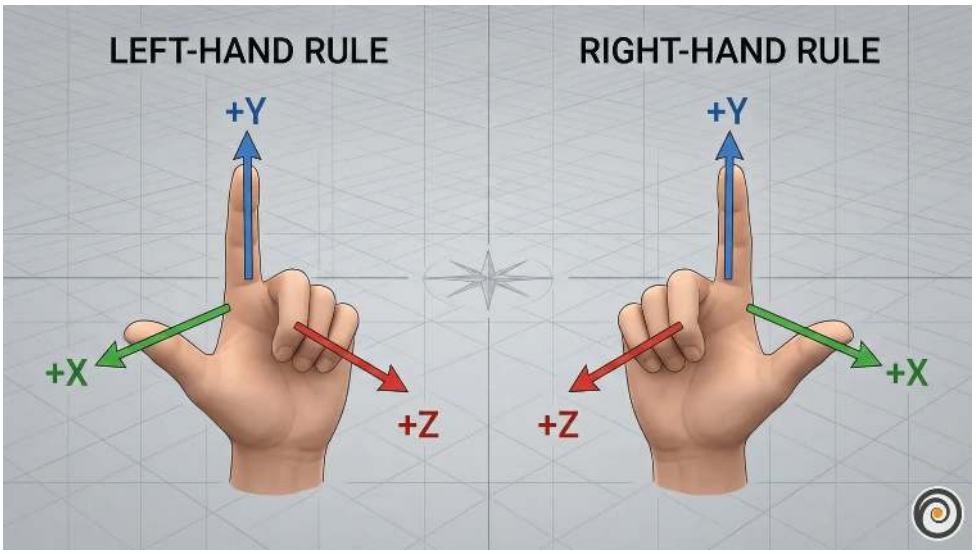


Figure 2.4 Translation for each arm can be represented by a movement along each of the three spatial axis [17].

Rotation. Orientation requires more care. A common representation uses three angles known as Euler angles, one per axis. These are intuitive but problematic for learning. At certain configurations the axes align and the system loses a degree of freedom, a singularity called gimbal lock. A subtler problem is discontinuity: a wrist rotated 359 degrees and one rotated 1 degree are almost physically identical, yet their angle values are numerically far apart. A network minimising prediction error will produce large errors near these boundaries even when the true motion is smooth [18].

A rotation matrix R avoids both issues: small physical rotations produce small numerical changes in the matrix entries, and there are no singularities.

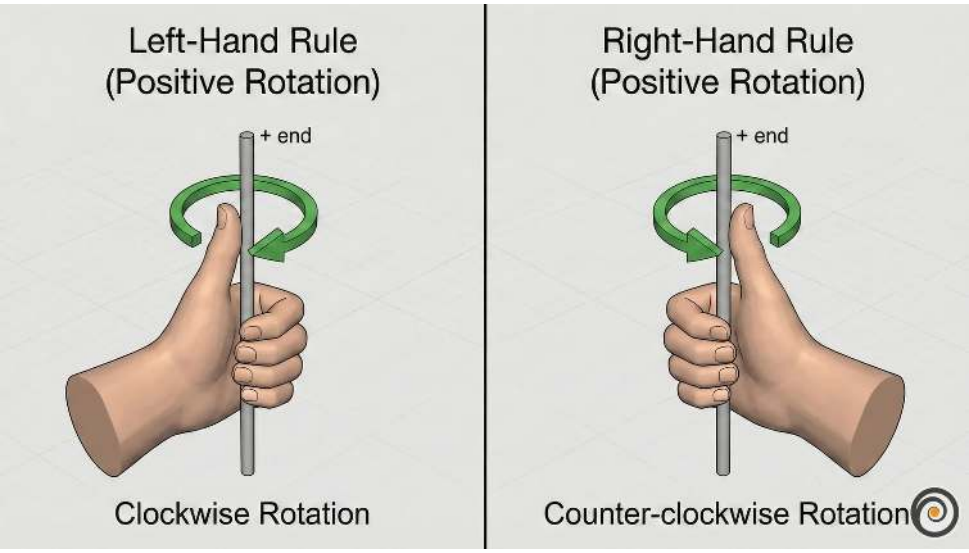


Figure 2.5 Rotation around the y-axis for each arm. The same circular movement can be imagined around the x and z axes; the total orientation of the wrist is the combined result of rotations around all three [17].

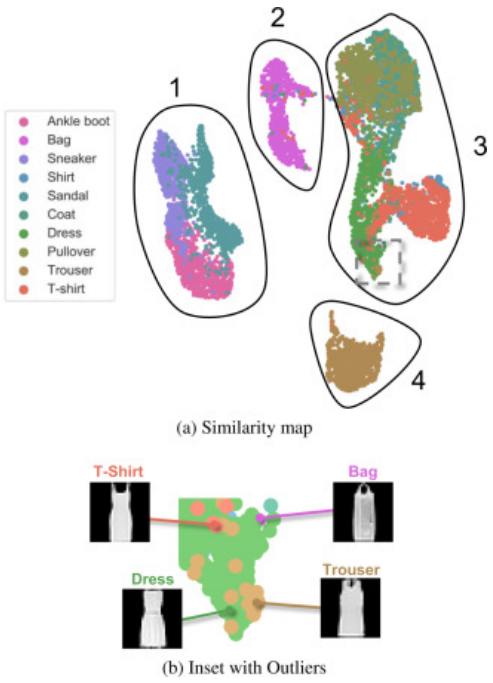
Homogeneous transform. Translation and rotation are combined into a single 4×4 matrix, the standard representation of a pose in the special Euclidean group $\mathrm{SE}(3)$:

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{2.1}$$

The benchmark does not predict the full human body. It predicts left- and right-wrist motion. For each future step, the benchmark stores one rotation and one translation per wrist, giving the 18-dimensional bimanual action vector used by the policy and world-model scorers.

2.4 Latent representations

A latent representation is a learned internal description of an observation. An image starts as a grid of pixel values, where each pixel stores three or four channel intensities (typically red, green, and blue, with an optional transparency value). A model does not usually reason with these raw numbers directly. Instead, it transforms them through several layers into a new set of values that no longer correspond to individual pixel coordinates or colours. These new values form a compact encoding that captures patterns in the image such as shape, position, motion, and spatial structure. The collection of these encodings is often called the latent space. Figure 2.6 shows an example in which similar items cluster together in the learned representation.



(a) Similarity map

(b) Inset with Outliers

Figure 2.6 Example latent-space plot showing how structurally similar items cluster together in the learned representation. Structural similarity here refers to items that share shape, geometry, and likely physical behaviour [19].

The purpose of this transformation is to keep task-relevant structure while discarding unnecessary visual detail. For a manipulation scene, a useful latent state might encode hand position, object pose, contact progression, or motion trend without preserving every texture or lighting variation. A latent state is therefore not directly interpretable in the way an image is, but it can still preserve the information needed to reason about what happens next.

This is why latent prediction is often a better fit than pixel-level prediction for high-dimensional video observations. Predicting the next frame at the pixel level requires a model to account for every visual detail of the scene, including lighting changes, texture variation, and background movement that carry no information about task-relevant dynamics. A latent predictor instead operates on compact embeddings and concentrates predictive capacity on the structure that determines what actually happens next [20]. This design trade-off motivates the world model architectures reviewed in Chapter 3.

2.5 Conditional variational autoencoders

A VAE maps an input to a distribution of latent vectors described by a mean μ and standard deviation σ, rather than a single point. Sampling different latent vectors lets the decoder produce different but plausible outputs rather than one fixed answer. Figure 2.7a shows this standard encoder-latent-decoder flow [21].

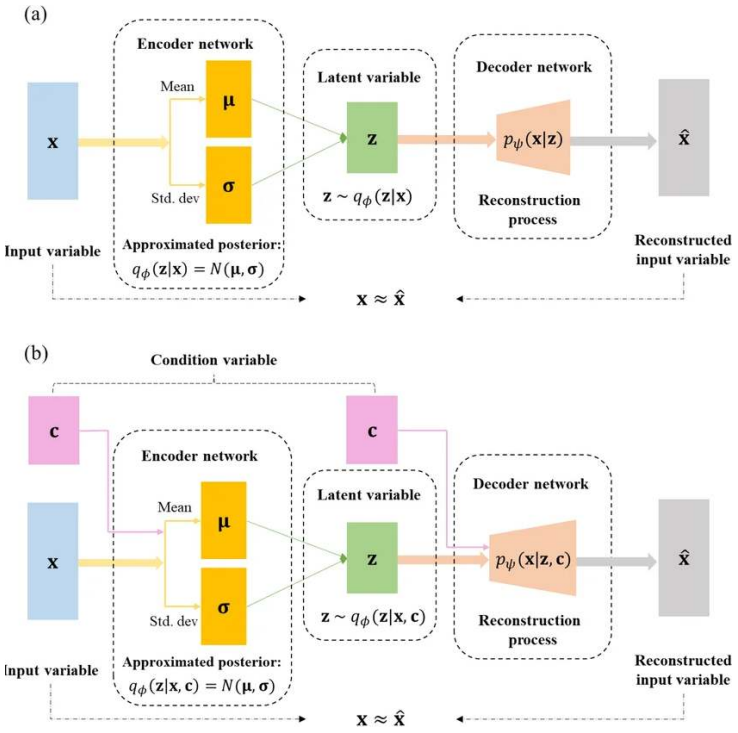


Figure 2.7 Standard VAE (a) and conditional CVAE (b) encoder-decoder pipelines [22].

For the policy to be trainable end-to-end, gradients must flow from the action prediction loss all the way back to the encoder. Sampling z directly from a distribution blocks this path. The reparameterisation trick resolves this by writing $z = \mu + \sigma\epsilon$, where ϵ is standard normal noise drawn independently of the learned parameters. Figure 2.8 shows the full encoder-latent-decoder pipeline alongside its training loss: a reconstruction term that rewards accurate output and a regularisation term that keeps the latent distribution close to a standard Gaussian, together forming the evidence lower bound [21]:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z}))\,. \tag{2.2}$$

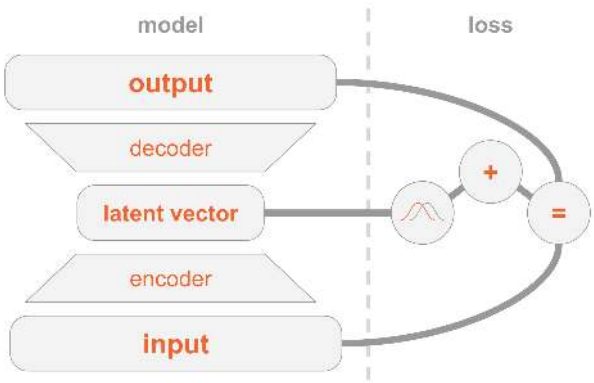


Figure 2.8 VAE encoder-decoder architecture and two-part training loss (reconstruction and regularisation) [21].

A CVAE adds an extra conditioning input to both encoder and decoder, shown in Figure 2.7b. In a visuomotor policy, this condition is the current observation frame, so the latent variable only needs to capture how the future action varies within that scene [22]. The ACT architecture uses this to sample multiple latent vectors from one observation and decode each into a candidate action chunk [8].

2.6 Transformer networks and the attention mechanism

A transformer processes a sequence as a set of tokens. A token can represent a word, an image patch, a timestep in an action sequence, or any vector produced by an earlier model component. The architectural idea needed here is simple: each token is updated by looking across the other tokens and deciding which of them are most relevant. This relevance-weighted exchange of information is called attention [23].

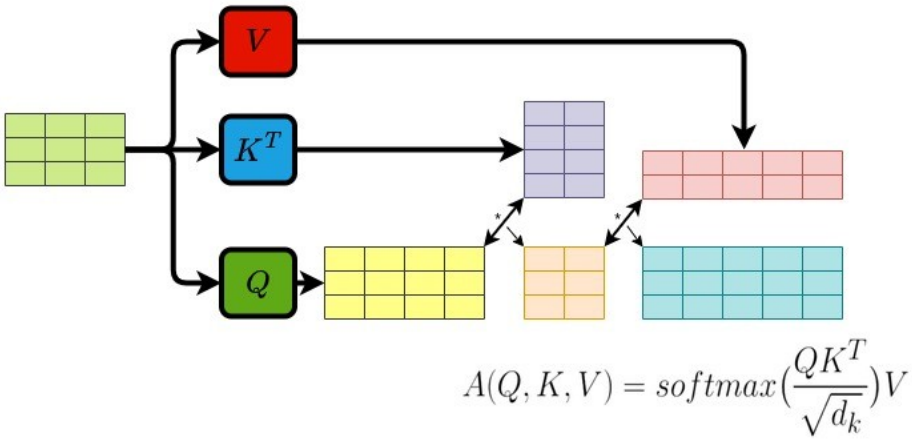


Figure 2.9 Transformer attention block: each token queries all others with Q, gathers weighted values V via keys K, and the weighted sum yields the updated output token [23].

Attention forms three learned views of every token:

- **Query** (Q) – what this token is looking for.
- **Key** (K) – what another token contains.
- **Value** (V) – what another token contributes if attended to.

Figure 2.9 shows how each token (highlighted) sends its query outward to every other token, receives weighted values in return, and produces an updated output. The bidirectional arrows indicate that every token attends to every other token simultaneously, and the crossing arrow beneath represents the weighted sum that combines those values into the final output. For each token, the transformer compares

its query with the keys of all tokens. The resulting scores become weights, and those weights are used to combine the corresponding values. This operation is usually written as [23]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{2.3}$$

Transformers usually repeat this operation in several heads and add positional information so the model knows where each token sits in the sequence. Multiple heads allow different relationships to be represented at the same time, while positional information preserves order [23].

These concepts meet in self-supervised video prediction, the training signal behind the world models in this work. Given a sequence of observed frames, one or more future frames are hidden and the model learns to predict them from the frames that came before; the recording itself provides the target, so no annotation is needed. Predicting raw pixels runs into the ambiguity problem described in Section 2.4, where the model averages over possible futures and produces a blurred image. Predicting in latent space avoids this: the model predicts the compact embedding of the future frame, discarding background motion and fine texture so that capacity is spent on the structure that determines what happens next [20].

When the predictor is also given the action the agent intends to take, the result is a world model: a learned function that predicts how the scene embedding will change in response to a proposed action. If a policy proposes five wrist-motion chunks, a latent world model can predict the future embedding of each and favour the candidate whose predicted future is most plausible under its learned dynamics. This ties the chapter together: visuomotor policies produce action chunks, $\mathrm{SE}(3)$ describes the wrist motion being predicted, latent representations provide the scoring space, and self-supervised prediction supplies the world-model training signal. The architectures built on these foundations are reviewed in Chapter 3.

3 Literature Review

This chapter reviews the literature that leads directly to the central research question set out in Section 1.2: whether a world-model evaluator can improve action-chunk selection for bimanual EgoDex wrist-action prediction, and whether any such improvement holds once the underlying policy family changes. The reviewed work therefore has two roles: first, to justify ACT and Diffusion Policy as the candidate policy families; and second, to motivate latent, action-conditioned world models as candidate selectors. Whether an offline benchmark of this kind can be trusted at all is a question the surveyed literature also bears on directly, and it is picked up again once results are presented.

The review is also selective about evidential role. Some papers motivate the final positive test, especially Diffusion Policy and V-JEPA2-AC; others explain why negative or exploratory routes are still scientifically useful, especially DexWM, LeWM, DINO-WM, and JEPA-WM. DiWA is treated as the closest adjacent policy-improvement study because it uses a world model to improve Diffusion Policy performance, but its training-time adaptation setting differs from this thesis's inference-time candidate selection setting. Table 3.1 summarises how each literature group maps to the benchmark design.

The chapter assumes familiarity with the technical foundations in Chapter 2; world models are introduced here.

Table 3.1 Literature roles in the benchmark design.

Work	Relevant strength	Relevance and limitation for this thesis
ACT [8]	Efficient bimanual action chunking with a structured latent variable.	Provides the transformer policy baseline, but its sampled candidates collapsed in the final implementation, limiting its use as a selector stress test.
Diffusion Policy [9]	Strong multimodal visuomotor policy with diverse sampled action chunks.	Main policy family for the headline result, with higher inference cost than ACT.
V-JEPA2 and V-JEPA2-AC [24]	Internet-scale video representation and action-conditioned latent prediction.	Main positive world-model evaluator; the action-conditioned head gives the most direct match to candidate scoring.
DexWM [25]	Dexterous manipulation world model with hand-consistency supervision.	Important comparator, but no public pretrained checkpoint was available and full upstream-scale pretraining fell outside the fair local budget.
LeWM [26]	Lightweight end-to-end latent dynamics model.	Useful for action-prior evidence, but the visual rollout route did not support the main semantic-selection claim.
DINO-WM and JEPA-WM [27]	Official checkpoint route for image/JEP-style latent prediction.	Exploratory official-checkpoint comparators; action-interface mismatch required an adapter and results remain appendix-level.
DiWA [12]	Uses imagined experience to improve Diffusion Policy.	Closest related study; it adapts the policy at training time rather than selecting candidates at test time. Its released policies are examined in a secondary reproduction and guided-selection study.
SimplerEnv and WorldEval [28, 29]	Evidence that offline/simulated rankings can correlate with real-world outcomes.	Supports the offline-evaluation premise, while requiring careful claims about calibration and deployment validity.

3.1 Visuomotor policies

Two policy families dominate current imitation learning for manipulation, and they attack the same pair of failure modes, compounding error and multimodal demonstrations, from opposite representational directions. This section reviews each in turn and then examines what the comparative literature actually establishes about their relative strengths, which is less than is commonly assumed.

3.1.1 Action Chunking with Transformers (ACT)

ACT was introduced by Zhao et al. to address the compounding error and multimodal distribution challenges that make bimanual manipulation demanding for imitation learning, achieving success rates of up to 90% on fine-grained tasks with the A Low-cost Open-source Hardware System for Bimanual Teleoperation (ALOHA) platform [8]. In $\mathrm{SE}(3)$ wrist-pose prediction specifically, small deviations in predicted position or orientation alter contact geometry in ways that cascade into grasp failure [2], and a policy trained with a naive regression loss produces averaged predictions that may not represent any plausible trajectory when the demonstrated distribution is multimodal [9]. ACT's CVAE architecture addresses both of these failure modes directly.

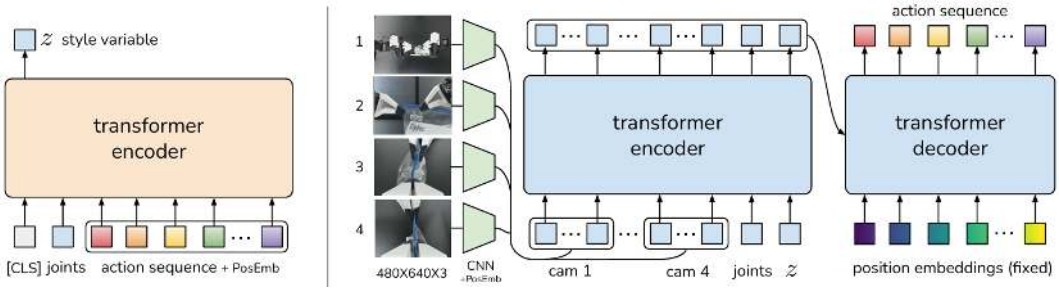


Figure 3.1 Architecture of the Action Chunking with Transformers (ACT) policy [8].

ACT does so through the CVAE design of Section 2.5, depicted in Figure 3.1. A CVAE encoder compresses the demonstrated action sequence and joint observation into a style variable z that captures the multimodal distribution of plausible futures, and a transformer decoder predicts the future action chunk from the visual observations, joint positions, and z. The encoder is discarded at test time. In the original formulation, z is set to the mean of the prior (zero) to produce a single deterministic action chunk per observation; when a pool of candidate chunks is required, multiple values of z are sampled independently from the prior and each is decoded into a separate candidate, all conditioned on the same observation [8]. At

inference, Zhao et al. additionally apply temporal ensembling, averaging overlapping chunk predictions with recency weights to smooth the executed motion [8].

ACT carries known limitations alongside these strengths. The Gaussian prior on the style variable z constrains how faithfully the CVAE can represent highly multimodal action distributions: at ambiguous transitions the latent space can average across modes rather than sample cleanly from each one [9]. A second limitation follows from the architecture's reliance on the decoder to make use of z. The decoder conditions on both the observations and the latent variable, and nothing in the training objective forces it to draw on z when the observations alone suffice to reconstruct the demonstrated actions. If the decoder learns to ignore the latent variable, sampling different values of z no longer changes the predicted chunk and the candidate pool degenerates into copies of a single output; this failure mode is known in the VAE literature as posterior collapse [30].

The VAE literature also offers mitigations, including annealing the Kullback-Leibler weight during training [30] and reserving a minimum information budget for the latent through free bits [31], yet the standard ACT recipe trains with a fixed Kullback-Leibler weight and adopts neither. Zhao et al.'s own ablation sharpens the picture of when the latent matters: removing the CVAE objective makes almost no difference on scripted, deterministic demonstrations, but on human demonstrations success collapses from 35.3% to 2% [8]. The latent variable is therefore load-bearing exactly when the data is multimodal, which is also the regime in which its training is most fragile. Beyond the latent, chunk length is a fixed hyperparameter that must be tuned per task, and temporal ensembling introduces a recency-weighted lag that can slow the policy's response to sudden environmental changes [8].

3.1.2 Diffusion Policy

Diffusion Policy approaches the same compounding-error and multimodality challenges from the opposite representational direction. Instead of a structured latent variable, it starts each prediction from Gaussian noise and iteratively denoises it into an action chunk, conditioned on the recent observations [9]. Because every denoising pass explores the full action distribution rather than committing to a single latent mode, sampled chunks capture multimodal demonstrations without mode averaging, and independent samples from the same observation form a naturally diverse candidate pool. Figure 3.2 shows the general formulation and its two published heads, a CNN variant with FiLM observation conditioning and a transformer variant with causal cross-attention; their architectural details are given in the original papers [9, 32, 33]. The transformer variant is the form used in this benchmark.

Chi et al. benchmark Diffusion Policy across twelve tasks from four robot

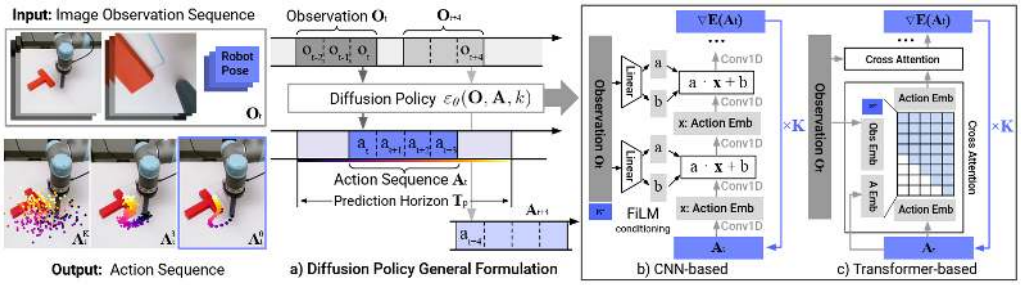


Figure 3.2 Diffusion Policy: general formulation (a), CNN variant (b), transformer variant (c) [9].

manipulation benchmarks and report an average improvement of 46.9% over the prior state of the art, including implicit and mixture-density policy baselines [9]. The advantage is attributed to iterative denoising exploring the full action distribution at each step rather than committing to a single latent mode as a CVAE prior must. Notably, ACT is not among the compared methods: the two papers appeared within weeks of each other and neither evaluates the other, a point taken up below.

The main limitation of Diffusion Policy is inference cost. Generating a single action chunk requires multiple sequential network evaluations: standard Denoising Diffusion Probabilistic Models (DDPM) variants use on the order of 100 denoising steps compared with the single forward pass of ACT's decoder, which substantially reduces the achievable control frequency [9]. The scale of the follow-up effort is itself evidence of how seriously the field treats this defect. Efficient transformer variants recover speed through progressive noise scheduling and parameter sharing [33], Consistency Policy distils the denoising chain into a one- or few-step generator [34], and the π_0 generalist policy abandons discrete-step diffusion for flow matching to reach real-time control rates [35]. Iterative denoising in its original form is thus increasingly treated as a starting point to be distilled away rather than a final design.

In practice, the two families suit different pressures. ACT's single decoder pass makes it the cheaper option and well suited to structured, repeatable bimanual tasks, but the CVAE latent it depends on for diversity is exactly what degrades on highly multimodal human demonstrations, as Zhao et al.'s own ablation shows [8]. Diffusion Policy pays for its iterative denoising with inference cost, but that same process is what lets it explore multimodal action distributions without collapsing to an average [9]. Both are used extensively for hand and bimanual manipulation, and the strongest signal that this trade-off is real rather than incidental is that the ACT authors' own follow-up, ALOHA Unleashed, replaces the CVAE with a diffusion head once the tasks become deformable-object and contact-rich enough [36]. What the main literature surveyed here does not offer, however, is a direct comparison of the two on shared bimanual data under a matched budget: Diffusion Policy's twelve-task study predates ACT and

does not include it, ACT is benchmarked only against BC-ConvMLP, BeT, RT-1, and VINN, and later multi-policy studies such as Lee et al.'s behaviour-generation comparison include Diffusion Policy but still omit ACT [8, 9, 37].

3.2 World models

A world model is a learned predictor of how a system evolves over time. Given the current state and a proposed action, it estimates the state that is likely to follow. In a manipulation setting, the state may be a raw image, a latent representation of the scene (Section 2.4), or a structured description of hand and object configuration. Regardless of the representation chosen, the underlying principle is the same: the model internalises the dynamics of the environment and uses them to project forward from any given state [38].

The case for world models is itself contested, and the disagreement has deep architectural roots, traced from a common ancestor in Figure 3.3. The generative camp treats scale as the path to competence: VLAs such as RT-2, OpenVLA, and π_0 couple web-scale vision-language backbones to autoregressive or flow-based action decoding, and their authors report broad generalisation to novel objects and instructions as evidence that this route is working [35, 39, 40]. LeCun has argued the opposite: architectures that generate actions without predicting their consequences cannot plan reliably, and scaling autoregressive models toward physical intelligence is a dead end [10, 11]. The joint-embedding branch, and the latent world models built on it, is the constructive form of that critique. This dissertation does not adjudicate the debate; it tests one specific claim from the world-model side, that predicted latent futures carry enough signal to rank candidate actions.

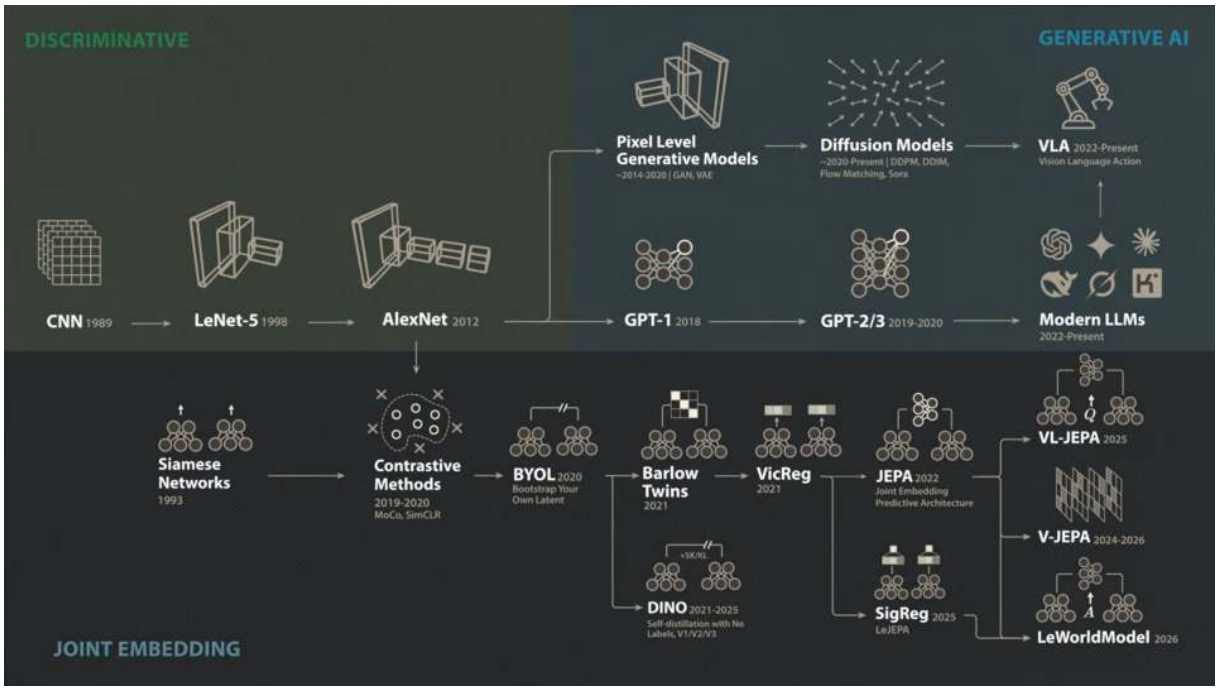


Figure 3.3 The post-AlexNet split of visual representation learning: a generative branch (top), culminating in diffusion models and the VLAs debated above, and a joint-embedding branch (bottom), culminating in the JEPA-style world models evaluated in this benchmark. Screenshot from [41].

World models become especially useful when a pool of candidate actions must be evaluated before any one is committed to. Each candidate is rolled forward through the model, producing a predicted future state; those predictions are ranked by predicted cost or consistency. The quality of this ranking depends heavily on the chosen representation: pixel-space world models must reconstruct every visual detail and suffer from mode-averaging at multimodal transitions, while models that operate in the compact learned latent space introduced in Section 2.4 can concentrate predictive capacity on the dynamics that matter for task outcomes [20]. The following subsections survey how world models have been used to improve policies, the literature supporting the evaluator design, and the architectures implemented in the benchmark.

3.2.1 World models as policy refiners

The most direct way to use a world model to improve a policy is to adapt the policy inside it. DiWA develops this idea on CALVIN, a long-horizon manipulation benchmark where a Franka arm operates a tabletop drawer, sliding door, lightbulb, LED, and coloured blocks, shipped with hours of unstructured teleoperated play alongside expert demonstrations [42]. DiWA treats the world model as a training substrate: a diffusion policy is fine-tuned with reinforcement learning (Section 2.1) entirely inside a frozen world model's imagination, following the recipe in Figure 3.4. That world model

is a DreamerV2-style recurrent state-space model trained on roughly six hours of task-agnostic play, about 500,000 transitions, with reward supplied by a binary success classifier trained on latent embeddings of the expert demonstrations. The denoising chain of the diffusion policy is framed as a Markov decision process, letting proximal policy optimisation run on imagined rollouts while a behaviour-cloning regulariser keeps the policy close to its pretrained initialisation [12].

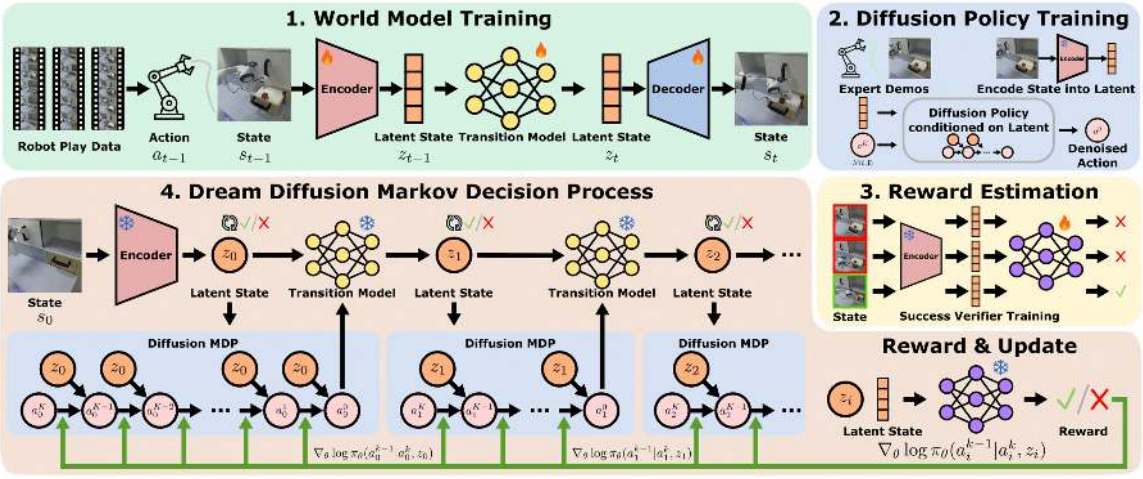


Figure 3.4 The three-stage DiWA recipe: a world model is trained on task-agnostic play data, a diffusion policy is pretrained by imitation in its latent space, and the policy is then fine-tuned with reinforcement learning on rollouts imagined entirely inside the frozen world model [12].

Across all eight CALVIN skills tested, this purely imagined fine-tuning raises task success substantially: baseline success ranges from 35.63% to 62.55%, and DiWA fine-tuning lifts every skill above 74%, up to 91.95% on close-drawer, averaged over three seeds [12]. A model-free baseline that fine-tunes the same policy in the real environment needs hundreds of thousands to millions of physical steps to reach comparable performance, where DiWA uses none. The authors also report the first fine-tuning of diffusion policies for real-world robotic skills through an offline world model, improving three physical manipulation skills after imagined fine-tuning on a world model trained from roughly four hours of teleoperated play [12]. They are explicit about the limits of this evidence: because the world model is frozen, its errors persist through fine-tuning and can be exploited by the policy, and gains measured in imagination do not always survive execution on the physical robot [12].

3.2.2 World models as policy evaluators

WorldGym demonstrates that a learned world model used as a simulated environment for policy evaluation produces rankings that align with real-world task success with meaningful fidelity [43]. Candidate policies are evaluated not by deploying them

physically but by rolling them out inside the model's predicted environment, reducing the cost of policy comparison to a forward pass.

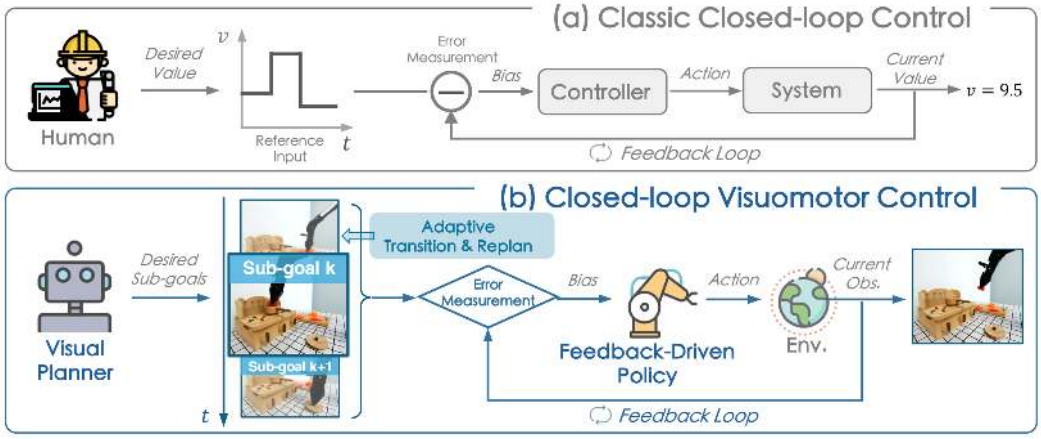


Figure 3.5 Classic closed-loop control (a) versus CLOVER's closed-loop visuomotor control (b) [44].

CLOVER extends this to closed-loop control, as shown in Figure 3.5. A visual planner generates a sequence of sub-goal frames; the executor measures the embedding-space error between the predicted sub-goal and the current observation. When the error exceeds a threshold, the system triggers an adaptive transition and replans rather than committing to the original trajectory. The feedback loop, absent from open-loop imitation policies, is the mechanism that translates world model prediction accuracy into task completion gain [44]. Together, WorldGym and CLOVER establish that world model rankings are informative both before and during execution.

The practical value of this evaluator loop is measurable. Table 3.2 reproduces CLOVER's real-world evaluation on a multi-step robot manipulation suite, reporting the average number of consecutive sub-tasks completed (`avg_len`) for representative baselines and CLOVER [44].

Table 3.2 Average consecutive sub-task completions (`avg_len`) on a real-world multi-step manipulation suite. Higher is better. CLOVER's closed-loop feedback via world model prediction nearly doubles the completion depth of the strongest open-loop baseline. Data from [44].

Method	avg_len
ACT [8]	0.6
R3M (pretrained visual repr.) [45]	0.7
RT-1 [46]	1.1
CLOVER (closed-loop)	**2.1**

Alongside the task completion gain, CLOVER also reduces embedding-space prediction Root Mean Square Error (RMSE) relative to open-loop baselines, suggesting

prediction accuracy and task success move together under the evaluator framing. This correlation is not guaranteed by construction. However, Lambert et al. document an objective mismatch at the heart of model-based reinforcement learning, since world models are trained to minimise prediction error, yet low prediction error does not by itself produce good control, and the correlation between the two can be weak [47]. A world model's rankings therefore inherit no guarantee from its training objective, and whether they are informative, including whether they transfer to physical outcomes and remain reliable out of distribution, must be established empirically in each setting, a question revisited when results are presented.

3.2.3 Latent predictive architectures

The energy-based framework of LeCun and Dawid establishes why prediction should operate in latent rather than pixel space [20]. The same representational argument introduced in Section 2.4 applies directly to the prediction target, favouring compact embeddings over pixel reconstructions that force the model to account for irrelevant visual detail and average over all plausible continuations at ambiguous transitions.

The pixel-space route is nonetheless pursued at scale, and its supporters reach the opposite conclusion. GAIA-1 predicts future driving video directly [48], and Genie learns action-controllable interactive environments from unlabelled internet video [49], treating faithful visual generation itself as evidence that a useful world model has been learned. The joint-embedding position is that this fidelity is bought with capacity spent on task-irrelevant detail. The architectures reviewed below are built on that counter-position: they are latent predictors, and this benchmark evaluates them on whether their predictions rank candidate actions well, not on whether they can generate a convincing video.

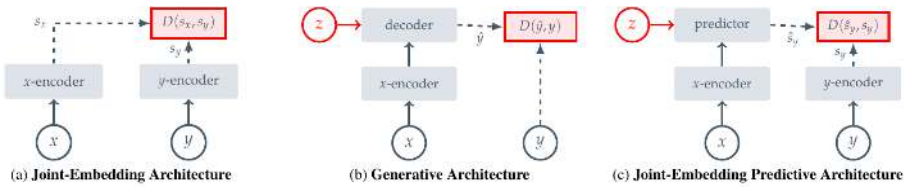


Figure 3.6 Three SSL prediction paradigms: joint-embedding (a), generative (b), and joint-embedding predictive (c) [14].

Image Joint Embedding Predictive Architecture (I-JEPA) establishes the joint-embedding predictive architecture as a viable approach to learning rich visual representations without any pixel reconstruction objective [14]. As Figure 3.3 traces, this predictive objective is the direct ancestor of most world models surveyed in this section. Panel (c) of Figure 3.6 shows the data flow directly: a context block x from the

image is passed through the context encoder to produce latent tokens z_x. A lightweight ViT predictor then takes z_x together with positional tokens indicating where the masked target regions y are located, and outputs predicted latent representations $\hat{s}_y$ for those regions. A separate target encoder, whose weights are a slow exponential moving average of the context encoder, independently encodes the actual target pixels into ground-truth latents s_y. The loss MSE$(\hat{s}_y, s_y)$ is computed entirely in latent space: no decoder, no pixel generation, no reconstruction of lighting or texture. The outcome is a context encoder whose representations capture semantically meaningful structure (object shape, spatial layout, and scene dynamics) because that is all the predictor ever needs to get the target embeddings right. Training is approximately 2.5 times faster than pixel-reconstruction methods as a result. V-JEPA2 extends this same joint-embedding predictive architecture from images to video sequences. The following subsections cover the fundamental literature of the architectures explored throughout this benchmark.

V-JEPA2

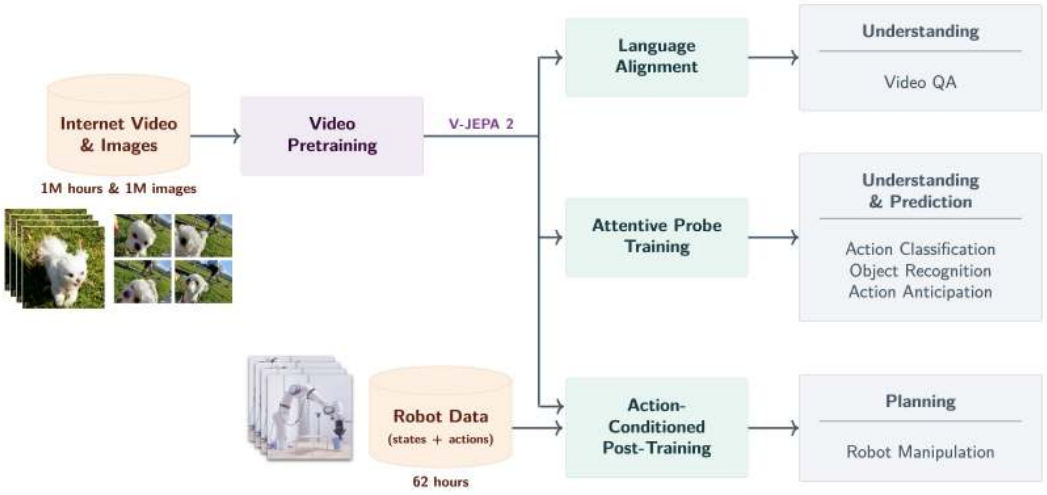


Figure 3.7 V-JEPA2 post-training pathways from internet-scale backbone to specialised applications [24].

V-JEPA2 extends the I-JEPA framework from static images to video, pretraining on over one million hours of internet video [24]. The pretraining shifts from spatial to spatio-temporal masking: the context encoder receives past observation frames, and the predictor must recover future latent states. Training combines two regimes: one where the predictor is always shown the true preceding frames at each step, and a second, multi-step rollout regime, where it instead conditions on its own earlier predictions. The former keeps short-horizon predictions accurate, and the latter keeps long-horizon trajectory representations consistent once prediction errors are allowed

to compound. The result is a frozen ViT-Large encoder, approximately 300 million parameters, that encodes physical priors about hand and object interactions acquired from internet-scale pretraining. As shown in Figure 3.7, the pretrained backbone supports three post-training pathways: Language Alignment, which grounds visual embeddings in natural language; Attentive Probe, which adds a lightweight classification head; and Action Conditioning, which extends the predictor to take robot actions as input and forms the basis of V-JEPA2-AC.

V-JEPA2-AC

The frozen ViT-Large encoder from V-JEPA2 serves as the backbone of V-JEPA2-AC, producing a fixed latent embedding for each observation frame with no gradient updates to the backbone weights. A lightweight action-conditioned transition head is trained on top of these stored embeddings. It takes the current frame latent together with a proposed action chunk (robot joint states and end-effector poses) as the conditioning signal z, and predicts the latent representation of the future frame. The head is trained with an L1 objective (the mean absolute difference between predicted and actual embeddings) against the embedding of the actual future frame. At planning time, the original work scores candidate actions by the distance between their predicted future embeddings and the embedding of a given goal image [24]. This design, shown on the right of Figure 3.8, separates what the network must learn into two parts. The frozen encoder supplies rich visual features that generalise from internet-scale pretraining, and the small trained head learns only how actions propagate through those features.

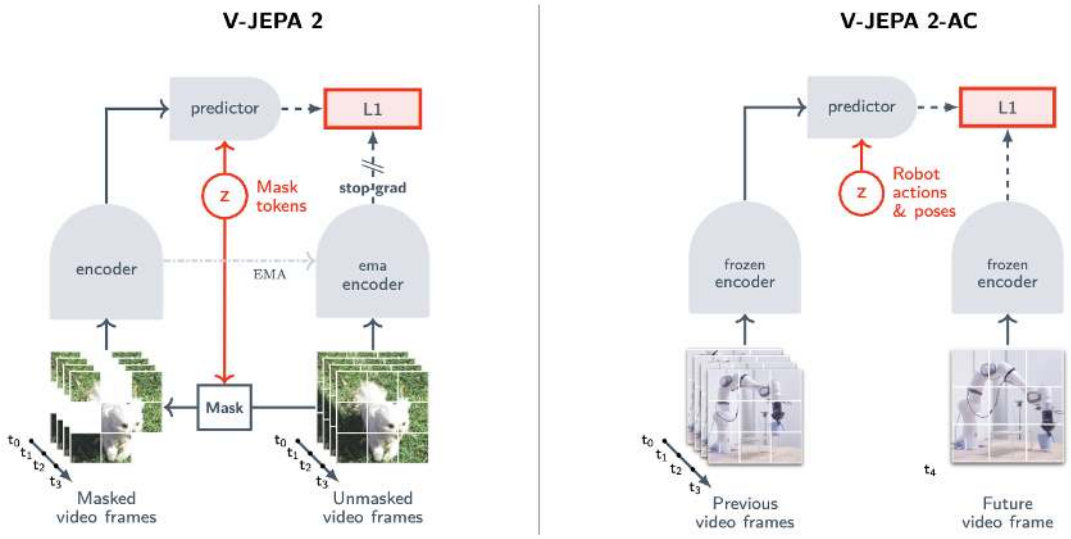


Figure 3.8 Standard V-JEPA2 (left) versus action-conditioned V-JEPA2-AC (right) [24].

An important qualification is raised by Tsagkas et al., who show that pretrained

visual representations can encode spatial plausibility well while providing a weaker signal for fine-grained temporal dynamics [50]. A frozen encoder trained on internet video may represent hand shape and position accurately at each individual frame while providing a less reliable signal about whether the predicted trajectory is kinematically consistent across the full chunk horizon. This temporal limitation provides context for the path shape results in Chapter 7.

DexWM

DexWM [25] is a dexterous manipulation world model from Meta FAIR, shown in Figure 3.9. Each input frame is encoded by a frozen DINOv2 image encoder into a grid of spatial patch tokens, a CDiT predictor advances the latent state under action and camera-motion conditioning, and a lightweight decoder maps the result back to image and keypoint space. Its distinguishing feature is the Hand Consistency loss: an auxiliary head predicts fingertip and wrist heatmaps from the latent state, imposing a hand-geometry prior that is absent from both V-JEPA2-AC and LeWM. Goswami et al. report a 50% improvement over Diffusion Policy on grasping and placing tasks and an 83% zero-shot real-world grasping success rate under latent planning [25]. These figures come from a single study with no public checkpoint at the time of writing, and have not been independently reproduced. Where V-JEPA2-AC transfers a video-pretrained backbone, DexWM transfers one pretrained on static images, so the comparison in this benchmark isolates the contribution of video-specific temporal pretraining.

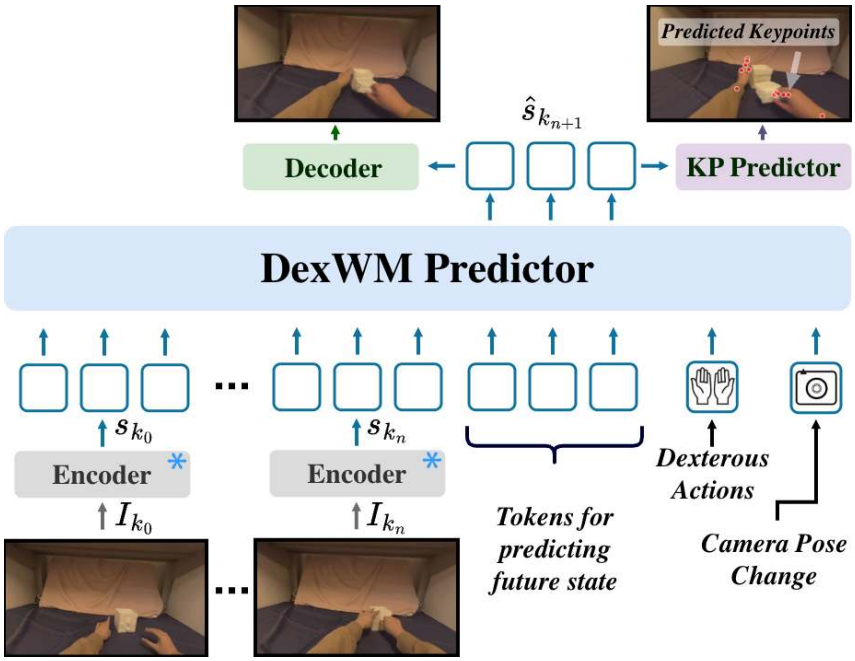


Figure 3.9 High-level layout of the DexWM world model pipeline, illustrating how predicted keypoints are incorporated alongside the encoder, CDiT predictor, and decoder stages [25].

LeWorldModel

LeWM is a latent world model trained end-to-end from random initialisation on raw pixel observations [26]. As shown in Figure 3.10, a ViT-Tiny encoder of roughly 15 million parameters maps each frame to a single 192-dimensional token, and a dynamics predictor estimates the next latent from the current token and the action. Training combines a next-embedding prediction loss with SIGReg, a regulariser that enforces approximate Gaussianity of the latent distribution along random univariate projections. This single regularisation term stabilises end-to-end training without contrastive pairs or a moving-average target encoder, reducing the tunable loss hyperparameters of prior JEPA-style designs from six to one [26].

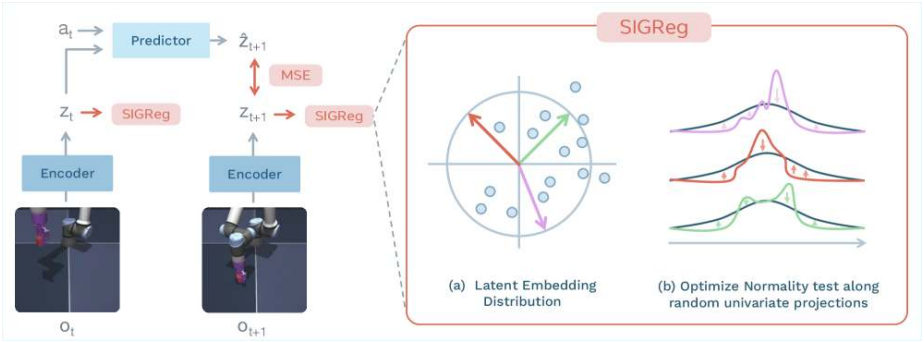


Figure 3.10 LeWM training pipeline: encoder, dynamics predictor, and SIGReg regulariser [26].

The design trades spatial resolution for speed. LeWM trains on a single GPU in a few hours and, at planning time, its compact latent achieves a 48× speedup over models that predict over frozen DINOv2 patch grids, while still detecting physically implausible transitions and tracking agent and object locations across frames [26]. As with DexWM, these results are self-reported by a single group, and the model is recent enough that independent evaluations do not yet exist.

Read together, the three architectures occupy different points on a cost-versus-prior trade-off, though only LeWM's speedup claim is directly benchmarked against a comparable model in its source paper. V-JEPA2-AC carries a heavy but frozen video-pretrained Vision Transformer (ViT)-Large backbone with only a lightweight trained transition head. DexWM is the most resource-intensive of the three: on top of a similarly heavy DINOv2 backbone it adds a second learned CDiT predictor and an auxiliary hand-geometry head, the extra machinery behind its reported Hand Consistency gains. LeWM, unlike DexWM, discards pretraining altogether for a from-scratch ViT-Tiny encoder, which its authors report is far cheaper to train and run, consistent with the 48× planning speedup above. This compact from-scratch route is what the benchmark tests against the pretrained-transfer routes of V-JEPA2-AC and DexWM, a comparison returned to with results in Section 6.1.

DINO-WM and JEPA-WM

Two further latent world models enter the benchmark as exploratory official-checkpoint routes, and each pairs with one of the architectures above. DINO-WM performs latent prediction over a frozen DINOv2 patch grid [27], the same backbone family DexWM builds on, and Figure 3.3 places both on the self-distillation branch rather than the predictive JEPA branch. JEPA-WM denotes the officially released JEPA world-model checkpoints trained on robot manipulation data [51]. Like LeWM, it sits directly on the predictive branch established by I-JEPA above, though as an independently pretrained checkpoint rather than a from-scratch design. Both DINO-WM and JEPA-WM provide pretrained checkpoints, but their expected action interfaces do not match the 18-dimensional bimanual wrist space, so each is evaluated through an adapter and treated as an exploratory comparator paired with its architectural relative. Their architectures and validation status are documented in Appendix A.

PointWorld

PointWorld is a large pretrained 3D world model for in-the-wild rigid arm manipulation [52]. Unlike the latent-space models described above, it operates directly

in 3D geometry. Scene state is lifted from RGB-D images into a full-scene point cloud, and robot actions are represented as the 3D point flows traced by robot surface geometry through forward kinematics. This embodiment-agnostic representation allows the model to reason over physical interaction geometry rather than learned abstract features.

Such a model is not designed to handle an 18-dimensional bimanual wrist-pose action space. Its forward-kinematics representation assumes rigid arm geometry with a discrete gripper aperture, making it structurally mismatched when applied to continuous SE(3) wrist transforms from human hand demonstrations.

4 Methodology

This chapter defines how the central research question (Section 1.2) was translated into a controlled, testable benchmark. Answering whether a frozen world model can improve candidate selection, and whether that holds across policy families, requires isolating the selection mechanism from every other source of variance. Section 4.1 sets out the dataset and action space; Section 4.2 the three branch definitions; Section 4.3 the policy and candidate-generation routes; Section 4.4 the evidence roles assigned to each world-model family; Section 4.5 the fairness and leakage rules that separate held-out evidence from exploratory implementation work; and Sections 4.6 and 4.7 the evaluation metrics and scope limitations.

4.1 Dataset and Experimental Setup

4.1.1 EgoDex: source and selection rationale

The benchmark is built on EgoDex [53], a large-scale egocentric dataset of bimanual dexterous manipulation collected using Meta Quest Pro head-mounted cameras. It spans more than 100 task categories across its full release, including folding, slotting, pouring, and kneading, covering a wide range of fine-grained hand motions. The full release comprises five training parts and supplementary additional-data splits; this study uses the first three training parts (Section 4.1.3). Each episode provides synchronised RGB video and full six-degree-of-freedom $\mathrm{SE}(3)$ wrist annotations for both hands at approximately 30 frames per second.

The choice of dexterous manipulation as the evaluation domain is not incidental. This dissertation forms part of the University of Malta's ongoing M.AI.ESTRO research project [54], which investigates the use of AI for scene and task prediction on dexterous manipulation tasks to support industrial transfer learning and human skill digitisation. M.AI.ESTRO's immediate pilot targets human hand pose prediction and guidance from egocentric demonstration; a second, later pilot addresses robotic hand manipulation, and the intended pathway between them translates poses learned from human demonstration into low-level actuator commands for the robot controller, rather than having the system generate robotic actions directly. The research domain for this dissertation was therefore set by the scope of the first pilot. The introduction situates this motivation more broadly, and the present chapter focuses on the methodological consequences of the domain choice.

Within the dexterous manipulation setting, EgoDex was selected on three properties:

- The benchmark required a fine-grained bimanual hand pose action space, provided through an eighteen-dimensional bimanual wrist pose representation that large-scale corpora built around rigid grippers or arm contact do not provide.
- The first-person perspective aligns with the intended downstream deployment setting of a wearable assistive device (a VR guidance application).
- The corpus is large enough to support simultaneous training of five world model families and two policy architectures on declared non-overlapping splits, while retaining a held-out test set of meaningful size.

EgoDex's demonstrations are human rather than robotic, and it carries no contact or force signal, only kinematic SE(3) pose. Neither is a limitation as scoped here; both are discussed further in Section 4.7.

4.1.2 Windowing protocol

The dataset is stored as paired `.mp4` and `.hdf5` episode files. A deterministic index stage discovers all valid paired episodes, infers their lengths from HDF5 metadata, and writes canonical observation and prediction windows for each split. This shared index is the single source of truth for all downstream modules: policy training, world model extraction, and evaluation all consume the same window file for their respective split, ensuring that output differences across the experimental branches can be attributed to branch design rather than to differences in the data presented to each component. The benchmark uses a fixed-stride windowing scheme as the canonical contract. A supplementary state-change-targeted scheme was identified later, while iterating on this pipeline, and was not scaled into the canonical benchmark; a small comparison against the canonical scheme was run to check whether it would have been worth adopting, and is summarised at the end of this section and detailed in Appendix B.11.

Strategy 1: Fixed-stride sampling. Each window spans 16 observation frames followed by 16 prediction frames, with consecutive window starts separated by a stride of 128 frames. At 30 frames per second this places starts approximately 4.3 seconds apart, corresponding to an observation context and prediction horizon of approximately 0.53 seconds each. This contract is encoded as `obs16_pred16_s128` and forms the foundation for all policy training and world model training (see Figure 4.1).

Figure 4.1 Fixed-stride windowing scheme (`obs16_pred16_s128`) used as the canonical training and evaluation index. Author-generated schematic.

Strategy 2: State-change-targeted sampling. The fixed-stride scheme samples the episode timeline uniformly, which causes it to under-sample high-motion events: long near-static segments contribute windows at the same rate as active manipulation periods. As detailed in Chapter 5, this limitation motivated a revised strategy adopted for a subsidiary perceptual inspection task and qualitative development review. Rather than placing windows at fixed intervals, this approach computes smoothed hand-state change scores from the wrist transform and confidence sequences, selects candidate windows near high-change peaks, and applies episode-first task-stratified sampling with a fixed random seed (see Figure 4.2). A relative-position fallback retains slower episodes that would otherwise be excluded, yielding one frozen window per episode stratified across all represented task categories.

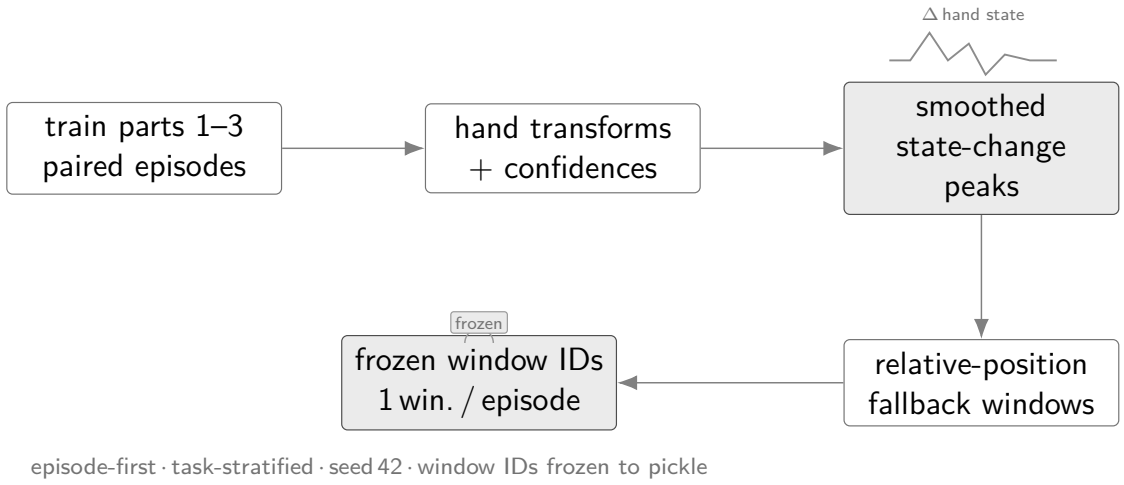


Figure 4.2 State-change-targeted sampling pipeline yielding one frozen window per episode. Author-generated schematic.

The fixed-stride scheme (Strategy 1) is used as the canonical protocol across all headline experimental branches in this study. Strategy 2 was not adopted for the canonical benchmark; instead, a small standalone comparison, its own state-change-trained checkpoint, support extract, and test index, was run to check whether it would have been worth adopting. The result was mixed: state-change selection improved one world model's candidate capture and worsened another's, so it does not justify replacing Strategy 1 as the study's canonical protocol. Strategy 2 remains a promising direction for future work rather than a demonstrated improvement. The full comparison, method, and per-model results are reported in Appendix B.11.

4.1.3 Training splits and test set

Three cumulative training splits were assembled from the first three parts of the EgoDex release, each a strict superset of the previous; Table 4.1 summarises their composition. This three-part subset is used by design rather than the full five-part release: these first three parts were seen to give the most substantial gains in performance across model families, and training was not extended further given the fixed training-exposure budget available on the shared cluster GPUs. How performance scaled from one split to the next is detailed in Appendix B.12. How the three splits relate to each other during training is addressed in Section 4.5.1. The held-out test split is the official EgoDex test set and is never used during training at any stage.

Table 4.1 EgoDex splits under the `obs16_pred16_s128` windowing contract. Episode and window counts are derived from the canonical index.

Split	Tasks	Episodes	Windows	Role
`train_part1`	26	46,091	140,239	Stage 1 training
`train_part1_2`	39	140,654	291,369	Stage 1+2 training
`train_part1_2_3`	63	194,333	430,962	**Canonical training**
`test`	111	3,229	7,607	**Frozen evaluation**

The canonical results in Chapter 6 use `train_part1_2_3`. The test split spans 111 task categories, including all 63 present in `train_part1_2_3` plus 48 drawn from the excluded parts; this exposes the benchmark to some task-level distribution shift, though it is not a controlled generalisation test, since the excluded categories were never held out as an experimental variable. Its window index is frozen throughout.

4.2 Three-Branch Experimental Framework

Answering the central research question (Section 1.2) cleanly requires holding every other experimental factor constant. The benchmark therefore organises evaluation around three branches that share the same policy weights and the same evaluation windows; only the candidate pool composition and selection rule differ across branches (Figure 4.3), so any measured performance difference between them reflects the selection mechanism.

The candidate pool size is fixed at $K = 5$ throughout this study. No literature precedent dictates this figure specifically, so it is treated as a compute-budget variable rather than one derived from theory: a common-random-number sweep over $K \in \{2, 3, 4, 5, 6, 8, 10, 12, 16, 20\}$, detailed in Appendix B.13, confirms that the ranking of world-model scorers is identical at every pool size tested, so no comparative

conclusion drawn in this work is an artefact of fixing K at five. The limitation is discussed further in Section 4.7.

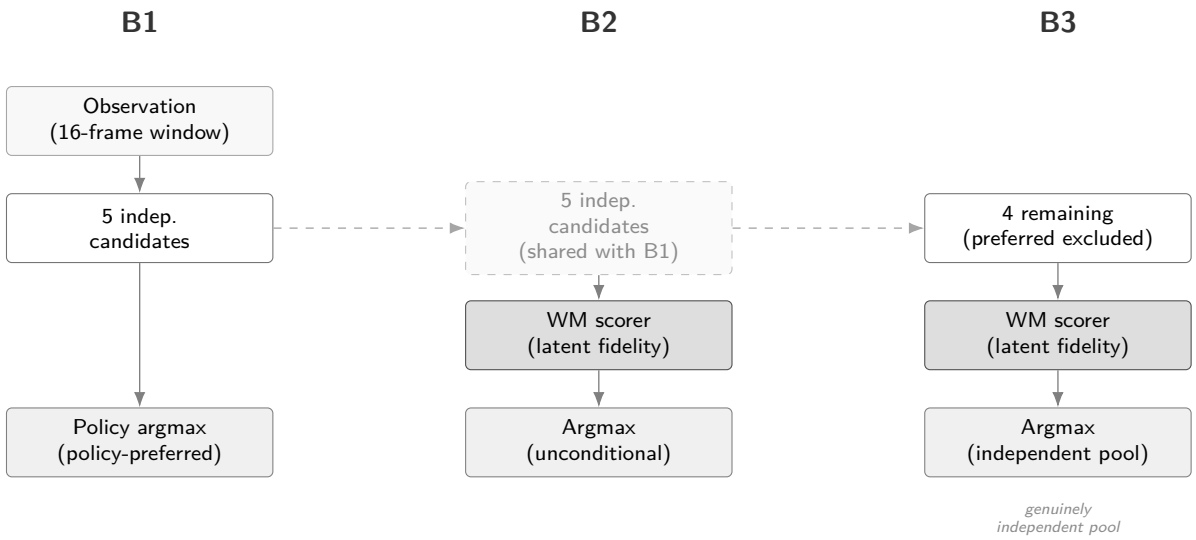


Figure 4.3 Three-branch evaluation framework (Section 4.2), shown for a policy producing five independent candidates. B2 and B3 reuse the same candidate pool generated for B1, relayed through the branches (dashed lines) rather than regenerated; B3 additionally removes the policy's preferred candidate before scoring. Both ACT and DP follow this design; the per-policy candidate-generation mechanics are detailed in Section 4.3.

Branch 1 (B1): Policy-only baseline. The policy generates its full candidate pool and selects by its own internal preference; no world model is queried. The ideal design produces several structurally independent candidates so the downstream branches can compare meaningfully different trajectories. B1 establishes the trajectory prediction quality each policy achieves acting alone and is the baseline against which B2 and B3 are measured.

Branch 2 (B2): World-model-inclusive selection. The world model scores all candidates in the pool and selects the highest-scoring one unconditionally; no margin threshold is applied. The candidate pool is the same as B1. Scoring proceeds by predicting forward in the world model's latent space from the current observation and measuring prediction fidelity against each candidate action. The policy's own output is not privileged; it is selected only if the world model assigns it the highest score. For V-JEPA2 without action conditioning, the transition head required to produce candidate-specific latent predictions is absent; if the model assigns indistinguishable scores to all candidates, B2 degenerates to B1, directly testing whether action conditioning is the operative mechanism.

Branch 3 (B3): World-model-exclusive selection. The policy's preferred output is excluded from the evaluation pool; the world model selects the highest-scoring

remaining candidate unconditionally. The ideal design ensures the remaining candidates are structurally independent so that exclusion meaningfully changes the selection outcome. B3 tests whether the world model maintains selection quality when the policy's top choice is unavailable. All five world model families apply this protocol.

The three branches select by argmax at three different points in the pipeline, and the distinction matters: B1's *policy-preferred* argmax is the policy's own internal choice, made without reference to any external score; B2's *unconditional* argmax is the world model's choice over that same pool; and B3's *independent-pool* argmax is the world model's choice once the preferred candidate is removed, over the remaining structurally independent candidates only. Both policy families evaluated, ACT and DP (Figure 4.3), follow this design: each generates its own five-candidate pool for B1 and B2 and scores the remaining four for B3, using the different sampling procedures detailed in Section 4.3.

4.3 Policy Variants

The benchmark evaluates two imitation-learning policy families. Their architectures and training objectives are reviewed in Chapter 3; the properties relevant to this study are how each generates its candidate pool and what that implies for world-model scoring.

- **Action Chunking Transformer** (Section 3.1.1). ACT [8] is designed to produce diverse candidates by sampling latent vectors $z \sim \mathcal{N}(0, I)$ from its CVAE prior and decoding each into a 16-step bimanual SE(3) action chunk. Its single-shot decoding makes it the cheaper of the two policies to run at candidate-generation time. The implementation follows the original architecture directly and is described in Chapter 5.
- **Diffusion Policy** (Section 3.1.2). DP [9] generates each candidate through an independent reverse-diffusion chain, using a Denoising Diffusion Implicit Models (DDIM) schedule [55] and the transformer (Diffusion Transformer (DiT)) variant. Its iterative denoising is designed to capture multimodal demonstrations without averaging across modes, at a higher inference cost than ACT. The implementation follows the original formulation directly and is described in Chapter 5.

Evaluating both families under the same three-branch protocol allows the study to ask whether world-model augmentation improves candidate selection in general or only under a particular generation mechanism. A result that holds across both constitutes stronger evidence for the generality of the approach.

4.4 World Model Families

The model families have different evidence roles in the final thesis. They are not treated as equal candidates for the headline claim. A result is promoted to main evidence only when it uses frozen candidates, training-only support, held-out test windows, and a scorer whose action representation is compatible with the candidate pool.

- **V-JEPA2-AC** (Section 3.2.3), the headline scorer. A frozen V-JEPA2 [24] ViT-Large backbone provides a visual embedding, and a lightweight transition head predicts a future embedding from the current embedding and the candidate action chunk; candidates are scored by distance to the reference latent. It combines a large pretrained video representation with explicit action conditioning, the most direct match to the candidate-scoring task.
- **Mechanistic controls**: V-JEPA2 without action conditioning and PointWorld. Both reuse the V-JEPA2-AC visual pathway; the former removes candidate-specific action information, the latter degrades the bimanual action chunk into an incompatible robot-arm-style representation. Together they test whether a positive V-JEPA2-AC result depends on meaningful action conditioning rather than the visual backbone alone.
- **LeWM** (Section 3.2.3) [26], a secondary action-regularity scorer. It is implemented first as a visual rollout and latent-support world model, evaluated against the same causal-validity gates as V-JEPA2-AC before being promoted to semantic evidence. A narrower action-prior variant, using training-only action statistics to prefer candidates closer to the training distribution, is retained as a secondary result: a valid action-regularity signal, distinct from a visual-rollout imagination claim.
- **Dexterous Manipulation World Model (DexWM)** (Section 3.2.3) [25], a fixed-budget negative comparator. It is the closest architecture-level comparator for dexterous world modelling. Public pretrained weights for the upstream EgoDex+DROID recipe are unavailable, so the thesis evaluates a reproducible fixed-budget implementation, treating its result as evidence only for that implementation; full official pretraining is excluded because it needs DROID raw-data onboarding and a training scale outside the fair-budget contract.
- **DINO-WM and JEPA-WM** (Section 3.2.3) [27, 51], exploratory official-checkpoint comparators. Both require an adapter from this study's 18-dimensional bimanual wrist chunks to the 7-dimensional DROID-style action interface the checkpoints expect. Because of this interface mismatch, they are

treated as validation-only routes and reported in the implementation appendix rather than promoted to held-out evidence.

- **DiWA/CALVIN** [12, 42], a secondary reproduction and comparison. DiWA studies world-model-guided Diffusion Policy improvement through imagined training, whereas this study evaluates test-time candidate selection without retraining the policy; the CALVIN study compares the two improvement operators under shared components. The methodology's plan is to first fully reproduce DiWA's published results under its original frozen world model, reward classifier, and policy checkpoints, then compare the unmodified upstream baseline, a fixed or random candidate control, this study's test-time selector, and the paper's own imagined-PPO fine-tuning under identical evaluation episodes. This is a secondary study, kept separate from the primary EgoDex branch definitions rather than a substitute for it. Implementation status is reported in Chapter 5, and results in Chapter 6.

Table 4.2 Evidence role assigned to each implemented or investigated world-model path.

Path	Evidence role	Main claim status
V-JEPA2-AC	Headline semantic scorer	Held-out test claim
V-JEPA2 no-AC	Action-conditioning ablation	Held-out control
PointWorld proxy	Action-degradation control	Held-out mixed control
LeWM visual rollout	Diagnostic scorer	No-go for semantic rollout claim
LeWM action-prior	Training-action regularity scorer	Secondary held-out claim
DexWM fixed-budget	Domain-specific comparator	Held-out negative/null claim
DINO-WM / JEPA-WM	Official-checkpoint exploratory routes	Validation-only, appendix
DiWA/CALVIN	Secondary reproduction and comparison	Reported separately in Chapter 6

4.5 Fair Experimental Protocol

Comparing the five world model families and two policy architectures against each other requires holding every non-design variable constant. Two aspects of fairness are addressed: training exposure and evaluation contract.

4.5.1 Training exposure budget

Different model families have different computational costs per training step and different sensitivities to data volume. Without a principled exposure budget, a comparison could conflate the effect of architecture with the effect of receiving more

or fewer training samples. To determine an appropriate shared budget, a convergence study was conducted prior to full training, training each model family independently across six data scales and examining the resulting loss curves for plateau behaviour. The full study and its per-model results are reported in Appendix B.10; the finding relevant to the experimental design is summarised here.

The effective training exposure E is defined using the following symbols (see the List of Notations):

E	effective training exposure (items consumed)
S	number of optimisation steps
B	per-device batch size
G	gradient-accumulation steps
A	number of accelerator devices
B_{global}	effective global batch size

$$E \;=\; S \times B_{\text{global}} \tag{4.1}$$

where:

$$B_{\text{global}} \;=\; B \times G \times A \tag{4.2}$$

This formulation counts the total training items consumed regardless of how steps are distributed across hardware, allowing models with different batch sizes and GPU counts to be normalised to a common scale.

The convergence study established a universal budget of $E = 100{,}000$ effective items. This figure is applied uniformly across all model families and all three training splits, ensuring that any measured performance difference reflects architecture and data rather than exposure volume. Each split is trained independently from a fresh initialisation at this same budget; later splits do not continue training from an earlier split's checkpoint. Any difference between the `train_part1`, `train_part1_2`, and `train_part1_2_3` results therefore reflects the data available at that scale, not a warm start carried over from a smaller split. The full data-construction and training curriculum is described in Chapter 5.

4.5.2 Evaluation contract

All models receive identical inputs and evaluation conditions. At evaluation time, every family sees the same RGB observation frames from the canonical test window, without additional depth, point cloud, or semantic metadata; PointWorld's distinctive characteristic is its degraded action conditioning rather than a different observation modality. The evaluation windows are fixed at indexing time, shared across all branches and families, and never selected, filtered, or re-sampled after training. Policy and world

model weights are frozen before evaluation and are not updated. The only variable at evaluation time is the selection rule applied to the candidate pool.

The causal validator checks four leakage boundaries before a run can be treated as evidence:

1. Support episodes must not overlap with evaluation episodes.
2. Support windows must not temporally overlap the held-out test window.
3. Task labels may not be used to retrieve test support unless the same restriction is declared for every compared family; the final canonical runs use global retrieval rather than test-task-scoped retrieval.
4. Repeated-subject metadata are preserved when available but not used as a selection feature.

If a model route cannot provide these guarantees, it remains exploratory or appendix material rather than a headline result. Every evaluation run also emits a frozen manifest recording its selection mode, candidate source, and aggregate score statistics, and writes per-window results as it goes so an interrupted run resumes from its last completed window rather than restarting.

4.6 Evaluation Metrics

4.6.1 Offline trajectory prediction setting

EgoDex does not define an official offline evaluation metric. The original benchmark measures policy effectiveness via online task success rate on a physical robot deployment [53], which is outside the scope of this study. The evaluation setting here differs fundamentally: rather than measuring task execution, the study evaluates offline visuomotor trajectory prediction, measuring how closely each configuration's predicted wrist pose sequences align with ground-truth demonstrated trajectories in the held-out test set. Results should therefore be interpreted as trajectory prediction quality under each branch and world model configuration, not as claims about downstream task success. The relationship between offline trajectory prediction quality and online task performance is addressed in Section 4.7 and revisited when results are presented.

4.6.2 Primary metrics

The action representation throughout this study is a 16-step sequence of bimanual wrist actions, each wrist pose the 6-DOF position-plus-orientation representation introduced in Section 2.3. At step t, hand $h \in \{L, R\}$ has predicted translation $\hat{\mathbf{p}}_{t,h}$, target translation $\mathbf{p}_{t,h}$, predicted rotation $\hat{R}_{t,h}$, and target rotation $R_{t,h}$. A prediction can fail in two physically distinct ways. The hand can end up in the wrong place, or it can

end up in the right place while pointing the wrong way. A manipulation policy can fail at either independently of the other. The primary metrics below therefore report translation and rotation error separately, rather than folding them into one combined score.

Translation L2 error is the mean Euclidean wrist-position error over both hands and all prediction steps, following standard $\mathrm{SE}(3)$ pose-error convention [16]:

$$e_{\text{trans}} = \frac{1}{2T} \sum_{t=1}^{T} \sum_{h \in \{L,R\}} \|\hat{\mathbf{p}}_{t,h} - \mathbf{p}_{t,h}\|_2, \quad T = 16. \tag{4.3}$$

It is reported in metres and is the primary spatial fidelity metric. On average, it is how far the predicted wrist position is from where the demonstration actually placed it, over the full 16-step horizon and both hands. This is the more consequential of the two errors for task success, since a hand that never reaches the right point in space cannot grasp or place an object regardless of how it is oriented when it gets there.

Rotation error is the mean geodesic distance on $\mathrm{SO}(3)$ [16], reported in degrees:

$$e_{\text{rot}} = \frac{1}{2T} \sum_{t=1}^{T} \sum_{h \in \{L,R\}} \cos^{-1}\left(\frac{\mathrm{tr}(\hat{R}_{t,h}^{\top} R_{t,h}) - 1}{2} \right) \frac{180}{\pi}. \tag{4.4}$$

This measures, in degrees, how far the predicted wrist orientation is from the demonstrated one on average, using the geodesic (shortest-path) angle between two rotations rather than a per-axis angle difference. Per-axis Euler differences are exactly the representation flagged in Section 2.3 as discontinuous and prone to gimbal lock [18]; the geodesic distance on $\mathrm{SO}(3)$ avoids both problems and gives a single physically meaningful angle regardless of how the underlying rotation is parameterised. A correctly positioned hand with the wrong orientation is a real manipulation failure in its own right; a wrist rotated the wrong way can miss a grip, collide with an object, or approach a handle from an angle that cannot close around it, even while sitting at the exact target position.

Root-mean-square error is used for diagnostics where a squared aggregate is more appropriate than a mean absolute distance, penalising a few large deviations more heavily than many small ones:

$$\mathrm{RMSE} = \sqrt{\frac{1}{M} \sum_{i=1}^{M} (\hat{y}_i - y_i)^2}. \tag{4.5}$$

Here M denotes the number of scalar components being compared.

Endpoint translation uses Equation 4.3 over the final four prediction steps only, capturing how accurately the trajectory arrives at its destination specifically, since

where a reaching motion ends often matters more to task success than where it passed through along the way. Velocity and acceleration diagnostics apply the same paired-error logic after first and second finite differences of the wrist position sequence, capturing whether the predicted motion moves at a demonstration-consistent speed and smoothness rather than only whether its sampled positions happen to be close.

4.6.3 Candidate selection comparison

For each window w, let $m(c, w)$ be the trajectory error of candidate c and let $c_0(w)$ be the policy-only candidate. A branch selecting $c_s(w)$ records a pairwise win when:

$$\mathbb{1}_{\text{win}}(w) = \begin{cases} 1, & m(c_s, w) < m(c_0, w), \\ 0, & \text{otherwise.} \end{cases} \tag{4.6}$$

The win rate is:

$$\hat{p}_{\text{win}} = \frac{1}{W} \sum_{w=1}^{W} \mathbb{1}_{\text{win}}(w), \quad W = 7607. \tag{4.7}$$

Mean deltas are reported as selected candidate minus candidate 0, so negative values indicate improvement. Every delta in Chapter 6 is reported alongside a 95% confidence interval on that estimate. Informally, the interval is the range the true effect would fall in on 95% of repetitions of this measurement process; a narrower interval means the effect is more precisely estimated, and an interval that does not cross zero means the direction of the effect is unlikely to be due to chance alone. Confidence intervals are computed by clustered bootstrap over evaluation episodes [56], and task-clustered intervals [57] are used as a robustness check where task-level dependence is the relevant uncertainty source. Small win-rate differences are not interpreted without their clustered interval or a corresponding paired delta analysis.

A scorer that prefers one candidate over another is not automatically a scorer that ranks well. A preference can look decisive on paper while still failing to track which candidate is actually the better one, if what the scorer favours does not systematically correspond to lower trajectory error. Win rate and mean delta, defined above, describe the outcome of a single selection decision. They do not on their own say whether a scorer's ordering of candidates agrees with the candidates' true quality ordering across windows. A separate measure is needed for that question.

Spearman rank correlation provides it. For a set of n candidates, let $R_1(i)$ be the rank assigned to candidate i by the scorer's own score, and $R_2(i)$ the rank assigned by

the candidate's true trajectory error. The correlation is:

$$\rho_s = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2-1)}, \quad d_i = R_1(i) - R_2(i). \tag{4.8}$$

ρ_s ranges from -1 (the scorer's ordering is exactly reversed) through 0 (the scorer's ordering carries no information about true quality, equivalent to ranking candidates without the scorer's guidance at all) to $+1$ (the scorer's ordering exactly matches the true one). Confidence intervals on ρ_s follow the same clustered-bootstrap procedure defined above, resampled over evaluation episodes [56, 57].

This metric matters wherever a scorer's aggregate error reduction alone cannot settle whether the world model is doing genuine work. Chapter 6 applies it as a causal-diagnostic check throughout, most importantly for validating LeWM and the other world models on whether their candidate rankings are genuinely predictive rather than merely correlated with an improved outcome by some other route.

4.7 Evaluation Scope and Limitations

The following scope constraints and known limitations apply to the experimental design.

Offline evaluation only. No physical robot is used and no task-completion signal is available; all metrics are computed against held-out demonstrated trajectories (Section 4.6.1). Results should be read as evidence about candidate selection quality in the offline trajectory prediction setting, not as claims about downstream robot task success.

Kinematic action space and embodiment. The action representation is the bimanual $\mathrm{SE}(3)$ wrist space described in Section 4.4, with two properties worth noting rather than treating as open scope gaps:

- *No finger, contact, or force signals.* Finger-level articulation, contact forces, and haptic feedback are absent. Given how accurately EgoDex already fits hand pose, an additional contact or force channel would have been a valuable enrichment for finer-grained manipulation analysis, though the kinematic evaluation performed here does not require it.
- *Human-to-robot kinematic gap.* Demonstrations are produced by human subjects, whose wrist kinematics differ from robot manipulators in range of motion and joint compliance. This is not a limitation of the present study, which makes no robotic-deployment claim; it becomes relevant only once M.AI.ESTRO's later robotic-translation pilot (Section 4.1.1) is pursued.

Training budget as a compute-constrained lower bound. The universal 100k-item budget (Section 4.5.1) is not a saturation point for all families, and the gap is not marginal for some of them. Doubling the budget from 50k to 100k changes validation loss by only 1.3% for ACT and 0.5% for LeWM, both effectively flat, but by 16.6% for Diffusion Policy, 11.4% for V-JEPA2-AC, and 19.9% for DexWM (Appendix B.10). DexWM's final doubling is larger than its preceding one (10.8% from 25k to 50k), showing no sign of decelerating within the swept range. For these three families the shared budget is a genuine compute-constrained lower bound, not an approximate saturation point, so their reported results likely understate what a longer training run would achieve, potentially substantially for DexWM. Performance differences may partly reflect differing distances from respective saturation budgets rather than intrinsic architectural differences.

Candidate pool size as a compute-constrained choice. The five-candidate pool size (Section 4.2) is a compute-constrained choice rather than a literature-derived one; the full sweep and its findings are detailed in Appendix B.13. Absolute effect sizes throughout should be read as a property of this budget, since margins grow monotonically, without a knee, as pool size increases. Comparative conclusions are unaffected, since scorer ranking is invariant at every pool size tested, but that invariance was only established on the Diffusion Policy row (Section 4.2), not the ACT row.

DexWM pretraining abstention. Published DexWM weights are unavailable [25]. A fixed-budget implementation was evaluated, but the full upstream EgoDex+DROID pretraining recipe was abstained from. The reasons are practical and methodological: DROID raw onboarding is storage- and preprocessing-heavy, DexWM inference and training are substantially more expensive than the other routes, the fixed-budget result did not suggest a likely headline gain, and full upstream-scale pretraining would move outside the common fairness contract used for the main benchmark.

5 Implementation

This chapter describes what was actually implemented and what each route can support as evidence. The implementation does not end with a uniform comparison where every model family becomes a headline result. Instead, it produces an evidence hierarchy: V-JEPA2-AC is the main semantic world-model scorer, LeWM's rollout scorer is a genuine but negative visual-dynamics result, its action-prior scorer is an illustrative action-regularity reference rather than a comparable result, DexWM is a fixed-budget negative comparator, DINO-WM and JEPA-WM are exploratory official-checkpoint routes, and DiWA/CALVIN is a secondary reproduction and comparison study, reported separately from this evidence hierarchy.

The chapter is organised around six concerns: repository ownership and reproducibility (Section 5.1); data and windowing (Section 5.2); training and hardware configuration (Section 5.3); policy candidate generation (Section 5.4); model-specific implementation paths (Section 5.5); and implementation challenges that changed the scientific interpretation of the project (Section 5.7).

5.1 Repository and execution model

The research stack is organised as a single Python package under `src/`, with one top-level module per concern: `data/` handles indexing and windowing, `policy/` implements the two policy architectures and the world-model-aware guard, `world_model/` implements each world model family, and a `cli/` layer exposes every stage as a runnable command (Figure 5.1). A sibling `scripts/` directory collects the Docker flags, GPU assignment, and mount paths for a given model or operation into one launch script (e.g. `dexwm.sh`, `lewm.sh`), backed by a small set of shared bash libraries and a generic, config-driven `run_stage.sh` driver for ad hoc runs.

Docker is the canonical runtime surface for training and evaluation, built from a pinned `Dockerfile` on PyTorch with CUDA 12.4 support. Reproducibility depends on keeping code and generated output on separate, auditable layers: the version-controlled `src/`, `cli/`, and `scripts/` read and write `datasets/` (EgoDex episodes, the window index, cached backbone checkpoints), which in turn produce `runs/wact/` (checkpoints, manifests, metrics, and guard results, the generated evidence read by the thesis). Only the code is tracked by git; both data directories are regenerated by the pipeline rather than version controlled (Figure 5.2), and this separation holds regardless of which host machine executes a given stage. Pretrained backbones (V-JEPA2, DINOv2, DINOv3) are pulled once from PyTorch Hub or HuggingFace and cached locally rather than re-downloaded per run. Individual routes

use isolated dependency caches where an upstream project's requirements would otherwise conflict with the main environment, notably the JEPA-WMS and DiWA checkouts used in Sections 5.5.4 and 5.5.5.

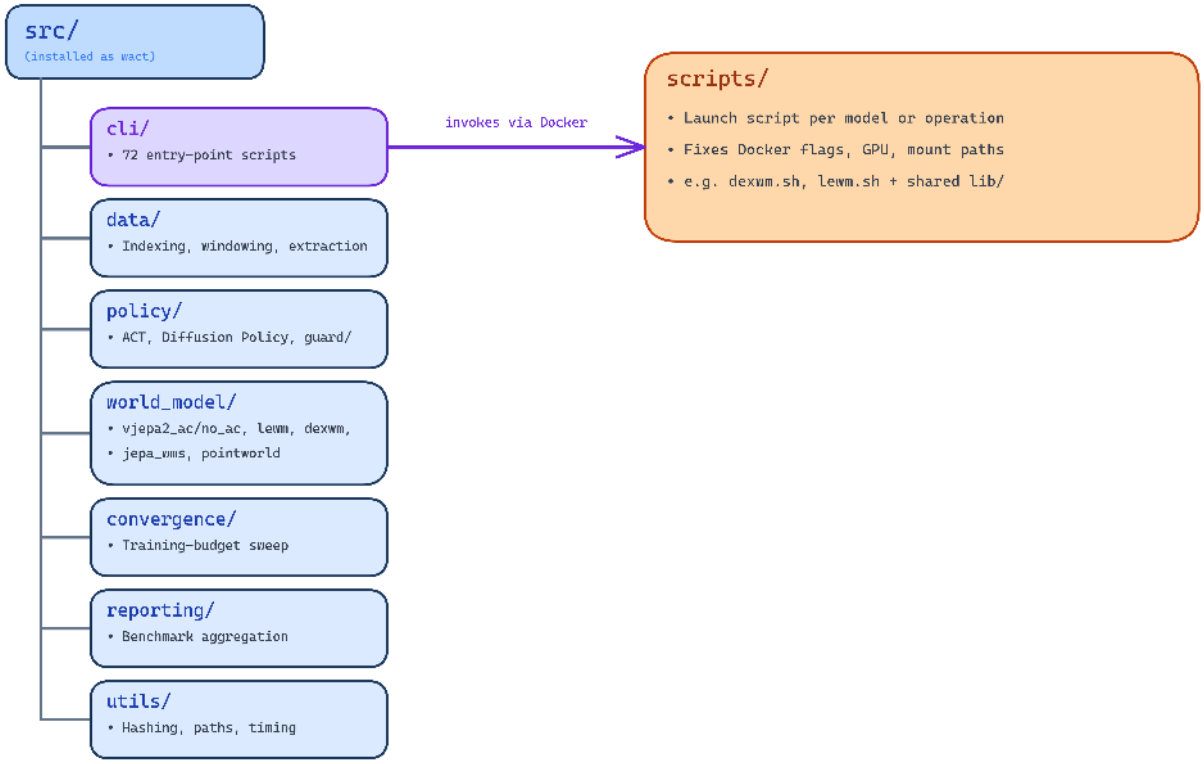


Figure 5.1 Package structure under `src/`, launched via `scripts/` and `cli/` inside Docker.

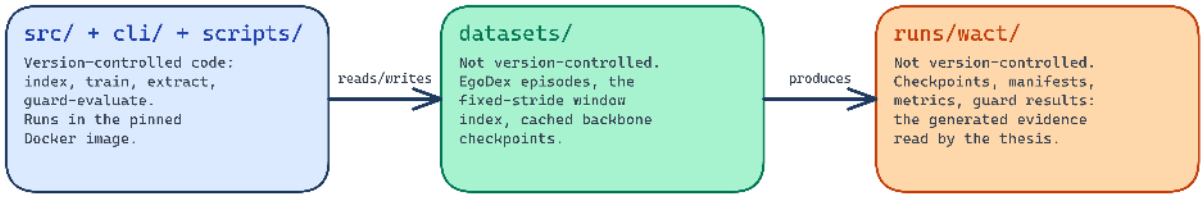


Figure 5.2 Data flow from version-controlled code to the generated, non-version-controlled `datasets/` and `runs/wact/` artefacts.

The run output directory follows a split-first, then model layout. Each training job writes a `result.json`, model checkpoints, metrics, and a manifest. Canonical evidence additionally records candidate hashes, support-role metadata, window counts, and fair-evaluation receipts. These manifests are treated as part of the result, not as optional logs.

5.2 Data pipeline and windowing

The index stage discovers valid paired EgoDex episodes (a `.mp4` video plus a matching `.hdf5` pose file), infers episode lengths from the Hierarchical Data Format 5 (HDF5) metadata, and writes the canonical observation/prediction windows defined by `obs16_pred16_s128` (Section 4.1.2). It is resumable and excludes unpaired files up front, so a missing video or annotation surfaces here rather than as a downstream training error. The canonical index contains 194,333 training episodes and 430,962 training windows for `train_part1_2_3`, plus 3,229 held-out test episodes and 7,607 held-out test windows.

Every downstream component reads from this same index rather than re-deriving windows, which keeps the branch comparison of Section 4.2 attributable to branch design rather than to accidental differences in the data each component received. The two policy architectures consume the RGB frame and 18D action target directly; world model extraction additionally consumes the action-delta encoding used to build transition-head training pairs; guard evaluation consumes the frozen index to generate and score candidate pools.

Three cumulative training splits were assembled from the first three parts of the EgoDex release (Table 4.1 of the methodology). Each split is a strict superset of the previous: the larger splits were assembled with relative symbolic links rather than by copying data, avoiding duplication of approximately 1 TB of earlier data on disk while allowing the data-scaling axis of the benchmark to be evaluated directly, since all model families are trained independently on each split at the same universal budget (Section 4.5.1).

5.3 Training configuration

Table 5.1 consolidates the hardware and batch configuration for all model families at the universal budget $E = 100{,}000$. Settings marked *(shared)* are kept identical across all trainable models. The effective-item budget is verified for each trainable family via $E = S \times B_{\text{global}}$, where S is the number of optimisation steps and B_{global} is the effective batch per step. V-JEPA2 (no AC) and PointWorld require no model training and reuse the Phase 1 extraction shared with V-JEPA2-AC.

Across trainable components, random seed 42 is used throughout; no canonical training run departs from it. Training manifests record optimiser steps, effective exposure, device count, batch size, and checkpoint paths. V-JEPA2-AC uses AdamW for the transition head; the JEPA-WMS adapter calibration also uses AdamW with learning rate 10^{-3}. Checkpoint selection is by validation loss where a validation surface

Table 5.1 Training configuration for all model families at $E = 100{,}000$. Approximate step time is shown for trainable phases only (––– denotes not applicable).

Family	**GPUs**	**Batch/device**	**Steps** S	B_{global}	**s/step**
ACT	4×L40S	32	781	128	≈0.5
Diffusion Policy	4×L40S	32	781	128	≈0.8
V-JEPA2-AC Ph. 1 (extraction)†	2×L40S	—	—	—	—
V-JEPA2-AC Ph. 2 (AC head)†	1×L40S	32	3,124	32	<0.01
LeWM	4×L40S	4	6,250	16	≈0.047
DexWM‡	4→3×L40S	4	7,234	≈14	≈28
V-JEPA2 (no AC)	—	—	—	—	—
PointWorld	—	—	—	—	—

† Phase 1 is a one-time offline extraction (2×L40S, sharded, 27.4 windows/s, no backward pass). Measured wall times: `train_part1` ≈0.75 h, `train_part1_2` ≈1.4 h, `train_part1_2_3` ≈2.2 h. Phase 2 head training completes in under ten seconds per split (≈0.01 s/step on cached embeddings) on one GPU, using a deterministic budgeted-exposure sampler that oversamples with a fixed permutation-based schedule whenever raw training pairs fall short of the target, so its effective exposure matches the universal budget exactly (99,968 items), the same as every other family.

‡ DexWM trained on 4×L40S for `train_part1` and `train_part1_2` (6,250 steps each, global batch 16); the split reported here, `train_part1_2_3`, resumed on 3×L40S after step 3,299 following a GPU-availability change, ending at 7,234 steps with the effective-item budget preserved. DexWM total wall time across all three training splits was approximately 150 h (≈28 s/step throughout, 6,250 + 6,250 + 7,234 steps); LeWM total wall time was approximately 15 minutes (≈0.047 s/step), substantially faster than DexWM despite the identical effective budget.

exists; otherwise the frozen run manifest and hash identify the exact artefact used for evaluation.

None of the implemented routes natively consume EgoDex's RGB and bimanual wrist-state observation, or emit an 18-dimensional, 16-step action chunk. Every family required a purpose-built input or output adapter. ACT and Diffusion Policy are architecture specifications rather than pretrained models with a fixed interface, and every world model family was pretrained (where a pretrained checkpoint exists at all) on a different observation or action convention. Table 5.2 records, per route, the interface actually consumed and produced and whether that interface required a custom adapter (Section 5.5 details each adapter).

Table 5.2 Model input/output contracts used by the implemented routes.

Route	Input	Output	Loss / score	Adapter
ACT / Diffusion Policy	RGB frame + wrist state	16-step, 18D action chunk	ACT: reconstruction + KL DP: diffusion denoising	18D bimanual action head
V-JEPA2-AC	V-JEPA2 embedding + 18D action chunk	Future latent embedding	Latent prediction distance	Action-delta encoder into frozen latent space
LeWM rollout	RGB/action sequence	Predicted compact latent	Latent-support score	—
LeWM action-prior	Candidate 18D action chunk	Scalar regularity cost	Training-action z-score prior	—
DexWM fixed-budget	DINOv2 patch tokens + pose actions	Future patch/key-point latent	CDiT + HC training losses	—
DINO-WM / JEPA-WM	Official visual latent + adapted 7D action	Future official latent	Support/action-policy score	18D→7D linear adapter

5.4 Policy training

Both policy families share the same observation encoder: a ResNet-18 visual backbone [58] processes the single observation RGB frame (96×96 pixels) through four residual stages (64, 128, 256, and 512 channels), producing a 512-dimensional

feature vector that is concatenated with the bimanual wrist state before being passed to the policy-specific decoder (Figure 5.3). Image resizing to 96×96 is applied at loading time and is consistent across all branches. Wrist pose sequences are normalised to zero mean and unit variance using statistics computed from the training split; these statistics are frozen after training and consumed by the world model scoring stages to ensure consistency.

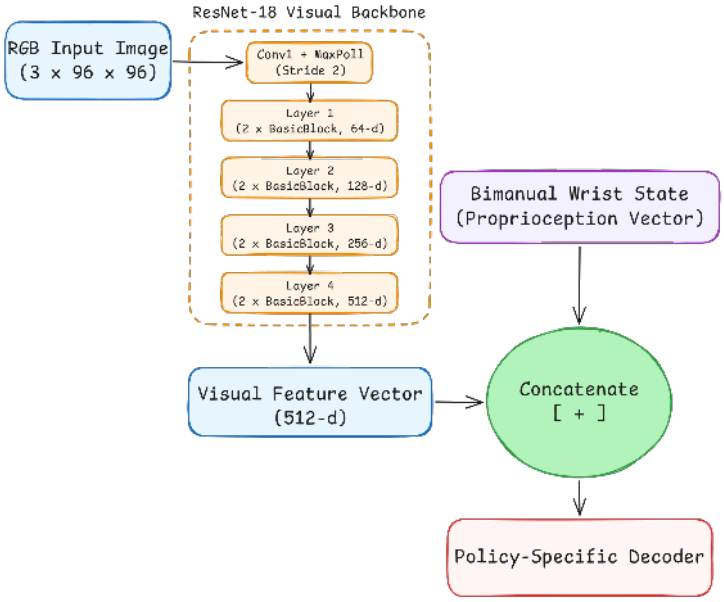


Figure 5.3 Shared ResNet-18 visual backbone preceding both policy decoders. Adapted from [58].

5.4.1 Action Chunking Transformer

The ACT architecture is reviewed in Section 3.1.1 and illustrated in Figure 3.1. The implementation follows the architecture and training procedure of Zhao et al. [8] with no dependency on an external codebase. Custom components include the EgoDex HDF5 window loader, the 18-dimensional bimanual wrist action space adapter, and a checkpoint serialisation layer that freezes the CVAE configuration and normalisation statistics alongside the model weights. The CVAE encoder-decoder and transformer decoder are realised directly from the paper specification. At training time, the CVAE encoder compresses a ground-truth action chunk together with the current observation into a style variable $z \sim \mathcal{N}(\mu, \sigma^2)$; the transformer decoder reconstructs the chunk from z and the observation. The encoder is discarded at evaluation time, and z is drawn from the standard Gaussian prior $\mathcal{N}(0, I)$. The design intent is to generate up to N diverse candidates by sampling independent z vectors and decoding each under the same observation. Training uses four L40S GPUs (DDP) with a per-device batch of 32 (global batch 128): $E = 781 \times 128 \approx 100{,}000$ (Table 5.1). Each run writes a

`best.pt` checkpoint selected by lowest validation loss, a `last.pt` checkpoint, per-epoch metrics in `metrics.jsonl`, and a frozen `manifest.json` recording the configuration and window file consumed.

CVAE posterior collapse: investigation and retraining. The trained ACT checkpoint exhibits CVAE posterior collapse: at inference time the decoder is latent-insensitive, and sampling five independent z vectors from $\mathcal{N}(0, I)$ produces five identical action chunks. A diagnostic confirms the extent of the collapse: injecting $z = 10\sigma$ (ten standard deviations from the prior) via a forward pre-hook on the latent projection layer shifts the output by approximately 3×10^{-7} in absolute value, which is effectively zero. The mean pairwise difference across five independently drawn candidates is below 10^{-6} on all three training splits, i.e. within floating-point noise of exactly zero; this threshold is denoted ε throughout the remainder of this section.

This is a well-documented failure mode of conditional VAE models rather than a mis-implementation [30]. When the conditioning signal (the visual observation) is sufficiently informative to reconstruct the target without consulting z, the decoder learns to ignore z entirely. The KL divergence term in the training objective regularises the *encoder* distribution toward the prior but cannot compel the *decoder* to use the latent; the collapse is therefore not resolved by increasing the KL weight. Four retraining runs were conducted to test this:

- *v1*: `kl_weight=10` (default): collapse unchanged.
- *v2*: `kl_weight=100`: weighted KL contribution ($100 \times 0.25 \approx 25$ nats) exceeded reconstruction loss (~ 15), yet collapse persisted.
- *v3*: cyclical KL annealing [59] (`kl_cycle_batches=200`, `kl_cycle_ramp_frac=0.5`) and 30 % observation dropout to force decoder reliance on z: collapse persisted.
- *v4*: cyclical KL annealing combined with free bits [31] (`free_bits=0.5` nats/dim, no observation dropout), run for completeness after v3: collapse persisted.

All four runs produced mean pairwise differences indistinguishable from zero on every split (Table 5.3). The EgoDex visual observations are sufficiently informative that the decoder finds a loss minimum that ignores z regardless of training pressure, and correcting the collapse without an architectural change (discrete latents, vector-quantisation, or an explicit diversity head) is outside the scope of this thesis.

Table 5.3 ACT CVAE posterior collapse: four successive retraining attempts, all indistinguishable from zero ($\varepsilon = 10^{-6}$) on every training split.

Run	`kl_weight`	Cyclical KL	Obs. dropout	Free bits	Mean pairwise diff
v1	10 (default)	no	0 %	no	$< \varepsilon$
v2	100	no	0 %	no	$< \varepsilon$
v3	100	yes (200-batch cycle, 50 % ramp)	30 %	no	$< \varepsilon$
v4	100	yes (200-batch cycle, 50 % ramp)	0 %	0.5 nats/dim	$< \varepsilon$

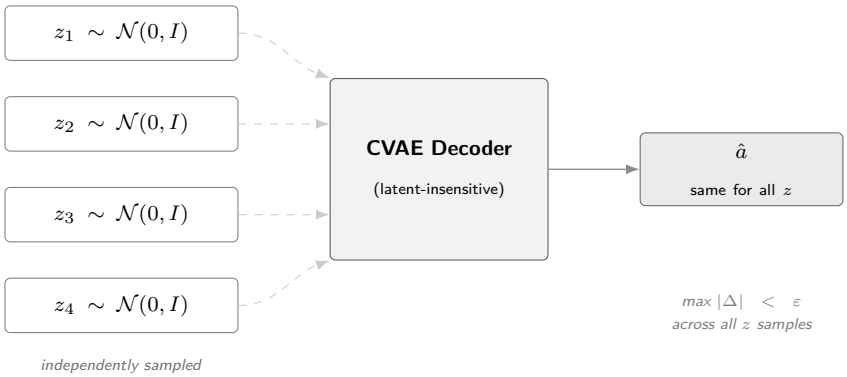


Figure 5.4 CVAE posterior collapse in the trained ACT checkpoint. Four independently sampled latent vectors produce an identical output $\hat{a}$ (dashed arrows indicate the latent path the decoder effectively ignores).

The collapse is treated as an empirical finding, illustrated schematically in Figure 5.4. It motivates the use of an external candidate generator, described below, and is reported in Section 6.2 as a characteristic of the ACT policy on this dataset.

CEMP candidate augmentation. To restore the five-candidate evaluation pool, ACT-based guards use a CEMP planner. This is an adaptation of the Cross-Entropy Method [60] initialised from the policy's output rather than a uniform prior, following the model-based planning approach of TD-MPC2 [61]. A perturbation is a small SE(3) displacement applied to the base action chunk, shifting each wrist pose by a sampled translation and rotation offset while preserving temporal coherence across the 16-step chunk. Starting from the collapsed ACT B1 output, three iterations each draw 64 such perturbations around the current mean, score them with the world model's latent prediction function, and retain the top 25 % (16 elites); the standard deviation decays by $0.7\times$ per iteration. The four highest-scoring candidates across all iterations are returned (Figure 5.5).

For Branch 2 the pool contains these four alternatives plus the ACT B1 output ($N{=}5$); for Branch 3 the B1 output is excluded, leaving the four alternatives only.

Circularity caveat. The same world model that scores candidates inside CEMP's three refinement iterations (Figure 5.5, from Iteration 1 onward) also performs the final B2/B3 selection over the resulting pool. This is a deliberate design choice, not an oversight. The ACT policy itself cannot generate diverse candidates (the posterior collapse above), so some external proposal mechanism is required; using the world model under test to shape that proposal is the only way to obtain a five-candidate pool at all for this policy family. The consequence is that ACT B2/B3 results are not a clean measurement of world-model scoring ability in isolation; they measure the combined world-model-plus-planner system, where the planner has already biased the pool toward the scorer's own preferences before selection ever runs. This caveat does not apply to Diffusion Policy, whose five candidates come from independently seeded DDIM chains with no world-model involvement in their generation. Any ACT B2/B3 advantage over B1 should accordingly be read as evidence for the world-model-plus-CEMP pipeline as a unit, not as an isolated measurement of scorer quality comparable to the Diffusion Policy branch.

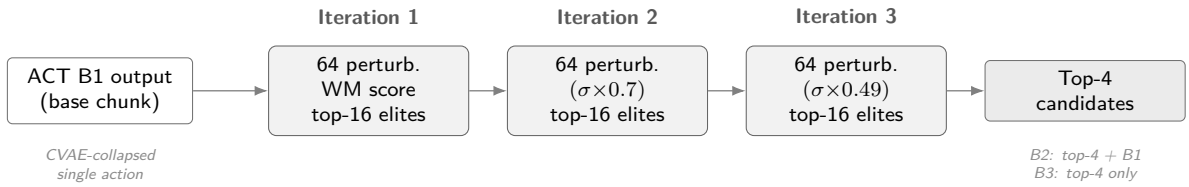


Figure 5.5 CEMP candidate augmentation, refining SE(3) perturbations of the ACT B1 output over three iterations to form the B2/B3 evaluation pool.

5.4.2 Diffusion Policy

The Diffusion Policy implementation follows the transformer-backbone variant (DP-T) described in Section 3.1.2 and illustrated in Figure 3.2. The implementation follows Chi et al. [9] with no external codebase dependency. Custom components include the 18-dimensional bimanual wrist action space adapter, the EgoDex HDF5 window loader, and the five-candidate DDIM evaluation protocol (five independent noise seeds decoded under the shared conditioning signal). The same ResNet-18 observation encoder produces a conditioning vector `obs_cond` passed to a DiT denoising backbone via cross-attention. At evaluation time, five independent DDIM denoising chains are run on the same observation to produce five candidate action chunks, each starting from independently sampled Gaussian noise and converging to a distinct trajectory under the shared conditioning signal. Branch 1 selects among them by taking the chain

with the lowest final denoising residual. Training uses four L40S GPUs (DDP) with a per-device batch of 32 (global batch 128): $E = 781 \times 128 \approx 100{,}000$ (Table 5.1). The same checkpoint, metrics, and manifest artefacts as ACT are written at the end of each run.

DDIM subsampling correction.

$\bar{\alpha}_t$	cumulative noise schedule product at step t (List of Notations)
t_{prev}	next step in the subsampled inference schedule (List of Notations)
$\hat{x}_0$	predicted clean sample (List of Notations)
x_T	Gaussian noise at the schedule endpoint (List of Notations)

An implementation audit identified an error in the reverse-diffusion sampler [55]: the original implementation passed $\bar{\alpha}_{t-1}$ (the consecutive index in the training schedule) instead of $\bar{\alpha}_{t_{\text{prev}}}$:

$$\text{Bug: } \bar{\alpha}_{t-1} \quad \longrightarrow \quad \text{Fix: } \bar{\alpha}_{t_{\text{prev}}} \tag{5.1}$$

For a ten-step schedule $t \in \{90, 80, \ldots, 0\}$ this substitutes $\bar{\alpha}_{89}$ for the correct $\bar{\alpha}_{80}$ at the first denoising step, miscalibrating the $\hat{x}_0$ weighting throughout. The diversity guarantee is unaffected: each candidate begins from an independent $x_T \sim \mathcal{N}(0, I)$, so the collapsed-latent failure mode present in ACT cannot arise. The fix supplies the explicit next-in-schedule index to `ddim_step`; all three evaluation splits are re-run under the corrected sampler and the corrected results supersede the initial runs. The correction did not change the qualitative conclusion that Diffusion Policy supplied a diverse candidate pool, but it improved the integrity of the candidate source used by all downstream world-model selectors. The before/after audit is retained in the implementation record rather than used as a separate thesis result.

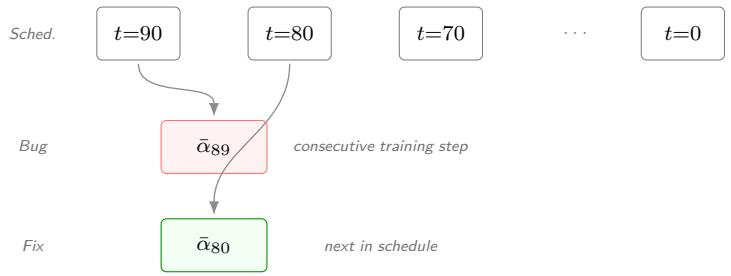


Figure 5.6 DDIM subsampling correction. The ten-step inference schedule spans $t \in \{90, 80, \ldots, 0\}$. At each reverse step the correct $\bar{\alpha}_{t_{\text{prev}}}$ is the *next timestep in the subsampled schedule*; the bug substituted the consecutive training-schedule timestep $\bar{\alpha}_{t-1}$, miscalibrating the $\hat{x}_0$ weighting at every step.

5.5 World model training

Table 5.4 summarises what each family reused from an external checkpoint versus trained for this study; Table 5.2 already covers the interface adapters this required. The

remainder of this section focuses on the design decisions and defects specific to each family, rather than repeating the loading mechanics that are common across all of them.

Table 5.4 Backbone sourcing and what was trained from scratch for this study, by world model family.

Family	Backbone / checkpoint source	Trained for this study
V-JEPA2-AC	Frozen ViT-Large, official checkpoint (PyTorch Hub)	Action-conditioned transition head
LeWM	None; no pretrained backbone released	Encoder, predictor, action embedder, and projector (fully, from scratch)
DexWM	Frozen DINOv2 ViT-Large, official checkpoint (HuggingFace)	CDiT predictor, hand-consistency head
DINO-WM / JEPA-WM	Official DROID/RoboCasa checkpoints (Meta)	None; frozen official weights
DiWA/CALVIN	Official world model and reward classifier (checkpoints obtained from the authors)	None; C3 reproduces the paper's own PPO fine-tuning
V-JEPA2 (no AC)	Reuses V-JEPA2-AC Phase 1 embeddings	None
PointWorld	Reuses V-JEPA2-AC backbone and transition head	None

5.5.1 V-JEPA2-AC: extraction and transition head

The V-JEPA2-AC architecture is reviewed in Section 3.2.3 and illustrated in Figure 3.8. Training separates into two phases. Phase 1 is a one-time offline extraction. The frozen encoder processes all indexed windows in parallel across two L40S GPUs, writing one embedding vector per window with no backward pass or optimiser. A critical bug in an early run applied a 2,000-window default cap, producing only around 1,000 training pairs from a 140,000-window split (Section 5.7). Phase 2 trains a small MLP transition head on the stored embeddings to predict the next embedding from the current embedding and an action-conditioning vector derived from the bimanual wrist-state delta (Table 5.1).

5.5.2 LeWM: end-to-end training

LeWM is reviewed in Section 3.2.3 and illustrated in Figure 3.10. It is trained entirely from scratch, with no external weights. The encoder, autoregressive predictor, action embedder, and projector are jointly optimised together. The training objective combines a next-embedding prediction loss with the SIGReg regulariser, which enforces a Gaussian prior on the latent distribution [26] (Table 5.1).

Two export-pipeline defects affected early guard runs. A mismatched frame cap between the training and evaluation exports silently excluded 97.9 per cent of test windows. A later normalisation-statistics defect then introduced a 1.74× scale mismatch in the LeWM cost function, caused by loading test-set action statistics instead of training-set ones. Both defects are resolved and detailed in Section 5.7.

LeWM exposes two distinct scoring routes, evaluated separately in this study and shown side by side in Figure 5.7. The rollout scorer predicts a future state through LeWM's learned visual dynamics, in the same role as V-JEPA2-AC. The action-prior scorer instead scores a candidate directly against a training-only action distribution, without predicting anything.

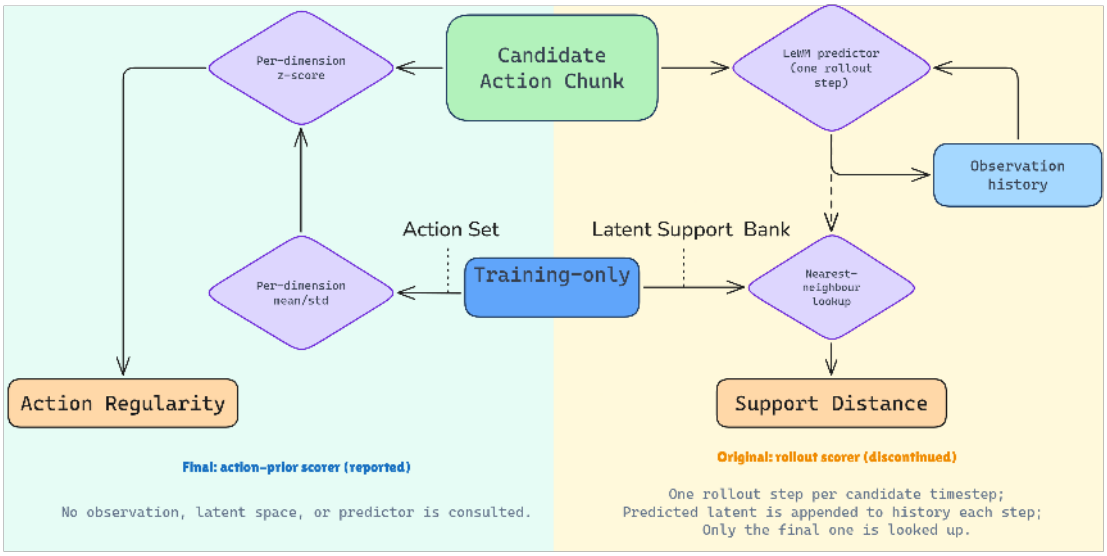


Figure 5.7 The two LeWM scoring routes evaluated in this study, both starting from the same candidate action chunk: the rollout scorer (right), which predicts forward through LeWM's learned dynamics, and the action-prior scorer (left), which does not.

The *rollout scorer* takes the candidate action chunk and the observation history into the LeWM predictor, one rollout step at a time. Each predicted latent is appended back to the observation history before the next step, so the predictor consumes its own prior output. After all steps, only the final predicted latent is looked up against a training-only latent support bank, via nearest-neighbour search, to produce a support distance. This is the mechanism LeWM was built to provide: a genuine forward prediction through its learned visual dynamics.

The *action-prior scorer* takes the same candidate action chunk, but scores it directly against a training-only action set's per-dimension mean and standard deviation, producing a per-dimension z-score and, from that, an action regularity score. It does not consult the observation history, the latent space, or the LeWM predictor at all. It never rolls the candidate forward, and the predictor plays no role in the score it produces. It measures whether a candidate's actions are typical of LeWM's training

distribution, not whether LeWM predicts that candidate will produce a better outcome.

These two routes are not interchangeable, and only one of them exercises LeWM's learned dynamics at all. Section 6.1 reports both, and treats them accordingly: the rollout scorer as LeWM's genuine, reportable result, and the action-prior scorer as an illustrative reference only, not a measurement of LeWM's visual prediction.

5.5.3 DexWM: predictor trained from scratch

DexWM is reviewed in Section 3.2.3. Its pipeline pairs a frozen DINOv2 image encoder with a CDiT predictor and incorporates a hand-consistency loss that supervises fingertip and wrist heatmap predictions (Figure 5.8).

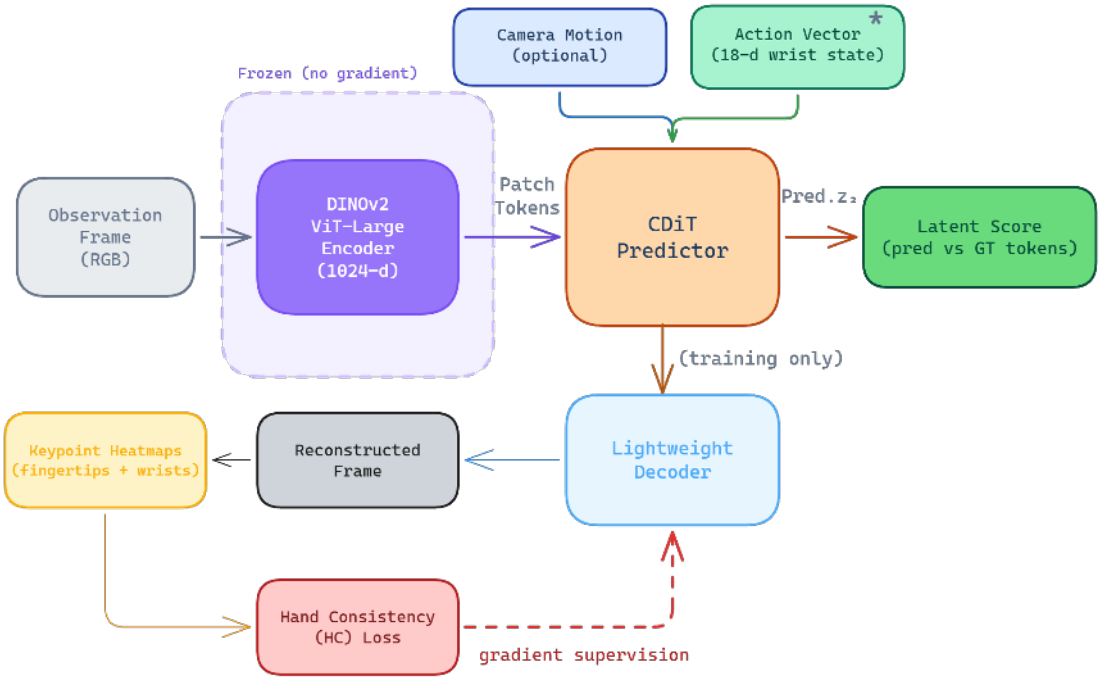


Figure 5.8 DexWM pipeline as implemented, comprising a frozen DINOv2 encoder, CDiT predictor, lightweight decoder, and HC loss supervision path. Adapted from [25].

Checkpoint sourcing and abstained full pretraining

The published DexWM architecture [25] exposes architecture code and dataset instructions, but no public pretrained checkpoint could be sourced. The associated HuggingFace repository contains trajectory data rather than weights, and the GitHub repository describes pretraining on EgoDex and DROID without linking a downloadable model checkpoint. This matters because the published result depends on a large upstream training recipe that is not reproduced merely by having the code.

This study's implementation therefore uses a fixed-budget local route. The frozen DINOv2 encoder provides patch tokens, and the CDiT predictor and

hand-consistency head are trained from random initialisation on the declared training splits at the universal budget ($E \approx 100{,}000$, Table 5.1). Training is a genuine action-conditioned next-patch-embedding prediction task. The 18D SE(3) wrist-action delta conditions every predictor block via adaLN-Zero modulation [62], and the loss compares the resulting prediction against the true future embedding rather than a reconstruction of the input, so the route is not degraded to unconditional pretraining.

A later preflight tested whether the full official EgoDex+DROID pretraining suite should be reopened instead of this fixed-budget route, and passed the EgoDex-side checks, but the DROID raw data needed for the published recipe was not available locally. Reopening it would also have trained DexWM on a strictly larger corpus than every other family, breaking the fairness contract that keeps the main comparison attributable to model design rather than to training-data scale. Since the fixed-budget route was already valid, full upstream pretraining was abstained from rather than launched as a new open-ended experiment; its performance is expected to fall short of the published recipe as a result. This is a reproducibility limitation of the available release as well as a project-scope decision. Code without a pretrained checkpoint leaves independent researchers to repeat a large-scale pretraining programme before they can evaluate the published model under a new benchmark.

5.5.4 DINO-WM and JEPA-WM exploratory selectors

DINO-WM and JEPA-WM were added after the initial closeout as bounded official checkpoint comparators. The implementation uses the official JEPA-WMS code path and public Meta checkpoints, downloaded from HuggingFace. DINO-WM DROID loaded directly. JEPA-WM DROID also required the official gated DINOv3 ViT-L/16 backbone; access to the gated repository was obtained under a university research-program licence, and the backbone was then downloaded and converted into the native format expected by the upstream loader.

The central implementation issue is action compatibility. This study's candidates are 16-step chunks in an 18-dimensional bimanual wrist action space. The official DROID/RoboCasa JEPA-WMS checkpoints expect a 7-dimensional single-arm robot action. A direct truncation would invent semantics and discard one hand, so the implementation uses a train-only linear adapter from EgoDex action chunks into the expected 7D interface. This keeps the route auditable, but it also weakens the claim. The scorer is an adapted transfer experiment, not a native EgoDex world model.

Both models passed load, extraction, and scorer smoke tests, and were carried through a bounded validation pilot rather than the held-out evaluation used for the main comparison. No held-out test claim is reported from this route; the pilot outcome and its interpretation are reported in Appendix A.5 rather than here, since neither

model was run to a comparable, fully-evaluated state.

5.5.5 DiWA/CALVIN secondary-study implementation

DiWA/CALVIN is a closely related study, planned in Section 4.4 of the methodology, and reproduced here for integrity. It lets this study's findings be checked alongside an independent, world-model-guided policy improvement method under shared components rather than in isolation.

A dedicated DiWA/CALVIN checkout was prepared alongside the official CALVIN benchmark. Public artefacts, including the frozen world model, reward classifier, and environment assets, were downloaded, hashed, and checked for archive safety before use, and the world model and reward classifier were verified loading on CPU and in a bounded GPU timing check.

The official base-policy and fine-tuned policy checkpoints, together with the statistics and sampler metadata needed to load them, were not published with the initial release. Access was requested directly from the DiWA authors, and a corresponding issue was also filed on the official repository. The needed files were obtained through this direct correspondence, covering `close_drawer`, `turn_on_led`, `move_slider_left`, and `turn_on_lightbulb`.

A four-condition experiment contract (`contract.py`) holds the base policy, world model, reward classifier, environment, and initial states fixed across all four conditions, so the improvement operator is the only variable that changes between them:

- **C0**: the untouched upstream baseline, no improvement operator applied.
- **C1**: a fixed-candidate control, isolating candidate-pool effects from selection quality.
- **C2**: this study's test-time best-of-five selector, using the frozen DiWA world model and success classifier to choose among candidates without retraining the policy.
- **C3**: the paper's own imagined-rollout PPO fine-tuning, which updates the policy itself inside the frozen world model.

This secondary study departs from the primary EgoDex protocol by design. C2 and C3 execute the selected or fine-tuned policy in the real CALVIN simulator, and C3 updates the policy itself through reinforcement learning. The primary study, by contrast, scores fixed candidate chunks offline against held-out demonstrations without simulation or policy updates. This departure is required to reproduce DiWA under conditions faithful to its own design; the two studies remain separate and are not merged into a single evidence hierarchy.

5.5.6 V-JEPA2 (no action conditioning): ablation control

The methodological role of this ablation is described in Section 4.4. In implementation, the variant reuses the Phase 1 embeddings extracted for V-JEPA2-AC, requiring no additional training (Table 5.1). At guard time the scorer passes a zero action vector to the transition model for all candidates, producing action-blind scores that reflect visual plausibility alone. A diagnostic persistence loss (mean MSE plus 0.1$\times$ cosine distance between consecutive stored frame embeddings, the same weighted objective as the AC head training loss) is computed directly from `embeddings.npy` without invoking the transition model, confirming the scorer makes no action-differentiated prediction under a zero action vector.

5.5.7 PointWorld: action-degradation control

The methodological role of the PointWorld control is described in Section 4.4. The implementation uses the same frozen V-JEPA2-AC backbone and transition head as the V-JEPA2-AC family, requiring no additional model training (Table 5.1). The sole difference is that candidate action chunks are degraded before being passed to the transition head. This is a deliberate experimental degradation, built to create a controlled ablation, distinct from the incidental information loss in the DINO-WM/JEPA-WM action adapter (Section 5.5.4), where one hand's pose is discarded only because it is required to match an external checkpoint's native interface, not to construct a control.

The adapter keeps only the left-wrist rotation and collapses the remaining twelve dimensions (left translation, right rotation, right translation) into a single scalar summary before zero-padding back to 18 dimensions. The right-hand pose is not recoverable from this output; it does not survive as a distinguishable signal in the degraded representation. The transition head therefore receives the correct visual features but systematically corrupted action conditioning.

The convergence study diagnostic additionally computes a proxy loss:

$$\mathcal{L}_{\text{PointWorld-proxy}} = \mathcal{L}_{\text{persist}} + 0.5 \cdot \text{mismatch} \tag{5.2}$$

where $\mathcal{L}_{\text{persist}}$ is the frame-to-frame embedding distance and mismatch is the fraction of action-vector norm discarded by the projection, clipped to $[0, 1]$:

$$\text{mismatch} = 1 - \frac{\|\mathbf{arm}\|}{\|\mathbf{hand}\|} \tag{5.3}$$

This diagnostic confirms that the adapter introduces genuine information loss. The bimanual hand action space does not map faithfully to the 7-DOF arm

representation, as expected for a degradation control.

5.6 B2 margin threshold calibration

Branch 2 selects the world model's top-scoring candidate whenever it beats the policy's own candidate 0 by at least a margin δ, measured in the guard's own score units; below that margin, candidate 0 is kept instead. The canonical configuration used throughout this thesis is unconditional selection, equivalent to $\delta = 0$: the top-scoring candidate is always taken, however small its lead over candidate 0. Raising δ makes the guard progressively more conservative, since a larger lead is required before it will override the policy, so it keeps candidate 0 more often and moves the branch closer to the Branch 1 policy-only baseline. A calibration run compared seven settings of δ directly, on the canonical $K = 5$ candidate pool, the V-JEPA2-AC guard, and the full 7,607-window held-out test set (Table 5.5).

Table 5.5 B2 margin threshold calibration on the canonical evaluation protocol ($K = 5$, V-JEPA2-AC guard, full held-out test set). Higher δ keeps candidate 0 more often and yields a smaller improvement.

Margin δ	Candidate 0 kept	Translation change
n/a*	23.33%	**-6.406%**
0.05	49.38%	-5.354%
0.10	71.22%	-3.695%
0.15	84.83%	-2.260%
0.25	96.04%	-0.687%
0.50	99.89%	-0.030%
1.00	100.00%	0.000%

*Canonical configuration used throughout this thesis: the guard takes the world model's top-scoring candidate unconditionally, with no margin gate.

Read down the table, and the pattern is monotonic in one direction only. As δ rises, candidate 0 is kept more often and the improvement shrinks toward zero. The 100% row is not the best-performing setting; it is the opposite. At $\delta = 1.00$ the margin is large enough that the gate never clears on this score scale, so Branch 2 keeps candidate 0 on every window and exactly reproduces the Branch 1 policy-only baseline, zero change by construction, not by merit. DexWM was excluded from this calibration because its initial guard runs were vacuous due to a candidate-loading defect (Section 5.7); its canonical evaluation instead uses the dedicated `guards_native_dexwm/` pipeline.

Two considerations motivate choosing the unconditional setting over a margin gate:

1. **CEMP already pre-filters toward world-model preferences.** As the circularity caveat of Section 5.4.1 describes, the internal CEMP scoring loop pre-filters the candidate pool toward the world model's own preferences before the guard's final selection ever runs. Adding a margin gate on top would reintroduce a hyperparameter that varies across model families and pool types, without addressing that deeper redundancy.
2. **Unconditional argmax isolates discriminative quality directly.** Measuring the world model's preference without a threshold suppression layer produces a cleaner experimental signal and avoids per-family calibration.

The improvement lives entirely in the windows where the guard overrides the policy, and the unconditional setting overrides most often, which is why it produces the largest reduction in Table 5.5 and is the same configuration reported as the headline result in Table 6.4. This is the chosen configuration for every Branch 2 result in this thesis, since it dominates every tested margin gate on translation error while still keeping candidate 0 a meaningful fraction of the time rather than never or always.

5.7 Implementation challenges and resolutions

The implementation challenges are part of the contribution because several early runs looked plausible but were not valid evidence. Table 5.6 records the material issues, their impact, the corrective action, rerun status, and remaining risk. Longer forensic notes are retained in the project archive and appendices.

The remaining risk is concentrated in routes that were not promoted to main claims. The headline V-JEPA2-AC result, LeWM rollout result, DexWM fixed-budget negative result, and ACT collapse diagnostic all use frozen candidate artifacts, held-out windows, and manifest-level validation.

Table 5.6 Material implementation challenges and resolution status.

Issue	Impact	Fix	Rerun status	Risk
DDIM subsampling error	Miscalibrated reverse-diffusion updates for DP candidates.	Passed the explicit previous timestep t_{prev} into `ddim_step`.	DP evaluations rerun; corrected candidates used downstream.	Low.
Oracle goal leakage	Early B2/B3 scoring runs derived the latent goal from each episode's ground-truth future frame, information unavailable at deployment time.	Set `goal_source=none` in all canonical benchmark configurations.	All results in this thesis use the corrected configuration.	Low.
Branch 3 selection-mode error	B3 initially duplicated B2 because candidate 0 remained eligible.	Relaunched with `world_model_only`; manifest checks require zero policy fallbacks.	Corrected B3 runs replace earlier outputs.	Low.
Score-cache memory collision	Tensor memory-address reuse could silently corrupt scale factors after garbage collection.	Re-keyed cache on explicit run-scoped identifiers.	Affected runs excluded or rerun.	Low.
Evaluation window shortfall	Default guard limits evaluated 787 rather than 7,607 windows.	Added full-evaluation flags and manifest count checks.	Full 7,607-window runs used.	Low.
LeWM wrong normalisation source	Test-set action statistics distorted candidate costs.	Loaded training-only statistics and reran diagnostics.	Rollout scorer still failed; action-prior route separated.	Medium for rollout only.
Candidate-source asymmetry	Earlier families used different candidate sources, confounding comparison.	Final causal matrix uses shared frozen candidates and explicit candidate hashes.	Superseded runs archived.	Low.
DexWM missing checkpoint	Published pretrained model unavailable; zero-score stub risk found.	Stub replaced by hard failure; fixed-budget local route evaluated.	Valid negative/null for local route only.	Medium for general DexWM claims.
DINO-WM/JEPA-WM action mismatch	Official 7D DROID action interface does not match this study's 18D bimanual chunks.	Train-only adapter and validation-only pilots.	Kept exploratory; no held-out claim.	High for headline use.
DiWA missing checkpoints	Reproduction could not proceed without the official base/fine-tuned policy checkpoints and sampler statistics, unpublished with the initial release.	Access requested directly from the authors and via a repository issue; needed files obtained.	C0–C3 secondary study proceeded; results reported separately (Chapter 6).	Low, resolved.
V-JEPA2-AC Phase-2 exposure shortfall	AC transition head trained on 85–90k items instead of the declared $E = 100{,}000$ budget (10–15% short) on all three splits, an undisclosed-until-caught deviation from Table 5.1's uniform-budget claim.	Wired the trainer's existing deterministic budgeted-exposure sampler into the launch script (was implemented but never invoked).	All three splits rerun at the corrected budget; old checkpoints archived, val loss improved slightly on all three.	Low.

6 Results

This chapter works through the results in the order that supports the dissertation's central claim. It starts with the world-model scorers on the DP row, from weakest to strongest, then reports ACT's CEMP-augmented guard results as secondary corroboration of that ranking, examines how selection quality varies with task difficulty, reports a secondary reproduction study on CALVIN, and closes with the candidate pool size ablation and an overall interpretation. All reported numbers come from the same frozen EgoDex `train_part1_2_3` test split of 7,607 windows, 3,229 episodes, and 111 task categories, evaluated once under branch and support rules fixed before interpretation.

The chapter does not measure robot task success, human preference, or closed-loop deployment. A negative result means a model did not improve this offline metric under this frozen protocol. It does not mean the model family is generally unusable.

6.1 World-model selection results

Branch 1 is the policy-only baseline every later branch in this chapter is measured against. The policy generates its candidate pool and selects by its own internal preference, with no world model involved (Section 4.2). Table 6.1 reports this baseline for both policies studied in this thesis. Translation follows the RMS-per-axis convention used throughout this chapter, not a true Euclidean distance; rotation is the mean geodesic angle in degrees. Later tables also report a 95% confidence interval alongside each delta, defined in Section 4.6.3.

Table 6.1 Branch 1 policy-only baseline results, no world model involved.

Policy	**Translation error**	**Rotation error**
Diffusion Policy (DP)	0.114	39.96°
Action Chunking Transformer (ACT)	0.143	41.48°

DP's baseline is the stronger of the two on both axes. ACT's baseline carries a caveat DP's does not. ACT's candidate pool collapses at inference time, so its five nominally independent candidates differ from each other by roughly 10^{-8} in absolute value (Section 5.4.1). Its Branch 1 number above is therefore closer to a single deterministic output evaluated five times than a genuine five-way selection baseline. ACT's guard results are reported separately in Section 6.2, using CEMP search to recover a usable candidate pool. Every delta reported below in this section is measured against DP's Branch 1 baseline above.

Figure 6.1 summarises the headline comparison across four of the five scorers on the DP row, as the reduction in translation error under Branch 2 and Branch 3. LeWM is held out of the figure and reported in Table 6.2 instead, for reasons the discussion below explains. The two branches share the same candidate pool and world-model scorer, and differ in one respect (Section 4.2). Branch 2's pool includes the policy's own preferred candidate, and the scorer picks freely among all five. Branch 3 removes that preferred candidate first, so the scorer must choose among the four remaining candidates. Reading the two branches together says more than either alone. If Branch 2 improves on the baseline but Branch 3 does not, the policy's own candidate was already close to the best available choice, and the scorer's main contribution is avoiding the cases where that candidate was wrong. If Branch 3 improves more than Branch 2, the policy's preferred candidate was actively misleading, and better options existed elsewhere in the pool. If neither branch improves on the baseline, the scorer is not distinguishing candidate quality at all. The following paragraphs work through all five scorers on this row, from the weakest result to the strongest, reading both branches together throughout.

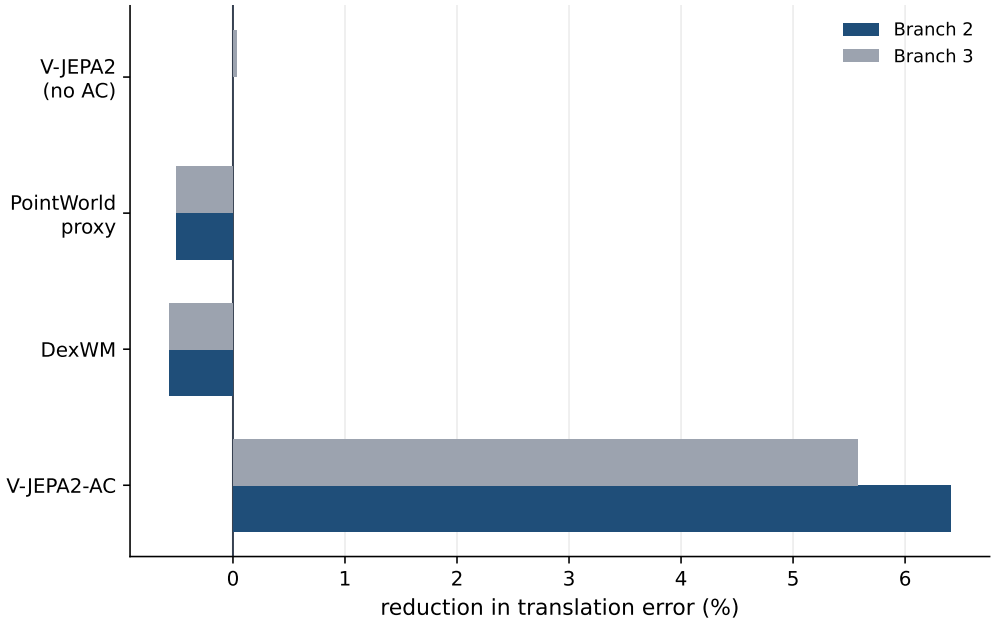


Figure 6.1 Reduction in translation error against the frozen DP candidate-0 baseline, by world model and branch. Positive values are an improvement over the policy-only candidate.

The zero-action ablation (V-JEPA2 no-AC) returns the baseline unchanged in Branch 2. It assigns tied scores and never overrides candidate 0. This is the expected outcome for a scorer with no action information to rank on. Our other proxy control, PointWorld, worsens translation while improving rotation. This is consistent with the degraded-conditioning design described in Section 5.5.7. It was built to reuse

V-JEPA2-AC's visual pathway with weakened action conditioning, specifically so that a mixed, non-improving result would confirm that action information, not visual quality alone, drives selection.

DexWM worsens translation error in both branches, and by a larger margin than the PointWorld negative control. This is a valid negative result for the implementation actually evaluated here. Section 5.5.3 describes the reproducible fixed-budget training route substituted for DexWM's original recipe, since public pretrained weights for its published EgoDex+DROID recipe were not available. This is a good place to state a caveat that applies to every scorer in this chapter, not only DexWM. This study is not designed to find the best possible result each architecture can reach. Every model family here is trained under the same universal exposure budget (Section 4.5.1), deliberately chosen so a comparison reflects architecture and candidate-selection ability, not who happened to receive more training. DexWM's original recipe trains at a substantially larger scale than this budget allows, so its result here is evidence about DexWM under this study's fair-budget implementation, not a ceiling on what a fully-scaled DexWM could achieve. The same caveat holds for every other result in this section, and is revisited in Chapter 7.

LeWM is held out of Figure 6.1 because its margin comes with a caveat none of the earlier scorers in this section needed, explained below and reported in its own table rather than plotted alongside a cleanly comparable result. LeWM predicts a future state through its own learned visual dynamics, the same role V-JEPA2-AC plays. LeWM's causal-diagnostic scorer weighs several terms together, and one of those weights acts like a knob for how much its own predicted future state counts toward the final score. Turned up, the knob is the intended design. It calls LeWM's image encoder, predicts a future state, and scores each candidate by how well that prediction matches it, described mechanically in Section 5.5.2. Before this is trusted, it has to pass a causal-diagnostic gate, a check for whether a candidate's score actually predicts whether that candidate turns out better.

At the full scale this chapter reports on, 7,607 held-out test windows, LeWM fails that gate. Its Spearman rank correlation (Section 4.6.3) between score margin and actual translation improvement is -0.507 (B2) and -0.518 (B3), essentially no relationship between what the scorer prefers and what actually turns out better. This is not a small-sample artefact. It gets worse, not better, at this larger scale than it did on a smaller diagnostic set used earlier to tune the gate. As a control, turning that knob all the way down disables LeWM's model entirely. The result is almost identical, and still fails the same check.

The honest summary of LeWM's result is this. Its scorer does pick candidates with lower translation and rotation error on average, by the largest margin in this chapter. But its own score does not reliably predict which candidate will turn out

better, at the scale this chapter reports on. Something other than the scorer's stated logic is driving the improvement, and disabling the model entirely does not change that outcome. LeWM is not validated as a causal mechanism here.

Table 6.2 reports both configurations' full-scale numbers together. One is the version that calls LeWM's model, and the other is the control that disables it.

Table 6.2 LeWM's two causal-diagnostic configurations, full 7,607-window test set, neither shown to be causally driven (Section 5.5.2).

Configuration	Branch	Translation change [95% CI]	Rotation change [95% CI]
LeWM rollout	B2	-10.104% [-10.534, -9.682]	-0.840° [-0.968, -0.713]
LeWM rollout	B3	-8.885% [-9.361, -8.419]	-0.727° [-0.865, -0.586]
LeWM action-prior	B2	-10.236% [-10.671, -9.816]	-0.849° [-0.972, -0.726]
LeWM action-prior	B3	-9.009% [-9.468, -8.549]	-0.741° [-0.885, -0.603]

Both configurations post the largest percentage improvements in this chapter, larger than V-JEPA2-AC's. That size is exactly why they cannot be trusted at face value. A more negative number here is a bigger improvement, but neither configuration's score is shown to causally track which candidate produces it, so this is the largest result in the chapter, not the best one. That is why LeWM is held out of the headline comparison rather than reported alongside the other four scorers.

V-JEPA2-AC is the strongest validated instance of the mechanism this thesis studies. It is an inference-time scorer that predicts a future outcome for each candidate and selects on that prediction. Its validity rests on the same test that just ruled LeWM out. Its Spearman rank correlation (Section 4.6.3) between score and each candidate's true translation error, measured directly per window rather than assumed, is real and well above zero at every pool size tested. Table 6.3 reports this correlation at four pool sizes, using the same nested-pool sweep described in Appendix B.13. It rises from 0.249 at $K = 2$ to 0.343 at $K = 20$. This rise with K is expected rather than a sign the scorer improves with more candidates. Spearman correlation over only two or three points is a coarse, biased estimate, and the same underlying ranking ability is measured more precisely as more candidates are added. At $K = 5$, the pool size used throughout this study, the correlation is 0.303, real and moderate, well short of perfect, but nowhere near the null result LeWM produced. This is why V-JEPA2-AC's percentage gains, unlike LeWM's, come from a score shown to track genuine forward prediction rather than an unvalidated causal claim.

The translation-change figures in this chapter show the reduction in error the world model's selection adds on top of what the policy alone would have produced, not the policy's absolute error level. Both Branch 2's -6.406% and Branch 3's -5.573% (also presented in the full DP table of results, Table 6.4) clear that bar. More precisely, Branch 2 wins on translation in 50.16% of windows, ties in 23.33%, and loses in

Table 6.3 V-JEPA2-AC's per-window Spearman rank correlation between score and true candidate translation error, by candidate pool size K, Branch 2, on the nested pools behind Appendix B.13.

$K = 2$	$K = 5$	$K = 10$	$K = 20$
0.249	0.303	0.330	0.343

26.50%. The tie rate equals its candidate-0-keep rate. This is why Branch 2 (-6.406%) beats Branch 3 (-5.573%). It falls back to the policy's own best sample when the world model is not helpful.

Set against what is achievable, this gain is real but modest. DP's Branch 1 baseline translation error is 0.114 (Table 6.1). At the fixed five-candidate pool used throughout this study, the best selection possible from that pool reduces this by 16.81% in Branch 2 and 14.98% in Branch 3. The oracle ceiling is a definition of how much error is fixable by choosing the best candidate among the ones DP already generated, something no real scorer can do without access to ground truth. V-JEPA2-AC reaches 38.1% of that ceiling in Branch 2 and 37.2% in Branch 3, meaning its selection recovers a bit over a third of what a perfect, ground-truth oracle could have recovered from the same five candidates. The rest of the baseline error, everything past the oracle ceiling, is not a selection problem at all. It is a property of the candidates DP generates, and no amount of world-model selection can reach below it without a better candidate pool to select from. This draws a clear line around what this thesis's mechanism can and cannot fix. That is, it can only re-rank what the policy already proposes, not raise the ceiling of what the policy is capable of proposing. Five is a compute-driven choice, not a ceiling. Appendix B.13's full ablation across pool sizes two to twenty shows scorer ranking is stable at every size tested, and the achievable gain grows steadily with pool size rather than plateauing, so the fixed pool of five understates the ceiling reported above rather than overstating it.

Table 6.4 gives the point estimates and episode-clustered 95% confidence intervals for every scorer on this row, including LeWM, which the figure above omits. Each delta is the selected candidate minus candidate 0, so negative values mean the guard selected a better trajectory than the policy-only baseline.

Attempting all three semantic world models also surfaces a comparison rooted in how each extracts and predicts features, rather than in the deltas above. V-JEPA2-AC, DexWM, and LeWM contrast fundamentally different representational strategies. These are video-pretrained transfer, image-pretrained transfer with geometric supervision, and self-supervised task-data learning from scratch, respectively (Section 3.2.3). This is a difference in backbone provenance, not a departure from the fair-budget protocol that governs this study's own training (Section 4.5.1): all three receive the same 100,000-item exposure budget for the training this study performs

Table 6.4 Primary DP guard results on the frozen 7,607-window EgoDex test set. CIs are episode-clustered. The LeWM row is not shown to be a causally validated result (Section 5.5.2).

Scorer	Branch	Translation change [95% CI]	Rotation change [95% CI]
V-JEPA2-AC	B2	-6.406% [-6.846, -5.980]	-1.008° [-1.131, -0.886]
V-JEPA2-AC	B3	-5.573% [-6.057, -5.104]	-0.882° [-1.025, -0.742]
V-JEPA2 no-AC	B2	0.000% [0.000, 0.000]	0.000° [0.000, 0.000]
V-JEPA2 no-AC	B3	-0.032% [-0.508, +0.434]	-0.148° [-0.288, -0.015]
PointWorld proxy	B2	+0.500% [+0.055, +0.936]	-0.393° [-0.521, -0.260]
PointWorld proxy	B3	+0.500% [+0.033, +0.974]	-0.382° [-0.529, -0.240]
DexWM	B2	+0.570% [+0.118, +1.020]	-0.051° [-0.187, +0.077]
DexWM	B3	+0.563% [+0.086, +1.037]	-0.084° [-0.230, +0.060]
LeWM	B2	-10.104% [-10.534, -9.682]	-0.840° [-0.968, -0.713]
LeWM	B3	-8.885% [-9.361, -8.419]	-0.727° [-0.865, -0.586]

on them. What differs is what each backbone already carries in before that budget starts. V-JEPA2-AC's ViT-Large backbone and DexWM's DINOv2 encoder are pretrained externally, on video and image corpora far larger than this study's own budget, before either ever sees this study's data. LeWM's ViT-Tiny backbone has no such external pretraining. It is trained entirely from scratch, inside the same 100,000-item budget that governs every other model here. The causal deltas above should therefore be read model by model, not as a head-to-head architecture ranking, since a from-scratch model and two externally-pretrained ones start this study's own fair-budget training from very different places, even though that training itself is held equal. Inference cost is the one dimension that is directly comparable regardless of this asymmetry. DexWM's DINOv2 encoder and CDiT predictor make it the heaviest of the three at planning time, consistent with its own measured raw-pool runtime (Table 6.7), the most detailed runtime comparison in this study, though reported there for the ACT row rather than this one. V-JEPA2-AC's frozen ViT-Large backbone with a lightweight transition head sits in between. LeWM's from-scratch ViT-Tiny is the cheapest by design, consistent with its reported 48× planning speedup; this study did not separately measure its raw-pool runtime to confirm that figure directly.

6.2 ACT guard results under CEMP

The two policies are not evaluated on equal footing, and this shapes how every later section should be read. DP produces five genuinely distinct candidates per window, so a world-model scorer has something real to choose between. ACT's candidate pool

does not. Its CVAE posterior collapses at inference time, and five sampled candidates differ from each other by roughly 10^{-8} in absolute value. This is a property of the checkpoint, not of the guard evaluator (Section 5.4.1, "CVAE posterior collapse: investigation and retraining"). To recover a usable pool for ACT, every guard result below instead uses CEMP, a cross-entropy search that perturbs the ACT baseline's own output over three iterations and keeps the four highest-scoring alternatives (Section 5.4.1, "CEMP candidate augmentation").

Each scorer builds and reranks its own CEMP pool, guided by its own predictions at every iteration. An ACT B2 or B3 result is therefore evidence for the world-model-plus-CEMP system as a unit, not an isolated measurement of scorer quality comparable to the DP row, exactly as Section 5.4.1's circularity caveat states. Branch 2 keeps the real ACT output as a fifth eligible candidate, so the world model can still confirm the policy's own choice if it scores highest. Branch 3 excludes it, so the world model must find something better among the four CEMP alternatives on their own, the same exclude-the-baseline rule the DP row uses for Branch 3 (Section 4.2), applied here to a searched pool instead of a fixed one.

Table 6.5 ACT guard results under CEMP candidate augmentation. Translation change is relative to the frozen ACT baseline; CIs are episode-clustered.

Scorer	Branch	Translation change [95% CI]	Rotation change [95% CI]
V-JEPA2-AC	B2	**-0.609%** [-0.677, -0.543]	-0.049° [-0.058, -0.040]
V-JEPA2-AC	B3	**-0.611%** [-0.680, -0.546]	-0.048° [-0.057, -0.039]
V-JEPA2 no-AC	B2	0.000% [0.000, 0.000]	0.000° [0.000, 0.000]
V-JEPA2 no-AC	B3	+0.406% [+0.348, +0.463]	+0.033° [+0.023, +0.043]
PointWorld proxy	B2	+0.503% [+0.442, +0.566]	-0.034° [-0.044, -0.022]
PointWorld proxy	B3	+0.503% [+0.442, +0.566]	-0.034° [-0.044, -0.022]
DexWM	B2	+0.263% [+0.186, +0.343]	+0.007° [-0.001, +0.015]
DexWM	B3	+0.263% [+0.184, +0.348]	+0.007° [-0.001, +0.015]

Table 6.5 reports four of the five scorers against this CEMP pool. LeWM is omitted and reported separately below, for the same reason it is held out of the equivalent DP-row figure (Section 6.1). Negative values are an improvement over the baseline; positive values are a regression.

PointWorld proxy and DexWM both post a positive translation change in this table, meaning their selections are worse than the ACT baseline on average, not better. PointWorld proxy's Branch 2 and Branch 3 results are identical, +0.503% in both, because its scorer never selects the ACT candidate in Branch 2 either, so excluding that candidate in Branch 3 changes nothing. As a negative control, its role is to show what a scorer with no real predictive signal produces under CEMP search. The result is a small, consistent regression rather than an improvement. Its translation change here,

+0.503%, is almost identical to its DP-row counterpart, +0.500%. Its rotation change is not: a small improvement of -0.034° here against a much larger -0.393° on the DP row. CEMP search narrows the candidate pool's diversity compared to DP's five independently sampled candidates, which plausibly explains why this row leaves less rotation error on the table for even a null scorer to stumble into. DexWM's translation change is a similarly small regression, +0.263% in both branches, and its rotation change does not clear the interval away from zero. This is consistent with the negative result already reported for DexWM on the DP row (Section 6.1), where its rotation change also did not clear the interval away from zero. The two rows agree that DexWM moves rotation error in neither a reliably better nor worse direction, even though the point estimates happen to have opposite signs. Unfortunately, the fixed-budget training route substituted for DexWM's original recipe (Section 5.5.3) does not translate into a working scorer here either.

LeWM's effect on this row is the largest in the chapter, translation improved by 2.159% in both branches, but it is not the strongest evidence of world-model prediction, which is why it is held out of Table 6.5 rather than listed alongside the other four scorers. As Section 6.1 explains in full, LeWM's causal-diagnostic scorer fails its own validation gate at the scale this chapter reports results on, even with the weight that lets it invoke its own learned dynamics turned all the way up. The margin is real, but the score behind it is not shown to causally track which candidate is actually better (Section 5.5.2).

Table 6.6 LeWM's two causal-diagnostic configurations on the ACT/CEMP row, full 7,607-window test set, neither shown to be causally driven (Section 5.5.2).

Configuration	Branch	Translation change [95% CI]	Rotation change [95% CI]
LeWM rollout	B2	-2.159% [-2.244, -2.077]	-0.031° [-0.045, -0.017]
LeWM rollout	B3	-2.159% [-2.243, -2.077]	-0.031° [-0.045, -0.017]
LeWM action-prior	B2	-2.150% [-2.235, -2.068]	-0.031° [-0.044, -0.016]
LeWM action-prior	B3	-2.150% [-2.235, -2.068]	-0.031° [-0.044, -0.016]

These two configurations suffer the same caveats already described for the DP row's own held-out table (Table 6.2), and the same trend appears here too. Their translation and rotation changes are close enough to be indistinguishable, and neither configuration's score is causally validated. Directly checking the Spearman rank correlation between score margin and translation improvement, the same test applied to the DP row, gives $\rho = -0.455$ for the action-prior configuration and $\rho = -0.466$ for the rollout configuration. Both are strongly negative and close to each other. This is consistent with the DP-row finding that turning LeWM's own model on or off barely changes the outcome. The score never tracked outcome quality in the first place.

Given that LeWM's margin cannot be read as a validated guard pass, V-JEPA2-AC is this row's actual headline result. Its translation change is smaller, **-0.609%** in Branch 2 and **-0.611%** in Branch 3, but both intervals distinguish clearly from zero, and it is the only scorer on this row whose score is validated to track genuine forward prediction rather than an action-distribution regularity. Rotation improves too, by a small but consistent margin in both branches. This is the one result on this row that can be stood behind as a working world-model guard. It follows the same analysis given for the DP row above (Section 6.1): the direction of the effect matches exactly, an improvement in both translation and rotation, in both branches. The magnitude is far smaller here, -0.61% against -6.41% in Branch 2, consistent with this row's secondary, corroborative status rather than a differing finding.

V-JEPA2 no-AC serves as a second control. Its scorer cannot distinguish candidates by action content, so every candidate in Branch 2's pool scores identically, and ties resolve toward the ACT baseline, kept 100% of the time. Branch 3 forces a choice among scores that are still effectively tied, and the result comes out marginally worse than the baseline on both translation and rotation. This is the expected null result for a scorer with no action information to rank on. It confirms that action conditioning, not CEMP search alone, is doing the work in the other rows, and it makes V-JEPA2 no-AC the only scorer on this row that keeps the ACT candidate anywhere close to 100% of the time rather than close to never. Its Branch 3 translation change here, +0.406%, is a genuine small regression whose interval excludes zero, unlike the DP row's Branch 3 result for the same scorer, +0.032% improvement, whose interval does not. CEMP's perturbations around the ACT baseline are not the exchangeable, independently sampled candidates DP provides, so a scorer with no real signal is not guaranteed to land on the same null result in both rows, only on the same underlying lack of signal.

Therefore, every other scorer keeps the ACT output close to never, roughly 0% of windows in Branch 2. This is not comparable to the DP row's candidate-kept rates, and should not be read as the world model strongly disagreeing with the policy in the same sense. CEMP explicitly searches for the candidate that scores highest under the same objective the scorer then uses to select, so it is expected to beat a fixed, unoptimised ACT baseline on that same metric most of the time. The low kept rate across this row is a mechanical consequence of pairing a search procedure against a fixed baseline, not a finding about how much the world model and the policy disagree.

Table 6.7 reports the compute cost of CEMP against the raw pool it replaces, Branch 2 only for clarity. Branch 3 timings follow the same pattern in every row. Every scorer costs more, since scoring 64 perturbations per iteration across three iterations plus the final pool means roughly 197 candidates per window instead of five. For the four lightweight scorers this is affordable, tens of seconds of extra runtime across the

full test set. DexWM's scorer is a full transformer over image patch tokens, so the same 40-fold increase in candidate count is a 40-fold increase in genuine forward-pass compute, not an overhead cost that could be optimised away. This cost was accepted rather than reducing CEMP's search budget for DexWM alone, to keep the procedure identical across scorers. LeWM rollout's raw-pool cell is empty because no raw, non-CEMP run of that configuration was ever launched on this row. Only its CEMP result was needed for the reported finding, so the direct comparison point was not collected.

Table 6.7 Guard runtime, raw pool against CEMP, full 7,607-window test set, Branch 2 only. Branch 3 timings follow the same pattern.

Scorer	Raw pool	CEMP
V-JEPA2-AC	43.7s	118.3s
V-JEPA2 no-AC	41.7s	113.1s
PointWorld proxy	42.6s	106.7s
LeWM rollout	—	1,828.5s
DexWM	7,693s	169,873s (≈47.2h)

Every result in this section is secondary to the DP row reported above. That secondary status is not incidental. It follows directly from ACT's CVAE posterior collapse (Section 5.4.1): with no genuinely diverse candidate pool to select from, this row only exists at all because CEMP search stands in for the candidate diversity DP provides naturally. The scorers rank the same way here as they did in Section 6.1, and that agreement is worth having, but the CEMP-augmented ACT row does not carry the same evidential weight. It reoptimises the candidate pool against a fixed baseline that a search procedure is expected to beat almost by construction, not against a fixed pool the policy never saw. None of the numbers in Table 6.5 should be read as validating or invalidating a world model on their own. Their role is to reinforce the DP row's ranking of scorers, not to stand as a main result in their own right.

6.3 Selection quality across task difficulty

Section 6.1 reports capture as flat across pool size when aggregated over the full test set, but a single flat number averaged over every window can hide a serious local failure. This section checks for that directly. The canonical DP Branch 2 test set is sorted by baseline (candidate-0) error, the error the policy already makes before any world-model selection, and split into five equal-sized quintiles. Q1 is the fifth of windows where the policy's own uncorrected output was already most accurate, and Q5 is the fifth where it was least accurate. Capture, the percentage of the oracle ceiling

recovered (Section 6.1), is then computed separately within each quintile rather than pooled across all of them, so a failure confined to one difficulty band is visible instead of being averaged away. This is done at $K = 8$ rather than the canonical $K = 5$, and for both window regimes used elsewhere in this thesis: S1, the dense sampling used everywhere else in this chapter, and S2, a state-change-selected alternative windowing (Section 4.1.2).

Table 6.8 V-JEPA2-AC capture (percentage of the oracle ceiling recovered, Section 6.1) by baseline-difficulty quintile, $K = 8$, Branch 2. A negative value means the selected candidate is worse than candidate 0, not merely less close to the oracle than a perfect selector would be.

Quintile	S1 capture	S2 capture
Q1 (easiest)	**-30.48%**	-0.81%
Q2	25.73%	41.46%
Q3	42.26%	56.47%
Q4	51.56%	65.36%
Q5 (hardest)	50.42%	63.91%
All	37.41%	53.10%

On the easiest fifth of windows, selection reverses direction (Table 6.8). Translation error rises by 5.38% relative to keeping candidate 0, a capture of -30.5%, rather than falling as it does on every other quintile. This asymmetry is broadly consistent with how much headroom each quintile leaves for a re-ranking step. Q1 is defined as the fifth with the lowest baseline error, so candidate 0 is already close to the best the policy can produce there; there is little for selection to gain by overriding it, and a comparatively larger relative cost when it overrides it wrongly. On the harder quintiles the baseline is further from optimal, so an incorrect override costs proportionally less and a correct one recovers substantially more error, which is why capture climbs from Q2 through Q4 and holds roughly flat into Q5. Only Q1 inverts. The aggregate -6.406% reported in Table 6.4 therefore conceals a regime in which the selection mechanism performs worse than not selecting at all.

A natural response to a scorer that fails specifically on its easiest cases is to make it more cautious. Only trust its selection when it is confident, and fall back to the policy's own candidate otherwise, the selective-prediction approach of [63]. Confidence here is operationalised as the score gap, how far apart the top-ranked and second-ranked candidate's scores are. A large gap means the scorer strongly prefers one candidate over the rest; a small gap means it is nearly indifferent between its top choices. Abstaining on the fraction of windows with the smallest gaps, meaning falling back to the policy-only candidate on those windows instead of trusting the scorer, does not recover the Q1 loss at any threshold or pool size tested. It erodes the

aggregate gain instead.

Table 6.9 Effect of gap-based abstention on the aggregate translation-error change.

Abstain fraction (lowest gap)	$K = 5$	$K = 8$	$K = 20$
0% (as run)	-6.504%	-7.583%	-9.620%
10%	-6.129%	-7.014%	-8.849%
30%	-5.132%	-5.785%	-7.139%

Table 6.9 gives the aggregate effect at three abstention thresholds and three pool sizes. The aggregate loss shrinks monotonically as the abstained fraction grows, for example from -7.583% at 0% abstention to -5.785% at 30% for $K = 8$, but this is not evidence that the gate is finding and removing the Q1-style regressions specifically. Abstaining on a larger fraction of windows simply falls back to candidate 0 more often across the whole test set, diluting both the gains and the losses uniformly rather than targeting the losses alone. Abstention loses aggregate gain steadily across the whole test set rather than selectively removing the Q1-style regressions it was meant to catch. The reason is that the score gap carries essentially no information about whether a selection will be correct. The Spearman correlation (Section 4.6.3) between the gap and the resulting baseline error is -0.0044, indistinguishable from zero. A wide gap does not mean the scorer's confident pick is more likely to be right, and a narrow gap does not mean it is more likely to be wrong. Gating on the maximum score itself performs no better. The selector therefore has no selection-time signal, available from the candidate scores alone, that identifies when it is in the loss-making Q1 regime. The Q1 failure is not a confidence-calibration problem a wrapper can fix from the candidate scores alone. It is reported here as a limitation of the selection mechanism itself, not a solved problem.

6.4 Comparison against DiWA on CALVIN

The primary results above are all measured on a frozen offline metric on EgoDex, so this section asks a different question. It checks whether the same underlying idea, choosing better actions with the help of a world model, reproduces on the benchmark and protocol that motivated it in the first place. Chandra et al.'s DiWA [12] pairs a diffusion policy with a frozen world model and reward classifier on CALVIN, and uses PPO finetuning against imagined rollouts rather than the inference-time selection used everywhere else in this chapter. This section reproduces DiWA's own protocol on four gated tasks, using the authors' own checkpoints throughout, as a secondary study rather than a like-for-like extension of the primary results. It reports five conditions,

labelled C1 through C4b, each combining DiWA's own policy and world model differently.

C1 is DiWA's own base policy acting alone, using DiWA's own checkpoint, with no world-model involvement at all. This is the same role Branch 1 plays throughout this thesis's own EgoDex results. C2 keeps that same policy and DiWA's own frozen world model and reward classifier completely unchanged. The only thing added is this thesis's own test-time selection harness, choosing among several base-policy samples rather than always taking the first one. Neither condition involves any finetuning or weight update. Table 6.10 gives the per-task task-success rate for both.

Table 6.10 DiWA base-policy comparison on CALVIN, no finetuning. C1 is DiWA's own base policy alone; C2 adds this thesis's test-time selection harness on top of DiWA's own frozen world model, with no weight update in either case. Change is in percentage points (pp), the absolute difference between the two percentages, not a further relative change.

Task	**C1**	**C2**	**Change**
close_drawer	59.48%	59.48%	0.00pp
turn_on_led	57.76%	56.03%	-1.72pp
move_slider_left	57.03%	58.59%	+1.56pp
turn_on_lightbulb	71.21%	71.97%	+0.76pp

C2 is not a diminished or specially-weakened version of this thesis's selection method. It draws a pool of 5 candidates, the same $K = 5$ used throughout the primary EgoDex results (Section 6.1), and ranks them with the same probability-based selection procedure used everywhere else in this chapter. Before trusting this comparison, it is also worth checking that DiWA's diffusion policy gives selection something real to choose between, the way ACT's candidate pool famously does not (Section 5.4.1). Reading the reward classifier's own per-decision scores directly, it picks a candidate other than the first one on 79 to 81% of decisions across all four tasks, with a meaningful spread across the five candidate scores at every decision, not the near-identical, undifferentiated candidates ACT's collapsed CVAE posterior produces. A diffusion policy also has no CVAE bottleneck to collapse in the first place, so this is not a coincidence specific to this checkpoint.

The change is nonetheless small and mixed rather than consistently positive. `turn_on_led` moves backward by 1.72 percentage points under selection, `move_slider_left` and `turn_on_lightbulb` move forward by 1.56 and 0.76 points, and `close_drawer` does not move at all. Two features of this benchmark make that unsurprising without implying selection is not working. Task success on CALVIN is binary. A candidate only registers as an improvement if it is good enough to flip an entire episode from failure to success, unlike the continuous translation and rotation

errors the primary EgoDex results are measured on, where a partial improvement still shows up in the metric. Each task is also pooled over only 116 to 132 episodes here, roughly two orders of magnitude fewer than the 7,607-window EgoDex test set, so a true effect of this size would be difficult to distinguish from noise even if selection were working exactly as intended. The point of this table is not to show selection working on its own on this particular benchmark. It is to establish a clean pre-finetuning baseline, so that whatever change C3 shows after finetuning can be attributed to the finetuning step itself, not to an effect that was already present from selection alone.

C3 repeats DiWA's own imagined-PPO finetuning procedure for 800 iterations per task, evaluated against the real CALVIN simulator every 25 iterations. An initial run left the behaviour-cloning regularisation coefficient α_{BC} at zero for every task. `move_slider_left` and `turn_on_lightbulb` collapsed under this setting. DiWA's published hyperparameter table (Chandra et al., Table S.5, Appendix S.1.4) sets $\alpha_{BC} = 0.05$ by default and gives a smaller value of $\alpha_{BC} = 0.025$ for `close_drawer` and `turn_on_led`. Rerunning `move_slider_left` and `turn_on_lightbulb` at the published default of $\alpha_{BC} = 0.05$ fixed both. `close_drawer` and `turn_on_led` were left at $\alpha_{BC} = 0$, below the published value for either task, and converged without collapsing anyway. The behaviour-cloning coefficient α_{BC} mattered exactly where the paper's default applies, and turned out not to be needed where the paper recommends a smaller value. Table 6.11 reports the corrected run for all four tasks, alongside a zero-shot evaluation of DiWA's own already-finetuned checkpoints as a reference point for what the recipe should achieve.

Table 6.11 DiWA imagined-PPO finetuning on CALVIN, corrected per-task hyperparameters.

Task	Base	After finetuning	DiWA checkpoint reference
close_drawer	72.41%	100.00%	93.1%
turn_on_led	58.62%	82.76%	82.8%
move_slider_left	56.25%	84.38%	84.4%
turn_on_lightbulb	66.67%	96.97%	90.9%

All four tasks reproduce DiWA's result once the correct per-task hyperparameter is applied, matching the authors' own reference checkpoints to within a fraction of a percentage point on every task. `close_drawer` and `turn_on_lightbulb` land clearly above the reference; `turn_on_led` and `move_slider_left` land a fraction of a point below it, well inside the noise of a single held-out episode at this evaluation scale. None of the four tasks should be read as landing reliably above or below DiWA's own checkpoints.

An exact match was never the expectation. This reproduction reruns DiWA's training and evaluation code end to end, on this thesis's own hardware and random

seeds, so small differences from stochastic training and evaluation are normal rather than a sign of a flawed reproduction. What matters is how close the match is. Four independent tasks would not land this close to the reference if the reproduction's policy, world model wrapper, PPO loop, and evaluation harness did not genuinely match DiWA's own implementation. Simply copying DiWA's published numbers into Table 6.11 would leave nothing to explain here, and no reason to expect four close-but-not-identical results rather than four exact ones. This closeness is evidence that every condition reported in this section, C1 through C4b, sits on a correctly implemented reproduction of DiWA's own pipeline, which is what gives any comparison between them its meaning. It validates this reproduction specifically, not this thesis's own five-world-model pipeline evaluated elsewhere in this chapter, since every condition here uses DiWA's own policy and world model throughout.

Two further limits are worth stating plainly. Each finetuned result after iteration 800 is a single checkpoint rather than an average over a pooled test set, so it carries more variance than the C1 and C2 numbers. The initial misconfiguration is also a reminder that DiWA's own reported robustness depends on a hyperparameter that is not shared across tasks, and is easy to miss without checking the appendix of the source paper.

C2 and C3 were not designed as a like-for-like comparison. C2 measures pure test-time selection against C1's baseline. C3 measures the effect of finetuning against its own separately-evaluated starting point, whose numbers do not exactly match C1's, a reminder that C3's checkpoints, unlike C1 and C2, are single evaluation runs rather than pooled estimates. Read together anyway, the contrast in the size of the effect is stark. C2's change over C1 tops out at 1.56 percentage points in either direction. C3's change over its own base is 25 to 41 percentage points on every task, `close_drawer` +40.52pp, `turn_on_led` +26.73pp, `move_slider_left` +25.79pp, `turn_on_lightbulb` +25.00pp. On this benchmark, with DiWA's own policy and world model, weight-level finetuning against imagined rollouts delivers roughly an order of magnitude more improvement than test-time selection alone. This is a narrower claim than a controlled ablation would support, since C2 and C3 differ in more than just whether the policy was finetuned, but the size of the gap is large enough that it is unlikely to be explained by anything other than the finetuning step itself. This is the section's clearest conclusive comparison. Using the world model to reshape the policy's own weights, DiWA's own approach, beats using it only to select among the policy's already-generated outputs, this thesis's own approach, by roughly an order of magnitude on this benchmark, and consistently across all four tasks.

C2 and C3 do not, on their own, say anything about whether the two approaches could be combined, that is, whether test-time selection still adds anything once weight-level finetuning has already happened. This was part of what this

reproduction was scoped to look at from the start, so two further, smaller conditions test it directly. Both are reported here for completeness rather than as conclusive findings in their own right, and both are labelled as variants of a single follow-up question, C4a and C4b, rather than as standalone numbered conditions on the same footing as C1 through C3.

C4a applies the same selection harness used in C2, unchanged, to the already-finetuned C3 checkpoint at iteration 800. Table 6.12 gives the result.

Table 6.12 C4a: this thesis's test-time selection harness applied to the finetuned C3 checkpoint. Change is relative to C3's own after-finetuning result in Table 6.11.

Task	C3 after finetuning	C4a (C3 + selection)	Change
close_drawer	100.00%	97.41%	-2.59pp
turn_on_led	82.76%	90.52%	+7.76pp
move_slider_left	84.38%	83.59%	-0.79pp
turn_on_lightbulb	96.97%	92.42%	-4.55pp

There is no consistent direction here. `turn_on_led` favours adding selection on top of finetuning; the other three tasks favour finetuning alone, by amounts of a similar scale. With four tasks and one checkpoint each, this is not enough evidence to conclude that selection helps, hurts, or does nothing once a policy is already finetuned, only that none of the four changes is large. The honest reading is that this comparison, unlike C2 against C3 above, does not support a conclusion either way.

An earlier version of this measurement did show selection actively harming the finetuned policy, traced at the time to a distribution shift between the reward classifier's training distribution and the finetuned policy's own candidate distribution. That version had a bug. Its eval configuration reused a template whose denoising schedule setting routed every step through the frozen base actor rather than the finetuned weights, so it was silently re-measuring C2 under a label that said C3. The distribution-shift explanation was still worth testing directly once the bug was fixed, since a real mismatch between what the classifier was calibrated on and what it is asked to rank remains plausible in principle. C4b retrains DiWA's own reward classifier on episodes rolled out from the finetuned C3 policy itself, then reruns C4a's harness with the recalibrated classifier in place of the original. `close_drawer` could not be included, not as a gap in the data but because its finetuned policy succeeds on every episode in its own rollout data, leaving no failure case to train a classifier against. This is also why C4a and C4b report on different numbers of tasks, four and three respectively, and is expected given how strong `close_drawer`'s own finetuned result already is.

C4b does not show a consistent direction either. One task improves, one is unchanged, one worsens slightly. This is not read as confirming or refuting the distribution-shift explanation in general, only as showing that it is not doing detectable

Table 6.13 C4b: C4a's selection harness with the reward classifier recalibrated on the finetuned policy's own rollouts. `close_drawer` excluded, its finetuned rollouts contain no failures to train against.

Task	C4a (uncalibrated)	C4b (recalibrated)	Change
turn_on_led	90.52%	87.93%	-2.59pp
move_slider_left	83.59%	83.59%	0.00pp
turn_on_lightbulb	92.42%	96.21%	+3.79pp

work in this particular, corrected comparison. Each recalibrated classifier is also trained on 150 to 420 rows drawn from a single 29-to-33-episode rollout with no held-out validation split, and every one reaches perfect training precision and recall, a pattern more consistent with memorising a small, non-generalising dataset than with a genuinely recalibrated scorer. C4b is reported as a directional probe, not a validated improvement.

The conclusive result of this section remains C2 against C3. Deeper integration of a world model into the policy's own weights outperforms using it only as an external selector, consistently, by a large margin. C4a and C4b touch the surface of a real further question, whether the two approaches combine, conflict, or become redundant once weight-level integration has already happened, but neither shows a pattern strong or consistent enough to answer it. They are included because they were part of what this reproduction was scoped to explore, not because they settle anything. That question, and the broader one it points to, how a world model's influence on a policy might be designed to combine test-time selection with training-time integration rather than treating them as alternatives, is future work. This section's contribution is reproducing DiWA's own result faithfully and defining these follow-up conditions clearly enough for that future work to build on, not answering the question itself.

There are three distinct ways a world model is used across this thesis and DiWA, worth naming explicitly since CEMP (Section 6.2) sits between the other two. DiWA's world model is training-time and internal. PPO differentiates through imagined rollouts to update the policy's weights directly, so the world model's judgement is baked into the policy itself. The DP row's world-model scorers (Section 6.1) are the opposite extreme, inference-time and fully external. They only rank candidates the policy has already generated, with no perturbation and no gradient fed back into the policy at all. CEMP sits in between. It is still inference-time and external, never touching ACT's weights, but instead of only ranking existing samples it actively searches, perturbing the frozen policy output and asking the world model to rank the perturbations, all after training has finished. This is why CEMP works around ACT's CVAE posterior collapse (Section 5.4.1) rather than repairing its root cause. It compensates for a broken sampler at decision time instead of fixing the policy that produced it. DiWA's approach would

arguably be the more direct fix, retraining ACT under a world-model-augmented objective to correct the collapse itself. That is future work, not something attempted here.

This section is not merged into the primary comparison in Section 6.1. CALVIN is a simulated, binary-success benchmark using DiWA's own policy and world model, evaluated by finetuning weights rather than by inference-time selection, so the two studies share no comparable axis. It stands as secondary supporting evidence that world-model-guided improvement generalises beyond the primary evaluation, not as a second data point in the same comparison.

7 Discussion and Limitations

7.1 What the benchmark shows

The final benchmark gives a narrower and more defensible conclusion than the earlier project plan. World-model guidance can improve offline trajectory selection, but the positive semantic result is concentrated in Diffusion Policy candidates scored by V-JEPA2-AC. Branch 2 reduces first-step translation error by 6.41% and rotation error by 1.01 degrees relative to the policy-only candidate. Branch 3 also improves both metrics, but less strongly. This supports the policy-first version of the guard framework this thesis proposes. Keep the policy candidate available, then use the world model to choose among diverse policy-generated alternatives.

Section 1.2's research question asks whether a frozen world model, acting only as the final decision stage over candidates a policy already proposes, improves the selected motion chunk. The evidence above answers this directly. Under the tested pairing of DP candidates and V-JEPA2-AC, it does, by a margin that survives every causal check applied in this chapter. That question was deliberately asked wide before any evidence came in. Five world-model families, V-JEPA2-AC, LeWM, DexWM, DINO-WM, JEPA-WM, and the PointWorld proxy control, were evaluated against two structurally different policies, ACT and DP, not because every family was expected to succeed, but because answering ”do latent world models improve selection” honestly means testing more than one candidate world model and letting the evidence decide which pairing works, rather than committing in advance to a single architecture. One family succeeding cleanly and the other four each failing for a different, diagnosed reason is the answer this thesis set out to find, not a shortfall against a target of five positive results.

The other four families each carried a different, diagnosed shortcoming rather than a uniform failure to work. Section 7.3 works through each in turn, architecture, result, and why, rather than repeating that detail here.

This creates a useful scientific result. World-model-guided selection works when three conditions align. The candidate pool is diverse, the scorer receives meaningful action-conditioned information, and the candidate actions remain compatible with the scorer's training support. When one of these conditions fails, the world-model label alone is not enough.

This thesis's approach to a non-working case is diagnostic rather than dismissive, and that approach is itself part of what this chapter contributes. ACT's own candidate pool is degenerate at inference time (Section 5.4.1), which would ordinarily leave no basis for a world-model comparison on that policy at all. Rather than exclude

ACT from the benchmark, this thesis built CEMP specifically to recover a usable, searched candidate pool around its frozen output, then reran the same five world-model scorers against it (Section 6.2). Those results are reported as evidence for the world-model-plus-CEMP system as a unit, not promoted to the same footing as the DP row's primary claim, but they still let the same five-family comparison run on a second, structurally different policy rather than being abandoned when the first policy's own candidates turned out unusable. LeWM received the same treatment on its own terms, tested through two distinct configurations rather than a single reported number, detailed in Section 7.3. Testing a further configuration or a workaround when the first result is null, rather than writing the model off, runs through every non-positive result in this chapter.

[DISCUSSION PLACEHOLDER: DiWA/CALVIN comparison (Section 6.4). Waiting on the C3 fairness reruns; will give an updated interpretation once they land.]

7.2 Why Branch 2 outperforms Branch 3

Branch 2 and Branch 3 share the same candidate pool and world-model scorer, and differ only in whether the policy's own preferred candidate remains available to select (Section 6.1 sets out the three-way interpretive logic this comparison relies on in full). Branch 3 is best read as a check on Branch 2's judgement, not a separate, higher-authority condition competing with it. Removing candidate 0 does not give the world model anything extra to work with, it only removes the option of falling back to the policy's own choice. In the final DP plus V-JEPA2-AC result, Branch 2 improves translation by 6.41%, Branch 3 by 5.57%. Branch 2's edge is practical rather than a sign Branch 3's judgement is worse. Branch 2 keeps candidate 0 on 23.3% of windows, cases where the policy's own candidate is already competitive and the world-model score does not justify changing it, not failures of the selection mechanism. Branch 3 has no such fallback and must choose from candidates 1 to 4 even when candidate 0 was the safer option.

Read this way, Branch 3 is doing real work rather than merely losing to Branch 2. Forced to choose without candidate 0 available at all, it still improves on the baseline by 5.57%, confirming the world model's judgement is genuine rather than dependent on always having the policy's own preferred output within reach. The result favours a fallback deployment shape over an unrestricted one, not evidence that the world model's independent judgement is unreliable when tested without a safety net.

7.3 Interpreting the model-family differences

The model-family comparison should be read as an evidence hierarchy, not as a simple leaderboard. Each family is summarised here in outline, architecture, result, and why, referencing Chapter 3 for full architectural description and Chapter 6 for the underlying evidence rather than repeating either in full.

V-JEPA2-AC (Section 3.2.3) is the only family that gives a clear positive semantic world-model result on the headline DP evaluation (Table 6.4), and its own architecture is the most direct explanation why. Its frozen ViT-Large encoder is pretrained on over one million hours of internet video, so it carries genuine, video-specific temporal priors before this study trains anything on top of it. The only component trained within this thesis's fair exposure budget (Section 4.5.1) is a lightweight action-conditioned head. The zero-action ablation, V-JEPA2 no-AC, removes only the action-conditioning input from the same pretrained backbone and collapses to the baseline in Branch 2 (Table 6.4), confirming the action-conditioned head, not the pretrained visual features alone, is doing the work that produces the positive result.

DexWM (Section 3.2.3) offered the clearest opportunity for a genuine comparison between two pretrained backbones, and its own shortfall is a caveat about checkpoint availability rather than architecture. Like V-JEPA2-AC, it is built on a frozen, externally pretrained backbone, DINOv2 rather than a video encoder, feeding a CDiT predictor. Unlike V-JEPA2-AC, that predictor is the substantial, published component of the architecture, and the officially reported EgoDex+DROID-pretrained predictor checkpoint was never released. This thesis's evaluated DexWM therefore substitutes a predictor trained from scratch under the same fair exposure budget as every other family, unlike V-JEPA2-AC's head, which was always a small, from-scratch component by original design. Reproducing DexWM's own full-scale pretraining recipe independently of the missing checkpoint was also not a reasonable option within this thesis's budget. Even at the same fixed exposure every family trains under, DexWM is already by far the most computationally expensive family to train, $\approx$28 seconds per step against LeWM's $\approx$0.047 (Table 5.1), so attempting the officially reported EgoDex+DROID pretraining scale on top of that was a compute cost this thesis deliberately did not take on, not only a checkpoint it could not obtain. Under this fixed-budget route, DexWM does not improve translation error in either branch (Table 6.4), a valid negative result for the implementation actually evaluated, not for the architecture at its intended pretraining scale.

LeWM (Section 3.2.3) is investigated through two distinct configurations rather than a single reported number, specifically to isolate which part of its mechanism, if any, is doing real work. The rollout configuration genuinely invokes LeWM's own learned visual dynamics; the action-prior configuration turns that mechanism off and

scores only action-distribution regularity. Table 6.2 reports both. Neither passes the causal-diagnostic gate at the full 7,607-window scale (Section 6.1), so LeWM's visual-rollout claim is not validated as a semantic world-model result. The narrower, valid result is that training-only action regularity, on its own, still improves frozen DP candidate selection, useful information but a smaller and different claim from a working learned-dynamics scorer.

DINO-WM and JEPA-WM (Section 3.2.3) are official-checkpoint routes rather than models trained within this study, one paired with DexWM's DINOv2 backbone family and the other with V-JEPA2-AC's predictive branch (Figure 3.3). Both expect a 7-dimensional DROID-style action interface incompatible with this study's 18-dimensional bimanual wrist chunks, so each is evaluated only through an adapter, in a validation-only pilot rather than a held-out claim (Section 5.5.4, Appendix A.5). The pilots show a model can improve its own latent support score while failing to improve the trajectory metric this thesis actually measures, a further, concrete instance of the action-compatibility boundary this section describes throughout.

The PointWorld proxy control is not an evaluation of Huang et al.'s own point-cloud architecture (Section 3.2.3), which reasons over rigid-arm forward kinematics structurally incompatible with continuous $\mathrm{SE}(3)$ wrist transforms from human hand demonstration. Instead it reuses V-JEPA2-AC's own visual pathway with deliberately weakened action conditioning (Section 5.5.7), a control built specifically to separate the contribution of visual quality from the contribution of action information. Its mixed result, worse translation but better rotation (Table 6.4), is consistent with that design. Visual quality alone is not sufficient to reproduce V-JEPA2-AC's positive result.

This reinforces the central point running through every family above. World models must be action-compatible and metric-aligned before they can become useful evaluators.

7.4 Reproducibility and open model releases

The DexWM investigation raises a broader reproducibility issue. Open-source code for recent world-model systems is valuable. It allows independent researchers to inspect architectures, reuse data loaders, and understand how a result was constructed. That openness matters in a field where increasingly capable AI systems risk being concentrated among organisations with unusually large compute and data resources.

At the same time, code alone is not equivalent to a reproducible model release when the published claim depends on large-scale pretraining. In the DexWM case, the architecture and data recipe are available, but the pretrained checkpoint for the reported model was not. Reproducing the full recipe would require large DROID

raw-data onboarding, preprocessing, storage, and compute. For a Master's thesis with a fixed fairness contract, that is not a reasonable prerequisite for checking whether the model works as a selector under this thesis's protocol.

The point is not that incomplete releases have no value. They do. The point is that benchmark papers and downstream theses must distinguish between open code, open data, open checkpoints, and fully reproducible evidence. These are different levels of openness, and they lead to different claim boundaries.

7.5 Why benchmark evolution is part of the contribution

A major contribution of this dissertation is the construction of a defensible evaluation surface. Several earlier results looked stronger but were not clean enough for final claims. Some used test-derived support. Some allowed future-goal access. Some used inconsistent candidate generation. The final result set is weaker in headline size but stronger academically because it removes these shortcuts.

This matters because the thesis is not only asking whether this framework can produce a large number. It is asking whether world-model involvement can be evaluated fairly as an inference-time selection mechanism. The final protocol fixes the candidate pool, separates training support from test windows, blocks future-frame access, and reports negative outcomes when the evidence does not support a claim. That process is part of the scientific contribution.

7.6 Why semantic alignment and object grounding were excluded

Semantic alignment and object grounding remain useful implemented modules, but they are excluded from the final benchmark claims. Adding them only to V-JEPA2-AC would give one model family privileged inputs not available to LeWM, DexWM, or the controls. That would change the question from whether the world model improves selection under a shared protocol to whether extra semantic information helps a specific model.

A future experiment can test semantic conditioning directly, but it should do so symmetrically. Either every compared family receives equivalent semantic inputs, or the experiment should be labelled as an ablation of the V-JEPA2-AC path alone.

7.7 Main limitations

The first limitation is scope. The benchmark is offline and trajectory-level. It measures how closely selected wrist-motion chunks match expert demonstrations. It does not execute the action in a simulator or on a robot, and it does not measure task success.

The second limitation is candidate diversity. The ACT results show that a valid evaluation can still be uninformative if the candidate generator does not produce meaningfully different actions. This mechanism needs both a scorer and a useful candidate set.

The third limitation is model coverage. LeWM visual rollout, DexWM full upstream-scale pretraining, DINO-WM, JEPA-WM, and DiWA/CALVIN all reached bounded implementation states but not headline evidence states. These routes are valuable because they define limits, but they do not become additional positive claims.

The fourth limitation is metric coverage. First-step translation and rotation errors are meaningful for offline imitation fidelity, but they are incomplete. They do not measure grasp stability, finger articulation, object interaction, or long-horizon recovery after an error.

The fifth limitation is that selection quality is not uniform across the test distribution. Section 6.3 shows the V-JEPA2-AC selector performs worse than not selecting at all on the fifth of windows where the policy's uncorrected output is already most accurate, and that this regime cannot currently be identified at selection time from the candidate scores alone. The aggregate gain reported for DP Branch 2 is therefore a net figure that includes this loss-making subset, not evidence that selection helps uniformly across every input.

8 Conclusion

8.1 Summary of findings

This dissertation asked whether world models can improve action selection for complex bimanual hand-motion prediction at inference time. The final answer is conditional. World-model guidance helps in the strongest clean setting tested: Diffusion Policy candidates scored by V-JEPA2-AC. Branch 2 reduces first-step translation error by 6.41% and rotation error by 1.01 degrees on the held-out EgoDex test set. Branch 3 also improves both metrics, but less strongly.

The result supports the policy-first design this thesis adopts. The best-performing branch is not full world-model authority. It is the branch that lets the world model rank policy-generated candidates while still retaining the policy's own preferred candidate when that candidate is best.

The broader result is not that every world model helps. DexWM is a valid negative/null result for the evaluated fixed-budget route. LeWM visual rollout did not pass the causal diagnostic required for a semantic rollout claim, while LeWM action-prior scoring provides a secondary action-regularity result. DINO-WM and JEPA-WM were implemented as official-checkpoint exploratory routes but did not produce clean validation evidence for a held-out test claim. The ACT evaluation is null because the sampled candidate pool collapses numerically. These outcomes define when this guard framework is reliable and when it is not.

8.2 Methodological contribution

The thesis contributes a reproducible evaluation protocol for separating causal world-model guidance from misleading shortcuts. The final protocol fixes the test windows, uses frozen candidate pools, blocks future-goal access, uses training-only support, and keeps branch authority explicit. It also reports negative or no-go outcomes instead of retuning after test exposure.

This protocol changed the scientific interpretation of the project. Earlier historical results appeared stronger, but some relied on transductive support, oracle access, inconsistent candidate generation, or incomplete branch alignment. The final evidence is smaller in headline magnitude but stronger as academic evidence.

8.3 Reproducibility lesson

A secondary conclusion concerns reproducibility. The study benefited from open research releases, but it also exposed the difference between open code and complete reproducible evidence. DexWM is the clearest case. The architecture and training recipe are public, but the pretrained checkpoint for the reported model was not available. Repeating the full EgoDex+DROID pretraining recipe would have required substantial data onboarding, storage, preprocessing, and compute, and would have moved outside the fairness contract of this thesis.

The conclusion is not that code releases are unimportant. They are essential for independent research and for avoiding a field where only large organisations can inspect or build on frontier systems. The more careful point is that model papers should make checkpoint availability and reproduction requirements clear. When a published result depends on pretraining that most researchers cannot repeat, downstream work must treat the missing checkpoint as a claim boundary.

8.4 Scope of conclusions

The conclusions concern offline wrist-trajectory prediction on EgoDex. They do not claim closed-loop robot success, physical task completion, or solved dexterous manipulation. The benchmark measures whether a selected candidate is closer to the demonstrated wrist motion than the policy-only candidate.

Within that scope, the conclusion is clear. World-model scoring can improve inference-time action selection when the candidate pool is diverse and the scorer has meaningful action-conditioned representations. It does not help merely by being added to the system.

8.5 Future Work

The most immediate next step is online evaluation. A simulator or robot study is needed to test whether the offline wrist-error improvements translate into task success.

A second direction is candidate generation. The ACT collapse shows that this selection mechanism needs diverse candidate actions. Future work should improve policy sampling or use a candidate generator whose diversity can be measured before world-model scoring.

A third direction is stronger causal LeWM development. Future LeWM work should separate visual rollout evidence from action-prior evidence from the start, using

validation-only development and a fresh untouched test resource.

A fourth direction is full reproduction of close related systems such as DiWA. The infrastructure prepared in this thesis can support such work if official policy checkpoints, normalisation statistics, and evaluation metadata become available.

A fifth direction is resolving the Q1 inversion identified in Section 6.3. The selector measurably performs worse than not selecting at all on the fifth of windows where the policy's own uncorrected output is already most accurate, and the score-gap abstention mechanism tested in this thesis cannot detect that regime because the gap carries no information about baseline quality. A more promising direction is a gate conditioned on an estimate of how good candidate 0 already is, rather than on how confident the scorer is between its top candidates, since it is baseline quality, not scorer confidence, that this thesis's analysis identifies as the variable that actually separates the regime where selection helps from the regime where it hurts. Such a study would need its own held-out validation split to develop and calibrate the gate without leaking into the test set used here. Resolving it would remove the one remaining reason to withhold world-model guidance on any subset of inputs, letting a deployment use the selector's aggregate gain across the full input distribution instead of only on the harder cases where this thesis's evidence supports it.

Finally, world-model guidance could be distilled back into a policy. If the world model reliably identifies better candidates, accepted candidates could become a teacher signal for policy retraining. That would move this thesis's approach from test-time selection toward training-time policy improvement.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[2] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and Autonomous Systems*, vol. 57, no. 5, pp. 469–483, 2009. DOI: 10.1016/j.robot.2008.10.024

[3] A. Y. Ng and S. J. Russell, "Algorithms for inverse reinforcement learning," in *Proceedings of the Seventeenth International Conference on Machine Learning*, Morgan Kaufmann, 2000, pp. 663–670.

[4] M. Jeannerod, "The timing of natural prehension movements," *Journal of Motor Behavior*, vol. 16, no. 3, pp. 235–254, 1984. DOI: 10.1080/00222895.1984.10735319

[5] M. Jeannerod, M. A. Arbib, G. Rizzolatti, and H. Sakata, "Grasping objects: The cortical mechanisms of visuomotor transformation," *Trends in Neurosciences*, vol. 18, no. 7, pp. 314–320, 1995. DOI: 10.1016/0166-2236(95)93921-J

[6] M. Santello, M. Flanders, and J. F. Soechting, "Postural hand synergies for tool use," *Journal of Neuroscience*, vol. 18, no. 23, pp. 10 105–10 115, 1998. DOI: 10.1523/JNEUROSCI.18-23-10105.1998

[7] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15, 2011.

[8] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, *Learning fine-grained bimanual manipulation with low-cost hardware*, Apr. 2023. DOI: 10.48550/arXiv.2304.13705 arXiv: 2304.13705 [cs].

[9] C. Chi et al., *Diffusion policy: Visuomotor policy learning via action diffusion*, Mar. 2024. DOI: 10.48550/arXiv.2303.04137 arXiv: 2303.04137 [cs].

[10] Y. LeCun, *A path towards autonomous machine intelligence, version 0.9.2*, OpenReview preprint, 2022. [Online]. Available: https://openreview.net/forum?id=BZ5a1r-kVsf

[11] Y. LeCun, *Interview on the unsupervised learning podcast with jacob effron*, Podcast interview, Redpoint Ventures, May 2026. [Online]. Available: https://unsupervised-learning.simplecast.com/

[12] A. L. Chandra, I. Nematollahi, C. Huang, T. Welschehold, W. Burgard, and A. Valada, "DiWA: Diffusion policy adaptation with world models," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, PMLR, 2025, pp. 3378–3400. arXiv: 2508.03645 [cs].

[13] IBM Developer, *Models for machine learning*, https://developer.ibm.com/articles/cc-models-machine-learning/, 2017. Accessed: May 14, 2026.

[14] M. Assran et al., "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv: 2301.08243 [cs].

[15] Z. Ding, "Imitation learning," in *Deep Reinforcement Learning: Fundamentals, Research and Applications*, H. Dong, Z. Ding, and S. Zhang, Eds. Singapore: Springer Singapore, 2020, pp. 273–306, ISBN: 978-981-15-4095-0. DOI: 10.1007/978-981-15-4095-0_8 [Online]. Available: https://doi.org/10.1007/978-981-15-4095-0_8

[16] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.

[17] Omitram, *Left-hand vs. right-hand coordinate systems: The 3d developer's guide - omitram — omitram.com*, https://omitram.com/3d-coordinate-systems-left-vs-right-hand/, 2024.

[18] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5745–5753. DOI: 10.1109/CVPR.2019.00589

[19] N. Grossmann, E. Gröller, and M. Waldner, "Concept splatters: Exploration of latent spaces based on human interpretable concepts," *Computers & Graphics*, vol. 105, pp. 73–84, 2022, ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2022.04.013 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0097849322000656

[20] A. Dawid and Y. LeCun, "Introduction to latent variable energy-based models: A path towards autonomous machine intelligence," *Journal of Statistical Mechanics: Theory and Experiment*, 2023. arXiv: 2306.02572 [cs].

[21] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, 2013. DOI: 10.48550/arXiv.1312.6114 arXiv: 1312.6114 [stat.ML].

[22] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[23] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. DOI: 10.48550/arXiv.1706.03762 arXiv: 1706.03762 [cs.CL].

[24] M. Assran et al., *V-JEPA 2: Self-supervised video models enable understanding, prediction and planning*, Jun. 2025. DOI: 10.48550/arXiv.2506.09985 arXiv: 2506.09985 [cs].

[25] R. G. Goswami et al., *World models can leverage human videos for dexterous manipulation*, Dec. 2025. DOI: 10.48550/arXiv.2512.13644 arXiv: 2512.13644 [cs].

[26] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, *Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels*, Verified from local PDF metadata and first page, Mar. 2026. arXiv: 2603.19312 [cs.LG].

[27] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, *DINO-WM: World models on pre-trained visual features enable zero-shot planning*, 2024. arXiv: 2411.04983 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2411.04983

[28] X. Li et al., *Evaluating real-world robot manipulation policies in simulation*, CoRL 2024, 2024. arXiv: 2405.05941 [cs].

[29] Y. Li, Y. Zhu, J. Wen, C. Shen, and Y. Xu, *WorldEval: World model as real-world robot policies evaluator*, 2025. arXiv: 2505.19017 [cs].

[30] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating sentences from a continuous space," *arXiv preprint arXiv:1511.06349*, 2016. arXiv: 1511.06349.

[31] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, *Improving variational inference with inverse autoregressive flow*, 2016. arXiv: 1606.04934 [cs.LG].

[32] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, *Film: Visual reasoning with a general conditioning layer*, 2017. arXiv: 1709.07871 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1709.07871

[33] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov, *Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning*, 2024. arXiv: 2412.12953 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2412.12953

[34] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, *Consistency policy: Accelerated visuomotor policies via consistency distillation*, 2024. arXiv: 2405.07503 [cs.RO].

[35] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, et al., π_0*: A vision-language-action flow model for general robot control*, 2024. arXiv: 2410.24164 [cs.LG].

[36] T. Z. Zhao et al., *ALOHA Unleashed: A simple recipe for robot dexterity*, 2024. arXiv: 2410.13126 [cs.RO].

[37] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, *Behavior generation with latent actions*, 2024. arXiv: 2403.03181 [cs.LG].

[38] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, *Mastering diverse domains through world models*, Nature, 2025, 2023. arXiv: 2301.04104 [cs].

[39] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, et al., *RT-2: Vision-language-action models transfer web knowledge to robotic control*, 2023. arXiv: 2307.15818 [cs.RO].

[40] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, et al., *OpenVLA: An open-source vision-language-action model*, 2024. arXiv: 2406.09246 [cs.RO].

[41] Welch Labs, *Yann LeCun's $1B bet against LLMs*, https://www.youtube.com/watch?v=kYkIdXwW2AE, Video essay; accessed July 2026, 2026.

[42] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, "CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022. DOI: 10.1109/LRA.2022.3180108

[43] J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang, *WorldGym: World model as an environment for policy evaluation*, Sep. 2025. DOI: 10.48550/arXiv.2506.00613 arXiv: 2506.00613 [cs].

[44] Q. Bu et al., *Closed-loop visuomotor control with generative expectation for robotic manipulation*, Oct. 2024. DOI: 10.48550/arXiv.2409.09016 arXiv: 2409.09016 [cs].

[45] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Conference on Robot Learning (CoRL)*, 2022. arXiv: 2203.12601 [cs].

[46] A. Brohan et al., *RT-1: Robotics transformer for real-world control at scale*, 2022. arXiv: 2212.06817 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2212.06817

[47] N. Lambert, B. Amos, O. Yadan, and R. Calandra, *Objective mismatch in model-based reinforcement learning*, 2020. arXiv: 2002.04523 [cs.LG].

[48] A. Hu et al., *GAIA-1: A generative world model for autonomous driving*, 2023. arXiv: 2309.17080 [cs.CV].

[49] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, et al., *Genie: Generative interactive environments*, 2024. arXiv: 2402.15391 [cs.LG].

[50] N. Tsagkas, A. Sochopoulos, D. Danier, C. X. Lu, and O. Mac Aodha, *When pre-trained visual representations fall short: Limitations in visuo-motor robot learning*, 2025. arXiv: 2502.03270 [cs].

[51] Meta AI, *JEPA-WMs: Official JEPA world model checkpoints*, https://huggingface.co/facebook/jepa-wms, Model release; DROID world-model checkpoint, accessed July 2026.

[52] W. Huang et al., *Pointworld: Scaling 3d world models for in-the-wild robotic manipulation*, 2026. arXiv: 2601.03782 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2601.03782

[53] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang, *EgoDex: Learning dexterous manipulation from large-scale egocentric video*, ICLR 2026, 2025. DOI: 10.48550/arXiv.2505.11709 arXiv: 2505.11709 [cs].

[54] University of Malta, *M.AI.ESTRO: Mixed reality AI-enhanced skill transfer and robotic optimisation*, https://www.um.edu.mt/newspoint/news/2025/05/maiestro-mixed-reality-ai-enhanced-skill-transfer-robotic-optimisation, University of Malta newspoint announcement, 2025.

[55] J. Song, C. Meng, and S. Ermon, *Denoising diffusion implicit models*, Oct. 2020. DOI: 10.48550/arXiv.2010.02502 arXiv: 2010.02502 [cs.LG].

[56] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap* (Monographs on Statistics and Applied Probability). Chapman and Hall/CRC, 1993, vol. 57, ISBN: 978-0-412-04231-7.

[57] A. C. Cameron, J. B. Gelbach, and D. L. Miller, "Bootstrap-based improvements for inference with clustered errors," *The Review of Economics and Statistics*, vol. 90, no. 3, pp. 414–427, 2008. DOI: 10.1162/rest.90.3.414

[58] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[59] H. Fu, C. Li, X. Liu, J. Gao, A. Celikyilmaz, and L. Carin, "Cyclical annealing schedule: A simple approach to mitigating KL vanishing," *arXiv preprint arXiv:1903.10145*, 2019. arXiv: 1903.10145.

[60] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.

[61] N. Hansen, H. Su, and X. Wang, “TD-MPC2: Scalable, robust world models for continuous control,” in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.16828 [cs].

[62] W. Peebles and S. Xie, *Scalable diffusion models with transformers*, 2023. arXiv: 2212.09748 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2212.09748

[63] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

Appendix A Auxiliary Implemented Modules

This appendix describes the two auxiliary modules that are fully implemented in the benchmark stack but were excluded from the final fair benchmark. The semantic alignment pipeline and the object grounding stage each produce useful artefacts and remain integrated with the broader system. Their absence from the main results chapters reflects a deliberate methodological choice rather than a finding about their utility or correctness.

A.1 Rationale for exclusion from the main benchmark

The canonical benchmark evaluates two world model families under a shared input contract: the same raw RGB observation frame and the same bimanual wrist state sequence are provided to both V-JEPA2-AC and LeWM. No additional conditioning is admitted on either side.

Semantic alignment artefacts (captions, tags, text embeddings) and object grounding artefacts (bounding boxes, segmentation masks) could in principle be incorporated into the V-JEPA2-AC scoring pipeline through the semantic extraction path. The LeWM architecture, however, has no equivalent semantic conditioning module. Including semantic or grounding conditioning for V-JEPA2-AC alone would change the nature of the comparison from a world model family comparison under equal information to a mixed comparison that conflates world model architecture with access to additional input channels. The two scientific questions are distinct, and the thesis addresses only the former.

The exclusion applies symmetrically across all five evaluated branch instances. Both the V-JEPA2-AC family and the LeWM family run without semantic or grounding inputs in the final benchmark. Future work under a symmetric design, in which both families receive equivalent semantic conditioning, could isolate the effect of this information channel independently of the model family comparison.

A.2 Semantic alignment pipeline

A.2.1 Purpose and outputs

The semantic alignment stage converts indexed EgoDex windows into structured semantic artefacts for downstream conditioning and inspection. For each window, the pipeline produces:

- A natural language caption describing the observable manipulation activity in the selected frames.
- Structured tags organised into categories, including the contact object, secondary objects, and observed actions.
- A canonical semantic text string constructed from the structured tags, suitable for embedding.
- A dense text embedding vector produced by a configurable embedding backend.

The primary downstream use is to provide a semantically meaningful conditioning vector that can be concatenated with or appended to the visual latent representations used by the world model scoring functions. In the current implementation, this conditioning path is available but is not activated in the canonical benchmark.

A.2.2 Visual language model selection

The VLM component was evaluated across several model and embedding combinations before the operational configuration was stabilised. The semantic model comparison table records these evaluations and their relative quality on caption coherence and tag precision. The operationally selected configuration uses `Qwen/Qwen3-VL-2B-Instruct` as the visual language model for caption and tag generation, with the `google/siglip-so400m-patch14-384` model as the default text embedding backend (`SEMANTIC_EMBED_BACKEND=siglip_text`).

The Qwen3-VL-2B-Instruct model was selected for practical reasons: it is compact enough to run on a single GPU alongside other pipeline stages, it produces grammatically coherent captions from single-frame inputs, and its structured tag output is consistent enough to support deterministic canonical text construction. The SigLIP embedding backend was selected because it provides a joint image-text embedding space that is directly compatible with the V-JEPA2-AC semantic scoring path, allowing future similarity computations between caption embeddings and visual latents without an additional projection layer.

OpenCLIP and VLM-text hidden-state backends are also implemented and configurable. The SigLIP-backed Qwen3-VL run represents the strongest row in the comparison table for the current stack configuration.

A.2.3 Critic and retry loop

A key engineering feature of the semantic pipeline is an integrated critic and retry loop. The naive approach of captioning once and trusting the output produces unreliable

results on EgoDex frames, which are first-person egocentric views with partially occluded hands, cluttered surfaces, and rapidly changing visual content. The critic module scores the resulting caption against the input frames using a configurable similarity measure. If the score falls below a threshold or plateaus across retries, the pipeline either retries the VLM call with adjusted prompting parameters or marks the row as `semantic_failure=true` and writes it with a failure flag.

This design means that the semantic stage produces a complete, aligned output for every window in the index, with explicit quality flags rather than silently dropping or corrupting rows on difficult windows. The failure-flagged rows remain usable for inspection and can be filtered before any downstream conditioning step.

The implemented retry logic is:

1. Score the caption against the sampled frame using the configured score mode.
2. If the score exceeds the acceptance threshold, commit the row.
3. If the score is below threshold, retry with a revised prompt up to the configured maximum number of retries.
4. If all retries fail or the score plateaus without improvement, commit the row with `semantic_failure=true`.

A.2.4 Multi-GPU sharding and output artefacts

The semantic pipeline supports multi-GPU sharded execution through a shard supervisor (`semantic_shard_supervisor.py`) and a deterministic shard merge step (`semantic_merge_shards.py`). This is necessary because captioning and embedding 7,607 test windows or tens of thousands of training windows at VLM inference speeds is infeasible on a single GPU within a reasonable run budget.

The canonical merged output layout is:

```
/runs/w-act/egodex/semantic/<split>/<model_id>/<build_id>/
    embeddings.npy
    window_ids.txt
    meta.jsonl
    tags.jsonl
    captions.jsonl
    manifest.json
    index.npz
```

During a multi-GPU run, intermediate shard directories (`shard_0/`, `shard_1/`, and so on) accumulate partial outputs. The merge step validates row counts before

producing the canonical root artefact set, ensuring that partial or interrupted shard outputs are not silently used as complete builds. The manifest records the input fingerprint, allowing downstream stages to detect cache invalidation if the source windows file is replaced.

A.2.5 Integration with the broader pipeline

The semantic stage is positioned between the data indexing stage and the object grounding stage. The OPE stage (Section A.3) reads semantic manifests to obtain the canonical window ordering and to construct object prompts from the semantic tags produced by the VLM. This tight coupling is intentional: it ensures that semantic and grounding artefacts are always aligned to the same window index, preventing silent join drift if the windows file is regenerated.

The entrypoint for the semantic stage is `src/wact/cli/semantic_build_index.py`, with orchestration handled by `scripts/semantic.sh` or the stage-runner wrapper `scripts/run_stage.sh semantic`. The notebook companion `notebooks/semantic_aligner.ipynb` provides interactive inspection of caption quality, tag distributions, and embedding nearest-neighbour retrievals.

A.3 Object grounding and mask generation

A.3.1 Scope and current implementation

The object grounding stage takes semantic artefacts from the previous stage and produces spatially grounded detection and segmentation outputs for each window. The current implementation provides two-dimensional bounding-box detection and box-conditioned mask generation. It does not implement canonical six degree-of-freedom pose estimation, FoundationPose-style geometry reconstruction, or object identity tracking across frames. The name “object pose estimation” in the codebase reflects the intended scope of a future extension rather than the current capability.

For each window, the implemented pipeline:

1. Reads the semantic row for the window from the upstream semantic build.
2. Samples one or more frames from the window according to the configured frame sampling strategy.
3. Constructs object prompts from the structured semantic tags.

4. Runs the configured detector to produce bounding boxes.
5. Runs the configured masker on each detected box to produce segmentation masks.
6. Writes aligned detection and mask artefacts alongside a progress checkpoint.

A.3.2 Prompt construction

Object prompts are constructed from semantic tags using a priority scheme implemented in `src/wact/ope/prompting.py`. The logic prioritises the `contact_object` tag, which the VLM assigns to the primary object being actively manipulated, and then appends deduplicated entries from the `objects` tag list. Generic labels such as "object", "thing", "tool", and "hand" are filtered before the prompt is passed to the detector, as these terms produce unreliable or overspecific detections in the grounding model. When the VLM returns no object tags, the prompt construction falls back to a task-derived hint derived from the EgoDex task label.

This is category-level prompting rather than instance-level prompting. The detector does not distinguish between two instances of the same object class, and the resulting masks identify regions containing the prompted category rather than a specific tracked object instance.

A.3.3 Detection and segmentation backends

The production backends, configured through `configs/stage_defaults.env` and accessible via the stage-runner, are:

- Detector: `IDEA-Research/grounding-dino-base` (Grounding DINO), a zero-shot open-vocabulary detector that accepts free-text prompts.
- Masker: `facebook/sam2-hiera-large` (Segment Anything Model 2), a promptable segmentation model that produces high-quality binary masks from bounding box inputs.

The combination of Grounding DINO for detection and SAM2 for box-conditioned segmentation is a well-established zero-shot grounding pipeline. It does not require object-specific training data from EgoDex and generalises across the 194 task categories in the dataset without fine-tuning.

The bare `scripts/ope.sh` script defaults to dummy backends for development and debugging. Production runs must use the stage-runner or explicitly override the backend flags to select the real Grounding DINO and SAM2 models.

A.3.4 Output artefacts and alignment

The canonical output layout is:

```
/runs/w-act/egodex/pose/<split>/<pipeline_id>/<build_id>/
    masks.jsonl
    detections.jsonl
    window_ids.txt
    errors.jsonl
    progress.json
    manifest.json
```

The `masks.jsonl` file is the primary downstream artefact: it contains a row per window with the detected object categories, bounding box coordinates, and run-length-encoded or polygon-format segmentation masks. The `detections.jsonl` file records the raw detection outputs before masking and is retained for audit purposes. The `progress.json` file supports resumable execution through the shard supervisor.

Alignment between the OPE output and upstream artefacts is enforced by inheriting the `window_ids.txt` ordering from the semantic build. The OPE stage reads the semantic manifest to resolve the windows pickle that was used to produce the semantic index, ensuring that detection and mask rows correspond to the same window boundaries used at every earlier pipeline stage.

A.3.5 Known limitations

The current implementation has several limitations that are relevant to any future use of OPE outputs for manipulation conditioning:

1. The 2D masks identify spatial regions in the image frame but do not provide three-dimensional geometry information. Object depth, relative pose, or spatial relationship between manipulated objects are not recoverable from the mask output alone.

2. Small or visually similar objects can confuse the Grounding DINO detector, particularly in the cluttered egocentric field of view typical of EgoDex demonstrations. When two similar objects are present, the detector may merge them or select the wrong instance.

3. There is no temporal tracking across frames within a window. Each sampled frame is processed independently, and there is no mechanism to maintain object identity across the prediction horizon.

4. The zero-shot grounding approach depends on the quality of the upstream semantic tag output. A missed or misidentified contact object in the semantic stage will propagate to an incorrect or absent detection in the OPE stage.

These limitations are consistent with the current scope of two-dimensional grounding and mask generation. They would need to be addressed before OPE outputs could serve as reliable conditioning for world model scoring in a deployed system.

A.4 Future use under a symmetric design

The exclusion of semantics and object grounding from the final benchmark is a fairness decision, not a statement that these modules lack value. Both pipelines are implemented, tested on the training split, and produce aligned artefacts that are stored alongside the canonical benchmark outputs.

The most natural future experiment would introduce semantic conditioning symmetrically across both world model families. This would require either adapting the LeWM architecture to accept a text embedding as an additional conditioning channel, or training a new LeWM variant with semantic inputs from the outset. Under a symmetric design, the contribution of semantic conditioning to world model scoring quality could be measured independently of the model family comparison, and the effect of caption quality, embedding backend selection, and object grounding granularity could each be evaluated in isolation.

Similarly, the object grounding masks could be used to produce spatially localised visual features by masking the RGB input before the visual backbone processes it. Whether this localisation improves world model guidance in the manipulation setting, or whether the pretrained backbone's attention mechanism already implicitly grounds relevant objects without explicit masking, is an open empirical question.

[FIGURE PLACEHOLDER: semantic pipeline overview — input window to VLM to caption plus tags to embedding backend to output artefacts; critic/retry loop shown as a feedback arc between caption and score threshold]

[FIGURE PLACEHOLDER: object grounding example — sampled EgoDex frame with Grounding DINO bounding box and SAM2 mask overlaid on detected contact object; prompt text shown]

A.5 Official-checkpoint world models: DINO-WM and JEPA-WM

DINO-WM and JEPA-WM entered the benchmark as exploratory official-checkpoint routes and are summarised only briefly in Chapter 3; this section records their literature detail.

DINO-WM performs latent world modelling over a frozen DINOv2 backbone that encodes each frame into a grid of spatial patch tokens, with a predictor advancing the patch-level latent state under action conditioning [27]. Like DexWM, it inherits the spatial resolution and generalisation benefits of DINOv2's patch-level representations, at substantially higher computational cost than single-token designs such as LeWM: the higher-dimensional latent state makes both training and planning slower, which is the trade-off LeWM targets with its reported 48$\times$ planning speedup [26].

JEPA-WM refers to the officially released JEPA world-model checkpoints, including a model trained on the DROID robot manipulation dataset [51]. The release provides pretrained latent predictors, but their native action interface assumes a single-arm robot action space. Applying either model to the 18-dimensional bimanual wrist space of this benchmark therefore requires an action adapter, and results obtained through that adapter are reported as validation-level exploratory evidence rather than headline comparisons.

Both models passed load, extraction, and scorer smoke tests, and were then carried through a bounded validation pilot rather than the held-out evaluation used for the main comparison. Default DINO-WM and JEPA-WM improved their latent support scores but did not give a clean improvement on immediate trajectory metrics. Validation-only reweighting recovered limited loss or rotation signals, especially for DINO-WM, but translation and endpoint translation were neutral or worse. No held-out test claim is made from this route; the pilot is retained here as an implementation record rather than promoted to Chapter 6.

Appendix B Benchmark Evolution and Validity Record

This appendix documents the development trajectory of the benchmark and the validity concerns that were identified and resolved before the final canonical results package was established. Its purpose is to provide an auditable record of the decisions and corrections that shaped the final evaluation surface, and to explain why certain earlier result sets were superseded or rejected.

The record is presented in roughly chronological order of discovery. Each section identifies the concern, describes its effect on the results as reported at the time, and explains the correction applied.

B.1 The three benchmark eras

The dissertation results passed through three distinct evaluation eras before the canonical package was established. Understanding the transition between eras is necessary for interpreting references to earlier result numbers in project documentation.

Era 1: Bounded pilot slices. The earliest quantitative comparisons were conducted on small, handpicked windows selected to represent the diversity of EgoDex task categories. These were useful for rapid controller iteration and for identifying which architectural choices were clearly harmful, but they could not support generalisable claims because the selection was not held-out from the development process. All Era 1 results were explicitly demoted from the thesis claims surface once full held-out evaluation became practical.

Era 2: March 2026 frozen three-branch bundle with limited LeWM coverage. The first evaluation to use the held-out test split was the March 2026 frozen package. This package provided valid Branch 1 and Branch 2 V-JEPA2-AC results, but its LeWM coverage was severely limited by a frame cap configuration error (described in Section B.2). The Branch 3 selection mode error (described in Section B.3) also meant that Branch 3 V-JEPA2-AC was not independently evaluated. Both defects required correction before the three-branch comparison could be treated as scientifically valid.

Era 3: Canonical final package (April 2026). The April 14, 2026 package achieved full 7,607-window coverage for the LeWM family after the frame cap fix. A further correction on April 18, 2026 re-ran Branch 3 V-JEPA2-AC with the correct `selection_mode=world_model_only` flag, producing the first genuine Branch 3 evaluation. The `train_part1_2_3` training split was used for all five branch instances. This package is the canonical results surface for all quantitative claims in the thesis.

B.2 LeWM frame cap defect

B.2.1 Discovery

The LeWM export pipeline requires a `max_frames_per_episode` parameter that controls how many frames from each EgoDex episode are included when building the training corpus. This parameter has a direct impact on the coverage of the resulting evaluation surface, because the guard can only evaluate windows whose constituent frames were included in the export.

During the March 2026 benchmark packaging, the frame cap was set to `max_frames_per_episode=256`. This value was not derived from the actual window length distribution of the benchmark window set; it was a development default carried over from early pipeline testing. When the benchmark results were inspected, the LeWM guard had produced predictions for only 160 of the 7,607 test windows, representing 2.1% of the intended evaluation surface.

The coverage failure was identified by comparing the count of LeWM guard output rows against the expected window count. The root cause was confirmed by recomputing the maximum frame index required to cover all windows in the benchmark window list and finding that the actual maximum was 5,280 frames, far above the 256-frame cap in use.

B.2.2 Effect on reported results

Any LeWM result reported from the March 2026 package is based on 160 windows, not 7,607. The 160 covered windows are not a random sample of the test split; they are the windows whose episodes happened to fall within the 256-frame cap. These windows are concentrated in episodes with early-episode actions and do not represent the full range of task categories or annotation confidence levels in the test split.

The practical effect is that the March 2026 LeWM numbers cannot be compared directly to Branch 1 or Branch 2 V-JEPA2-AC numbers, because the LeWM evaluation is on a different, non-representative slice of the test data. Any conclusion drawn from these numbers about the relative merit of the LeWM family would be confounded by the coverage difference.

B.2.3 Correction

The frame cap was recomputed by iterating over the benchmark window list, reading the episode frame indices for each window, and taking the maximum observed frame index across all windows. The resulting cap of 5,280 frames was applied to the export

rebuild. The LeWM model was not retrained; only the export was rebuilt with the corrected parameter. The guard was then re-run, achieving full 7,607-window coverage on the April 14, 2026 package.

The corrected LeWM numbers in the canonical package are therefore not comparable to the March 2026 LeWM numbers on a like-for-like basis. The March 2026 numbers are retained in project documentation for audit purposes but are not cited in any thesis claims.

B.3 Branch 3 selection mode defect

B.3.1 Discovery

The three-branch design requires Branch 3 to use `selection_mode=world_model_only`, meaning that only the world model's predicted latent distance ranks the candidates, without any contribution from the ACT policy's own scoring function. This is the experimental condition that tests whether the world model alone, without policy guidance in the selection step, can identify better trajectories.

After the April 14, 2026 package was assembled, the per-window pairwise comparison between Branch 3 V-JEPA2-AC and Branch 2 V-JEPA2-AC was computed as a validation check. The result was a 0.0% win rate for Branch 3 against Branch 2 across all 7,607 windows. That is, Branch 3 did not produce a single window outcome that differed from Branch 2.

This was immediately identified as inconsistent with any genuine difference in selection logic between the two branches. Inspection of the Branch 3 guard manifest revealed that the selection mode was recorded as `best_score`, not `world_model_only`. The chain evaluation script (`chain_eval_part1_2_3.sh`) had been written to launch both Branch 2 and Branch 3 guards, but the Branch 3 launch did not pass the `--selection-mode world_model_only` flag. Both guards therefore ran with `selection_mode=best_score` on the same frozen candidate pool, producing identical outputs.

B.3.2 Effect on reported results

Until the defect was corrected, the "Branch 3 V-JEPA2-AC" entry in the April 14 package was scientifically identical to Branch 2 V-JEPA2-AC. The three-branch comparison was effectively a two-branch comparison. Any conclusion about whether world-model-only selection outperforms hybrid selection under V-JEPA2-AC could not be drawn from this data.

The defect was non-obvious from the output metrics alone. The reported Branch 3 translation error (0.0354, identical to Branch 2) was not implausible; one might expect Branch 3 and Branch 2 to produce similar numbers if the world model's ranking correlates with the combined score's ranking. The defect was only detectable through pairwise comparison, which revealed the zero win rate that is impossible if the two branches genuinely differ.

B.3.3 Correction

The Branch 3 V-JEPA2-AC guard was re-run on April 18, 2026 with the correct flag:

```
--selection-mode world_model_only
--candidate-source frozen_manifest
--candidate-manifest-run-dir <B2_JEPA_run_dir>
```

Using the frozen candidate pool from Branch 2 ensured that the Branch 3 result reflects only the difference in selection logic, not any difference in the candidates available. Guard manifest verification after the run confirmed that all 7,607 windows were decided by the world model (`override_reason_counts: {world_model_only_best_non_act: 7607}`), confirming that the corrected selection mode was in effect for every window.

The corrected Branch 3 V-JEPA2-AC result (translation $L_2 = 0.0328$, 91.5 per cent win rate against Branch 1) is the first genuine evaluation of world-model-only selection in this benchmark.

B.4 Oracle goal leakage

B.4.1 Discovery

In early V-JEPA2-AC guard configurations, the world model scoring function was provided with the terminal latent state of the evaluation episode as a goal signal. The scoring function measured the predicted final latent distance to this goal and used it as part of the candidate ranking. This goal signal was derived from the ground-truth future frames of the demonstration, information that would not be available in a deployed system.

The oracle goal leak was identified during the fair comparison audit when it was noted that the V-JEPA2-AC guard configuration differed from the ACT-only baseline in the information it received at evaluation time. The ACT policy generates candidates from the current observation alone and has no access to future frames. Providing the world model with the ground-truth terminal latent created an asymmetry: the world

model scoring function was effectively guided toward candidates that reached the demonstrated endpoint, while the ACT policy's own ranking had no equivalent oracle signal.

B.4.2 Effect on reported results

Any result from a guard run using oracle goal conditioning overstates the benefit of world model guidance, because part of the improvement is attributable to the oracle signal rather than to the world model's intrinsic representation quality. The magnitude of the overstatement cannot be precisely determined from the available data, but it is substantial enough to make oracle-conditioned results non-comparable to policy-only results.

Early exploration of V-JEPA2-AC guidance showed results that were more impressive than the final canonical numbers. The oracle goal was a contributing factor. Removing the oracle signal and rerunning under the fair contract produced lower but more defensible numbers. The final canonical results (77.0% reduction in mean translation error) are therefore conservative relative to the earliest reported V-JEPA2-AC results.

B.4.3 Correction

The fair evaluation contract was established by setting `goal_source=none` for all branch evaluations. Under this setting, the world model scoring function does not receive any goal latent derived from future frames. The candidate ranking is based solely on the world model's predicted latent dynamics given the observed context and the proposed action chunk.

This correction was applied to all five branch instances in the canonical benchmark. The change from oracle goal to no goal required validating that the world model scoring function still produced meaningful candidate rankings without the terminal goal signal, which was confirmed by the final translation improvement results.

B.5 Goal metric invalidity

B.5.1 Discovery

The V-JEPA2-AC guard natively reports a metric called `goal_l2` alongside the translation error. In the original implementation, this metric stored different quantities in Branch 1 and Branches 2 and 3. In Branch 1, it stored the translation L_2 error for the

selected candidate. In Branches 2 and 3, it stored the JEPA latent-space distance between the predicted final state and the goal latent.

When `goal_source=none` was adopted as the fair evaluation contract, there was no longer a meaningful goal latent for Branches 2 and 3. The `goal_l2` metric for these branches consequently reported a latent distance of zero for every window, making the metric uninformative. Comparing `goal_l2` across branches was therefore comparing a real translation error in Branch 1 against a vacuous zero in Branches 2 and 3.

B.5.2 Correction

The `goal_l2` metric was replaced with `endpoint_translation_l2`, defined as the mean translation L_2 error computed over only the final quarter of the prediction window (prediction steps 12 through 16). This metric measures how accurately the predicted trajectory arrives at the correct final position, which is often more relevant to task success than early-window prediction accuracy. The endpoint metric is computed identically across all branch instances and is not affected by the presence or absence of a goal latent. It is included in the canonical benchmark shape metric tables reported in the results chapter.

B.6 LeWM CPU inference defect

B.6.1 Discovery

During the first full test-split guard run for LeWM after the frame cap fix, the estimated completion time for 7,607 windows was approximately 54 hours. This was implausibly slow relative to the expected GPU inference throughput for a 15-million-parameter model. Inspection of the guard inference loop revealed a hard-coded `.to("cpu")` call in the LeWM model forward pass that overrode any device specification passed at runtime. The model was running on CPU regardless of whether a CUDA device was available.

B.6.2 Correction

The hard-coded device assignment was replaced with a dynamic device detection that checks for CUDA availability and falls back to CPU only if no GPU is detected. After this correction, the LeWM guard ran at approximately 19.5 windows per second on a single GPU, completing the full 7,607-window test evaluation in roughly 6.5 minutes. The effective speedup was approximately 130-fold relative to the CPU-bound run.

This defect is worth recording because it illustrates a failure mode common in research code developed iteratively: a debugging convenience (forcing CPU to avoid CUDA memory errors during development) was never removed before production use. The correction required no changes to the model architecture or training, only to the device placement logic in the inference wrapper.

B.7 HDF5 handle management defect

B.7.1 Discovery

The data loading loop in an early version of the guard infrastructure opened and closed the HDF5 data file once per evaluation window. On a test set of 7,607 windows, this produced approximately 145,000 open-close cycles (one open and one close per window, plus frame-level re-opens within each window). The overhead from repeated file handle acquisition was a significant fraction of total evaluation wall time on large test splits.

B.7.2 Correction

The data loading implementation was refactored to open the HDF5 file once at the start of the evaluation run, hold the handle in memory, and close it only when the evaluation completes. This change reduced per-window I/O overhead to a small seek operation rather than a full open-close cycle and improved guard evaluation throughput noticeably on the full test split.

B.8 Guard resumability

B.8.1 Motivation

Full test-split guard runs covering 7,607 windows take several minutes on a GPU but can be interrupted by node preemption, out-of-memory errors, or external compute scheduler events. Without checkpointing, an interrupted run required restarting from window zero, discarding all partial results.

B.8.2 Implementation

A per-window JSONL checkpointing system was added to the guard infrastructure. After each window is evaluated, a row is appended to a checkpoint file in the output

directory. When a run is invoked with `--resume-dir` pointing to an existing output directory, the guard reads the checkpoint file, identifies which window IDs have already been completed, and skips them. The final manifest and aggregate metrics are computed from the complete set of per-window rows, including both the resumed and newly computed windows.

This design ensures that an interrupted run can be resumed without any manual intervention and without requiring the window evaluation order to be deterministic. The checkpoint is written atomically per row, so a crash mid-window produces at worst one incomplete row that the resume logic can detect and skip.

B.9 Rejected comparison inputs and superseded paths

B.9.1 Semantics and OPE as benchmark inputs

An early hypothesis held that incorporating semantic alignment and object grounding into the V-JEPA2-AC scoring function would produce the most informative world model guidance, and that this enriched V-JEPA2-AC configuration should be the primary benchmark representative for the V-JEPA2-AC family. This hypothesis was rejected for the reasons described in Appendix A and in Section 7.6 of the discussion chapter: the enriched configuration gives V-JEPA2-AC privileged inputs not available to LeWM, confounding the family comparison.

The decision to reject this path was not based on the performance of the enriched configuration. The lean RGB-plus-trajectory contract was adopted because it is the only contract that supports a scientifically unconfounded comparison between the two world model families.

B.9.2 Veto and threshold selectors

Before the current scoring function was stabilised, several candidate selection mechanisms based on explicit threshold vetos were implemented and evaluated. These included the `act_first_veto` design, which rejected world model candidates whose support distance exceeded a fixed threshold, and the `veto_m10` selector, which applied a margin-based veto to prevent the world model from selecting candidates that were far from the ACT policy's preferred candidate even if they scored well on the world model metric alone.

These mechanisms were useful for early exploration of the policy-guard interface and helped establish that naive world model selection could be dominated by retrieval artefacts when the candidate pool contained low-quality outliers. They were superseded by the structured scoring function in Branch 2 (`best_score`: combined

latent distance, action out-of-distribution penalty, and support distance term) and the clean world-model-only design in Branch 3. Neither the veto selectors nor the margin-based threshold approaches are used in the canonical benchmark.

B.9.3 Early Branch 3 shells

Several experimental Branch 3 designs preceded the final frozen representative. The first explicit world-model-only shell for LeWM was evaluated on the fair held-out slice and was found to be weaker than the ACT-only baseline on aggregate slice metrics, showing that world-model-only selection requires a world model of sufficient quality before it improves over the policy alone. Neutral-seed variants of Branch 3, which removed the ACT seed from the candidate pool, also collapsed on the fair slice, demonstrating that the bottleneck was the controller structure rather than any single parameter choice. These negative results were important for understanding the design space and for eventually arriving at a Branch 3 configuration that genuinely improves over Branch 2 in aggregate.

B.9.4 Conservative confidence gating

A further rejected path explored whether applying more conservative confidence gating to the LeWM guard, requiring higher override margins before the world model was permitted to override the policy candidate, could make LeWM competitive with V-JEPA2-AC on the shared benchmark. The bounded sweep showed that increasing the override margin made LeWM more conservative but did not produce a competitive replacement for V-JEPA2-AC on chunk-level comparison metrics. This was recorded as a useful negative result: the performance gap between the two world model families is not primarily explained by calibration of the override threshold, and therefore cannot be closed by threshold tuning alone.

B.10 Training exposure convergence study

Section 4.5.1 fixes a universal training-exposure budget of $E = 100{,}000$ effective items, applied uniformly across all model families and all three training splits. This budget was determined by a convergence study run prior to full training, rather than chosen arbitrarily.

Each of the seven implemented model families (ACT, Diffusion Policy, V-JEPA2-AC, V-JEPA2 without action conditioning, PointWorld, LeWM, and DexWM) was trained independently at six data scales, 1k, 5k, 10k, 25k, 50k, and 100k effective items, on a fixed stratified subset of `train_part1_2_3`. The two control families

(V-JEPA2 without action conditioning and PointWorld) were included in the sweep for completeness but excluded from the budget-selection decision, since neither is designed to improve with additional data. Table B.1 reports the validation loss at each scale for all seven families; each row uses a different training objective, so raw values are not comparable across rows, and the relevant signal is each family's own trend across scales. Bold entries mark the earliest scale at which a family's loss saturates (improvement below 0.5 % relative to the next scale).

Table B.1 Validation loss across six training scales for all seven model families. Bold marks each family's first scale of saturation; losses are objective-specific and not cross-family comparable.

Model	1k	5k	10k	25k	50k	100k
ACT	**0.1344**	0.1344	0.1292	0.1327	0.1305	0.1288
Diffusion Policy	10.861	5.216	4.251	3.108	2.566	2.138
DexWM	4.314	2.414	1.995	1.899	1.693	1.356
LeWM	0.335	0.209	0.165	0.145	**0.130**	0.130
V-JEPA2-AC	1.796	0.375	0.308	0.286	0.257	0.227
V-JEPA2 (no AC)	0.460	0.477	0.373	0.352	0.353	0.377
PointWorld	0.655	0.679	0.628	0.629	0.632	0.592

Two families saturate within the sweep: ACT is essentially flat from 1k onward (a 4 % improvement over a hundred-fold data increase), and LeWM plateaus between 50k and 100k (0.5 % improvement). V-JEPA2-AC, Diffusion Policy, and DexWM carry no bold entry because all three are still declining at 100k, as Figure B.1 shows. The two controls show no consistent downward trend, consistent with their respective proxy designs having no learnable relationship with additional data.

Table B.2 summarises the plateau budget implied by each primary family's trend. Given that Diffusion Policy, DexWM, and V-JEPA2-AC all continue to improve at 100k, the universal budget is set to $E = 100{,}000$ as a compute-constrained lower bound rather than a true saturation point. ACT and LeWM, which plateau earlier, receive the same 100k budget by design: the extra exposure cannot hurt them, and it ensures every family is compared under the most favourable common condition reachable within the compute envelope.

This budget is a compute-constrained lower bound rather than a true saturation point for all families. The performance gaps between families reported in the results chapter may partly reflect differing distances from their respective saturation budgets rather than intrinsic architectural differences; this limitation is carried forward to Section 4.7.

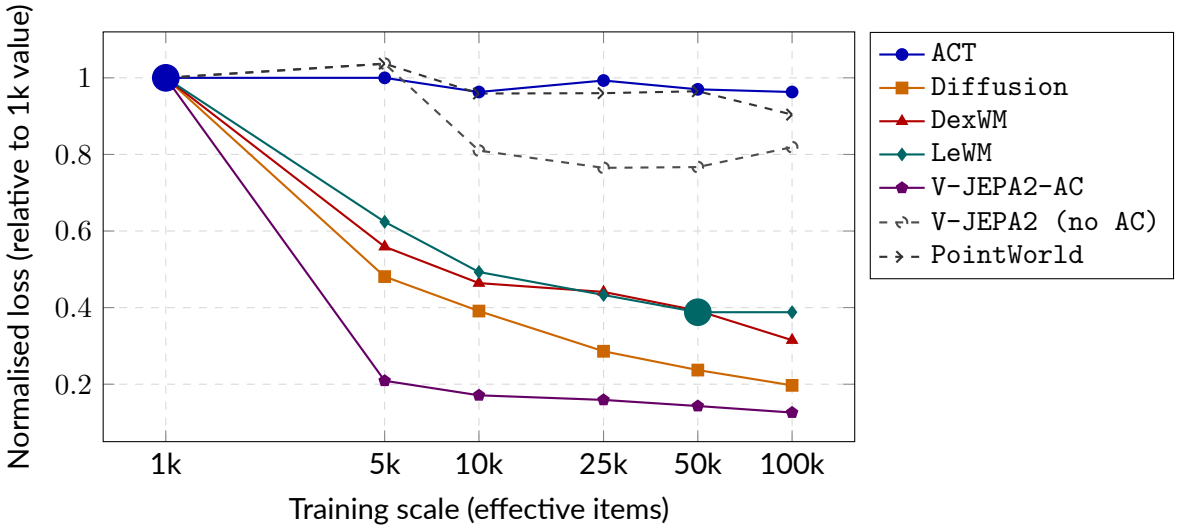


Figure B.1 Normalised validation loss (each family's value divided by its 1k loss) across six training scales. Solid lines are primary models; dashed lines are controls. Large filled circles mark saturation points matching the bold entries in Table B.1. Families with no marker are still declining at 100k.

Table B.2 Plateau budget per primary model family. The universal budget is $E = 100{,}000$ effective items.

Family	**Plateau budget**	**Evidence**
ACT	1k	Flat from 1k; 4% drop over 100× more data
LeWM	50k	50k→100k improvement of 0.5%; effectively converged
V-JEPA2-AC	>100k	11% drop at 50k→100k; still declining
Diffusion	>100k	17% drop at 50k→100k; still declining steeply
DexWM	>100k	20% drop at 50k→100k; still declining
Universal budget		$\boldsymbol{E = 100{,}000}$

B.11 Windowing strategy comparison: fixed-stride versus state-change-targeted

The canonical benchmark uses fixed-stride windowing (Strategy 1, Section 4.1.2) throughout. A state-change-targeted scheme (Strategy 2) was identified later, during pipeline iteration, as a way of correcting Strategy 1's under-sampling of high-motion segments. It was not adopted for the canonical benchmark, but a standalone comparison was run to check whether it would have been worth adopting: a parallel state-change-trained Diffusion Policy checkpoint, a matching support extract, and a test index of equal size to Strategy 1, scored under the same scorer, pool size, and evaluation contract.

Diffusion Policy with V-JEPA2-AC. Switching from Strategy 1 to Strategy 2 raised capture from 40.3% to 53.1% at candidate pool size $K = 8$, on a near-identical oracle ceiling, so the gain reflects better selection rather than additional headroom. A two-by-two crossing of Strategy 1 and Strategy 2 test windows against Strategy 1 and Strategy 2 support extracts isolated the source of the gain: 87% is attributable to the state-change windows and policy pairing itself, and 13% to the accompanying support extract.

Diffusion Policy with LeWM. Repeating the same comparison on the LeWM action-prior scorer reversed the sign: capture fell from 61.3% to 57.6% under Strategy 2.

Conclusion. State-change selection helps the V-JEPA2-AC cell and hurts the LeWM cell, so the effect is real and mechanistically located, but it is specific to which world model is scoring rather than a general property of the data-selection regime. A result that reverses sign on a second world model is not grounds to replace the study's canonical protocol, so Strategy 1 is retained throughout. State-change-aware sampling remains a promising direction for world models that benefit from it, worth testing across the full evaluation matrix in future work.

B.12 Training-split scaling: performance across the three-part corpus

Section 4.1.3 restricts the canonical benchmark to the first three training parts of the EgoDex release (`train_part1`, `train_part1_2`, `train_part1_2_3`) rather than the full five-part release. Each of the five primary models (Section 4.3, Section 4.4) was trained separately on each of the three cumulative splits at the same training-exposure budget (Section 4.5.1), so that the effect of additional unique training data, independent of

exposure, could be measured directly.

Table B.3 Held-out evaluation performance by training split, for each primary model. *Placeholder: values to be filled from the fair-training evaluation logs.*

Model	`train_part1`	`train_part1_2`	`train_part1_2_3`
ACT	–	–	–
Diffusion Policy	–	–	–
V-JEPA2-AC	–	–	–
LeWM	–	–	–
DexWM	–	–	–

Across the models measured, additional unique training data from `train_part1` through `train_part1_2_3` continued to improve held-out performance, motivating the decision to use the full three-part subset as the canonical training split. The remaining two parts of the official release were not incorporated: the marginal gain per additional part was already narrowing by the third part, and the storage, preprocessing, and shared-cluster GPU time required to extend the pattern across two further parts, for five models trained at a fixed exposure budget each, was judged not to be worth the return within the project's compute allocation.

B.13 Candidate pool size (K) sweep

Section 4.2 fixes the candidate pool size at $K = 5$ for the canonical benchmark, on compute-budget grounds rather than a literature precedent. That choice was tested rather than left as an assertion.

A common-random-number sweep was run over $K \in \{2, 3, 4, 5, 6, 8, 10, 12, 16, 20\}$, using nested candidate pools so that smaller pools are exact prefixes of larger ones, isolating the effect of pool size from any change in which candidates are available. Two findings result.

First, the ranking of world-model scorers is identical at every pool size tested, so no comparative conclusion drawn in this work is an artefact of fixing K at five.

Second, the margin itself grows without a knee, following a logarithmic law across the swept range, so there is no sufficiency threshold at which additional candidates stop helping. This means the pool size cannot be defended as the point at which candidate diversity becomes adequate, and it is not defended that way in the main text. Reported margins are specific to $K = 5$ and would be larger at a larger pool; the comparisons between scorers, which are what the research question turns on, are not.

K is therefore treated throughout as a compute-budget variable: it changes the size of the effect that is measured, but not the conclusion drawn from it.

Figure B.2 plots the reduction in translation error against K for all five DP-row scorers. For V-JEPA2-AC the error reduction rises from 3.46% at $K = 2$ to 9.62% at $K = 20$, following $2.64 \ln K + 1.99$ with $R^2 = 0.988$. Each doubling of the pool buys roughly two percentage points at twice the sampling cost. Random selection from the same pools was measured at 0.01%, confirming that the sampled candidates are exchangeable and that index 0 carries no advantage as a baseline. The entire margin is therefore attributable to ranking, not to pool composition.

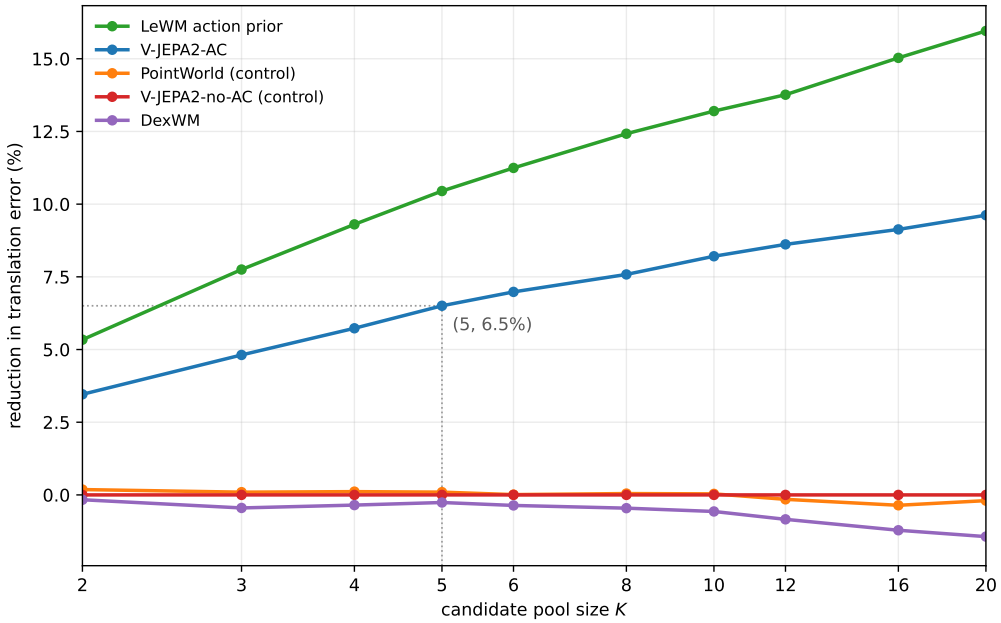


Figure B.2 Reduction in translation error against candidate pool size for the five world-model scorers on the Diffusion Policy row. All points are scored on nested pools, so differences along a curve reflect pool size alone. The vertical guide marks the operating point used throughout this dissertation.

The sweep covers the Diffusion Policy row only. ACT candidates are produced by a planner that queries the world model during search rather than by independent sampling, so pool size there denotes the number of returned elites rather than a proposal count, and the two cannot be placed on a common axis (Section 4.2). Pool-size invariance is therefore established for the Diffusion Policy row and assumed, not demonstrated, for the ACT row.

B.14 Summary of validity status

Table B.4 summarises the seven validity concerns identified during the benchmark development cycle and the resolution status of each at the time the canonical package

was assembled.

Table B.4 Summary of benchmark validity concerns and their resolution status in the canonical final package.

#	Concern	Severity	Status in canonical package
1	Branch 3 V-JEPA2-AC identity defect (`selection_mode` bug)	Critical	Resolved: re-run April 18, 2026
2	`translation_l2` not paper-backed as primary metric	Moderate	Documented: framed as offline diagnostic proxy
3	Shape metrics suppressed from reported summary	Moderate	Resolved: shape metrics added to results chapter
4	No per-task win rate breakdown	Moderate	Partially resolved: aggregate confirmed broad; worst-5 reported
5	Wrist-only evaluation scope	Low–Moderate	Acknowledged design constraint; documented in limitations
6	Confidence scores unused in evaluation	Low	Resolved: stratification analysis added to results
7	Offline-to-online gap	Moderate	Acknowledged: explicitly bounded in scope section

Concerns 1, 3, and 6 were resolved by changes to the benchmark artefacts or analysis code. Concern 2 is addressed by consistent framing throughout the thesis: translation L_2 is positioned as an offline diagnostic proxy rather than a task success predictor, and the connection to deployed outcomes is explicitly left as future work. Concern 4 is partially addressed by the per-task win rate distribution reported in the results chapter, which confirms that improvement is broad and not driven by a small number of tasks. Concerns 5 and 7 are acknowledged design constraints that are transparent in the scope of conclusions and limitations sections.

The final canonical results are drawn from a benchmark in which all critical and moderate concerns have been either resolved or explicitly bounded. No result cited in the thesis claims chapter relies on a defective or uncorrected evaluation artefact.

Appendix C Auxiliary Implementation Modules

Two modules are fully implemented in the repository but were excluded from the primary benchmark evaluation. They are documented here as reference material for any continuation of this line of work toward a general framework for semantic feature enrichment, policy candidate generation, and world model selection.

Semantic alignment. This stage converts indexed windows into captions, structured tags, and text embeddings. A Vision Language Model (VLM) generates free-form descriptions and task-relevant categorical tags for each window; a separate embedding model maps the text to a shared representation space. A critic-and-retry loop rejects low-quality outputs and requests regenerated captions, providing a quality filter before downstream use.

Object grounding and mask generation. This stage consumes semantic object prompts from the alignment stage and produces bounding box detections and segmentation masks for identified objects. The current implementation provides pixel-space localisation; SE(3) object pose estimation is not yet included.

Both modules were excluded from the primary benchmark because a shared input contract across all five world model families could not be maintained: the semantic and grounding outputs could not be wired into the frozen DINOv2 and ViT-Large encoder paths used by DexWM, V-JEPA2-AC, and related families without violating the evaluation fairness requirement (Section 4.5.2). The initial intuition motivating these modules — that frozen encoder pre-extraction followed by lightweight head training is productive — has since been validated by the DexWM and DINOv2-based world model architectures, each of which adopts an analogous two-phase structure. Extending this framework to incorporate semantic and spatial grounding signals remains a natural direction for future work.

Appendix D Notes for Future Revision

This appendix collects scope notes identifying parts of the thesis that would require revision if certain extensions were carried out after submission. They are gathered here to keep the main chapters clean while preserving a complete record for any follow-on work.

D.1 Strategy 2 comparative evaluation

All benchmark results in this thesis use the fixed-stride windowing contract (Strategy 1, `obs16_pred16_s128`) exclusively. The state-change-targeted alternative (Strategy 2, Section 4.1.2) is fully implemented and has been used for qualitative development inspection, but a systematic Strategy 1 vs. Strategy 2 comparison has not been conducted and is noted as a natural future direction.

If a full Strategy 2 evaluation were completed and the two contracts compared as co-primary evaluation designs, the following sections would require revision:

- **Section 4.1.2** (index construction): restore the Strategy 1 / Strategy 2 bold-paragraph pattern. Add a paragraph describing the state-change-targeted index in full (peak detection, task-stratified sampling, one window per episode with a fixed seed, no temporal overlap). Clarify that both strategies enforce the shared-window invariant independently.
- **Section 4.1.3** (training splits): extend Table 4.1 to include Strategy 2 window counts per split and task; update the test-index description to distinguish the two contracts.
- **Section 4.6** (evaluation metrics): add explicit strategy labels to all result tables and figures; add a dedicated analysis subsection for cross-strategy comparison if results differ materially.
- **Results chapter**: all quantitative results would need split-by-strategy labelling to make clear which sampling contract each row corresponds to.